

Lab 1 – Exercise 1: nội dung trò chuyện

Bạn

HANDS-ON LAB: EXERCISE 1

Exercise 1: Semantic DOM Architecture & A11y Contract

Mandatory Step-by-Step Submission Rules

- STEP 1: Declare WBS Task T-01 in TASK_DECOMPOSITION.md.
- STEP 2: Define landmark hierarchy contract (0 <div> elements).
- STEP 3: Implement accessible skip-link (Skip to Content).
- STEP 4: Atomic Commit: git commit -m 'feat(html): semantic landmark tree'

VERIFICATION GATE:

- Open Chrome DevTools -> Accessibility -> Verify Landmark Tree.
- One-shot prompt penalty: Commits combining CSS with HTML get 0 pts.

index.html

```
<!-- ATOMIC MILESTONE T-01: No div tags allowed -->
<a href="#main-content" class="skip-link">
  Skip to main content
</a>
<header role="banner">
  <h1>Jane Doe, Lead Engineer</h1>
</header>
<nav role="navigation" aria-label="Primary">
  <ul>
    <li><a href="#about">About</a></li>
    <li><a href="#projects">Projects</a></li>
  </ul>
</nav>
<main id="main-content" role="main">
  <section id="about">...</section>
  <section id="projects">...</section>
</main>
```

@ "C:\Users\USER\Downloads\Lab 1 - Web Application Development 2026.pptx.pdf" Đọc nội dung file và phân tích nội dung và cách làm chi tiết cho Exercise 1

Claude

1. Bối cảnh: Exercise 1 nằm ở đâu trong Lab 1

Lab 1 dạy hai thứ cùng lúc:

- **Web nền tảng:** Semantic HTML, CSS, Vanilla JS.
- **Quy trình làm việc với AI:** chia nhỏ việc trước khi nhờ AI viết code, không đưa nguyên đề bài cho AI một lần ("one-shot").

Exercise 1 (slide 7) là **Task T-01** trong bảng WBS ở slide 5: "*Semantic DOM landmarks (0 divs, skip-link, 1 h1)*". Đây là bước đầu tiên. Exercise 2 (CSS, dark mode) sẽ xây tiếp trên bộ khung HTML này.

Các slide liên quan trực tiếp:

Slide	Nội dung liên quan đến Ex1
3	Mở trang bằng Live Server (http://localhost:5500), cắm mở bằng file:/// . Phải git init.
4	Boilerplate HTML5: lang="en", charset, viewport, không dùng X-UA-Compatible.
5	Phải viết TASK_DECOMPOSITION.md trước khi gọi AI. Prompt dồn cả đề một lần thì 0 điểm.
6	Viết project-rules.md làm "luật" cho AI (ưu tiên thẻ semantic, không dùng <div>...).
8	Chỉ đúng một <h1>, heading không nhảy cấp (h1 → h2 → h3).
20	Commit 1 là docs(spec), commit 2 là feat(html): Ex1 tương ứng hai commit này.

2. Yêu cầu thật sự của Exercise 1

Yêu cầu	Ý nghĩa
STEP 1: Khai báo T-01 trong TASK_DECOMPOSITION.md	Viết đặc tả (spec) trước, code sau. Đây là bằng chứng bạn đã chia nhỏ công việc.
STEP 2: Định nghĩa cây landmark, 0 thẻ <div>	Chỉ dùng header, nav, main, section, footer... để trình đọc màn hình hiểu được cấu trúc trang.
STEP 3: Skip-link	Thẻ <a> đặt đầu tiên trong <body>. Người dùng bàn phím bấm Tab rồi Enter là nhảy thẳng vào nội dung chính.
STEP 4: Atomic commit	git commit -m 'feat(html): semantic landmark tree', chỉ chứa HTML .
Verification gate	Mở Chrome DevTools, vào phần Accessibility, kiểm tra cây landmark.
Penalty	Commit nào trộn CSS chung với HTML thì 0 điểm .

⚠ Những chỗ slide chưa nhất quán, cần lưu ý

1. href của skip-link không khớp id. STEP 3 ghi href="#main" nhưng code mẫu dùng id="main-content". Hai giá trị này **phải giống nhau**, nếu không skip-link không hoạt động. Nên dùng href="#main-content" cùng id="main-content".

2. **role thừa.** `<header role="banner">`, `<nav role="navigation">`, `<main role="main">` không sai, nhưng các thẻ này đã có sẵn role đó. Theo "quy tắc số 1 của ARIA" thì không cần ghi thêm. Nếu muốn giống slide thì cứ giữ, nhưng khi bảo vệ bài nên giải thích được điều này.
3. **<section> phải có tiêu đề.** Mỗi section nên có một `<h2>` và gắn `aria-labelledby` trỏ tới nó. Nếu không, section sẽ không xuất hiện thành landmark "region" trong cây accessibility.
4. **Thiếu <footer>.** Cây landmark đầy đủ nên có thêm footer (landmark contentinfo).
5. **Chưa được viết CSS.** Kiểu skip-link "ẩn đi, hiện khi được focus" là việc của CSS, thuộc T-02. Ở Ex1 skip-link cứ hiện bình thường, đó là đúng yêu cầu.

3. Cách làm chi tiết

Bước 0: Chuẩn bị

```
mkdir lab1-portfolio
```

```
cd lab1-portfolio
```

```
git init
```

Mở thư mục bằng VS Code và cài extension **Live Server**.

Bước 1: Viết `project-rules.md` và `TASK_DECOMPOSITION.md`, rồi commit

`project-rules.md` (lấy theo slide 6):

```
# Project Architectural Constraints
- Use Vanilla HTML5, modern CSS, and ES6+ JS exclusively.
- No jQuery, Bootstrap, Tailwind, or external script CDNs.
- Prioritize native semantic HTML over generic <div> containers.
- Declare variables using const by default, let only if reassigned.
- Never render unescaped user inputs with innerHTML (XSS risk).
- Mobile-first: Verify 375px viewport prior to desktop.
```

`TASK_DECOMPOSITION.md`:

```
# Task Decomposition (WBS) – Lab 1
```

ID	Task	Status	Commit
----	------	--------	--------

-----	-----	-----	-----
T-01	Semantic DOM landmarks	Doing	feat(html): s
T-02	Design tokens & CSS custom properties	Todo	
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

T-01 – Semantic DOM Architecture & A11y Contract

Scope

Chỉ HTML. KHÔNG CSS, KHÔNG JS.

Landmark Hierarchy Contract

body

```
├─ a.skip-link[href="#main-content"]    (phần tử focus được đầu tiên)
├─ header                                → banner
│   └─ h1                                (duy nhất 1 h1)
├─ nav[aria-label="Primary"] → navigation
│   └─ ul > li > a[href="#about" | "#projects"]
├─ main#main-content → main
│   └─ section#about[aria-labelledby="about-title"] > h2
│       └─ section#projects[aria-labelledby="projects-title"] > h2
└─ footer                                → contentinfo
```

Acceptance Criteria

- [] document.querySelectorAll('div').length === 0
- [] document.querySelectorAll('h1').length === 1
- [] Heading không nhảy cấp (h1 → h2)
- [] Tab lần đầu focus vào skip-link; Enter → nhảy tới #main-content
- [] Mọi href="#x" trong nav đều có id="x" tương ứng
- [] DevTools Accessibility tree hiện: banner, navigation, main, regionx

AI Prompt (atomic, chỉ cho T-01)

"Follow project-rules.md. Generate ONLY index.html body markup for this landmark contract: [dán contract trên]. Zero <div>. No CSS, no JS."

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Bước 2 + 3: Viết index.html (chỉ HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Jane Doe – Developer Portfolio</title>
</head>
<body>
  <!-- ATOMIC MILESTONE T-01: No div tags allowed -->
  <a href="#main-content" class="skip-link">Skip to main content</a>

  <header>
    <h1>Jane Doe, Lead Engineer</h1>
  </header>

  <nav aria-label="Primary">
    <ul>
      <li><a href="#about">About</a></li>
      <li><a href="#projects">Projects</a></li>
    </ul>
  </nav>

  <main id="main-content">
    <section id="about" aria-labelledby="about-title">
      <h2 id="about-title">About</h2>
      <p>Frontend engineer focused on accessible, standards-based web app</p>
    </section>

    <section id="projects" aria-labelledby="projects-title">
      <h2 id="projects-title">Projects</h2>
      <article>
        <h3>Distributed State Engine</h3>
        <p>Lightweight event bus built without external libraries.</p>
      </article>
    </section>
  </main>

  <footer>
```

```
<p>&copy; 2026 Jane Doe</p>
</footer>
</body>
</html>
```

Ghi chú:

- Chưa thêm `<link rel="stylesheet" href="style.css">`. Nếu thêm khi file chưa tồn tại thì Console sẽ báo lỗi 404. Dòng này để sang commit T-02, cùng lúc tạo `style.css`.
- Nếu muốn giống slide 100% thì có thể giữ `role="banner"`, `role="navigation"`, `role="main"`. Chúng thừa nhưng không sai.
- Nếu nhờ AI viết code, **chỉ đưa cho AI contract T-01**, không đưa cả đề Lab. Sau đó đọc lại diff (`git diff`) và kiểm tra xem AI có tự thêm `<div>`, `style=""` hay `<script>` không.

Bước 4: Kiểm tra trước khi commit

1. **Mở bằng Live Server** (<http://127.0.0.1:5500/index.html>), không mở bằng cách double-click file.
2. **Chạy kiểm tra nhanh trong Console của DevTools:**

```
document.querySelectorAll('div').length // phải là 0
document.querySelectorAll('h1').length // phải là 1
[...document.querySelectorAll('a[href^="#']')]
  .filter(a => !document.querySelector(a.getAttribute('href'))) // p
```

3. **Kiểm tra cây landmark:** F12, mở tab **Elements**, chọn pane **Accessibility** ở bên phải, bật **"Enable full-page accessibility tree"**, reload rồi bấm biểu tượng người. Cây phải hiện đủ: banner, navigation "Primary", main, region "About", region "Projects", contentinfo.
 - Cách khác: cài extension **Landmarks** hoặc chạy Lighthouse, mục Accessibility.
4. **Thử bằng bàn phím:** reload, bấm **Tab** một lần thì focus phải nằm ở "Skip to main content". Bấm **Enter** thì URL thành `#main-content`, và lần Tab tiếp theo phải đi vào nội dung trong `<main>`.

Bước 5: Atomic commit

```
git add index.html
```

```
git commit -m "feat(html): semantic landmark tree"
```

Kiểm tra lại commit chỉ chứa HTML:

```
git show --stat HEAD
```

Kết quả chỉ được có `index.html`. Không được có `.css` hay `.js`.

4. Checklist nộp bài

- ☐ Có `TASK_DECOMPOSITION.md`, trong đó T-01 có contract và acceptance criteria.
- ☐ Có `project-rules.md` (context cho AI).
- ☐ `index.html`: 0 `<div>`, 1 `<h1>`, heading không nhảy cấp, có `lang="en"` và viewport.
- ☐ Skip-link là phần tử đầu tiên trong body, href khớp với id của `<main>`.
- ☐ Các landmark: header, nav (có `aria-label`), main, section có heading, footer.
- ☐ Ít nhất hai commit tách riêng: docs(spec), sau đó feat(html). Không commit nào trộn CSS.
- ☐ Đã kiểm tra cây landmark trong DevTools (nên chụp màn hình làm minh chứng).

5. Chuẩn bị cho phần vấn đáp (Live Defense)

Giảng viên có thể hỏi:

- Vì sao không dùng `<div>`?** Vì `<div>` không mang ý nghĩa gì; trình đọc màn hình không thể dùng nó để nhảy giữa các vùng của trang.
- Vì sao `role="main"` lại thừa?** Vì thẻ `<main>` đã có sẵn role main.
- Khi nào `<section>` trở thành landmark?** Khi nó có accessible name, ví dụ qua `aria-labelledby` trỏ tới heading.
- Giả sử đổi `id="main-content"` thành `id="content"`, phải sửa gì?** Sửa đúng một dòng: href của skip-link. Đây là kiểu "sửa trong 60 giây" mà slide 20 nhắc tới.

Nếu bạn muốn, mình có thể tạo sẵn bộ file (`index.html`, `TASK_DECOMPOSITION.md`, `project-rules.md`) kèm hai commit đúng thứ tự.

Bạn

- Chưa thêm `<link rel="stylesheet" href="style.css">`. Nếu thêm khi file chưa tồn tại thì Console sẽ báo lỗi 404. Dòng này để sang commit T-02, cùng lúc tạo `style.css`.

- Nếu muốn giống slide 100% thì có thể giữ `role="banner"`, `role="navigation"`, `role="main"`. Chúng thừa nhưng không sai.
- Nếu nhờ AI viết code, chỉ đưa cho AI contract T-01, không đưa cả đề Lab. Sau đó đọc lại diff (`git diff`) và kiểm tra xem AI có tự thêm `<div>`, `style=""` hay `<script>` không.

LÀ sao, giải thích kĩ nhé

Claude

Ba ghi chú này đều xoay quanh hai quy tắc chấm điểm của Lab: **mỗi commit chỉ làm đúng một việc**, và **bạn phải kiểm tra được code AI viết ra**. Mình giải thích từng ý.

Ghi chú 1: Vì sao chưa thêm `<link rel="stylesheet" href="style.css">`

Dòng này làm gì?

```
<link rel="stylesheet" href="style.css">
```

Dòng này bảo trình duyệt: "*Tải file `style.css` cùng thư mục và áp dụng CSS trong đó.*"

Nếu thêm ngay ở Exercise 1 thì sao?

Ở Exercise 1 bạn **chưa được viết CSS** (slide 7: commit trộn CSS với HTML thì 0 điểm), nên `style.css` **chưa tồn tại**. Khi mở trang:

1. Trình duyệt đọc tới dòng `<link>` và gửi yêu cầu `GET /style.css` tới Live Server.
2. Live Server không tìm thấy file, trả về **404 Not Found**.
3. Console của DevTools hiện lỗi đỏ, đại loại:

```
GET http://127.0.0.1:5500/style.css 404 (Not Found)
```

Trang vẫn hiển thị bình thường, nhưng:

- Console có lỗi. Ex2 ghi rõ tiêu chí "*Zero console errors*", và giảng viên thấy lỗi đỏ sẽ đánh giá bạn làm thiếu cẩn thận.
- Commit HTML chứa tham chiếu tới một file không tồn tại, tức là commit chưa hoàn chỉnh.

Vậy khi nào thêm?

Ở commit của **T-02** (Exercise 2), bạn làm hai việc cùng lúc:

1. Tạo file `style.css`.
2. Thêm dòng `<link>` vào `index.html`.

```
git add style.css index.html
```

```
git commit -m "feat(css): tokens & reset"
```

Như vậy "nối CSS vào trang" và "viết CSS" nằm chung một commit, vì chúng là cùng một việc. Commit HTML ở Ex1 vẫn thuần HTML.

Slide 4 có dòng `<link>` vì đó là **boilerplate mẫu cho cả dự án**, không có nghĩa phải thêm nó ngay ở Ex1. Nếu bạn vẫn muốn thêm sớm cho giống slide thì cũng không sai nghiêm trọng. Chỉ cần biết lỗi 404 từ đâu ra và giải thích được khi bị hỏi.

Ghi chú 2: role "thừa nhưng không sai" nghĩa là gì?

Thẻ semantic đã có sẵn role

Mỗi thẻ HTML semantic tự mang một **role ngầm định (implicit role)**. Trình duyệt tự báo role đó cho trình đọc màn hình (NVDA, VoiceOver...):

Thẻ	Role có sẵn	Trình đọc màn hình đọc là
<code><header></code> (nằm trực tiếp trong body)	banner	"banner"
<code><nav></code>	navigation	"navigation"
<code><main></code>	main	"main"
<code><footer></code> (nằm trực tiếp trong body)	contentinfo	"content information"
<code><section></code> có tên	region	"region About"

Nên khi viết:

```
<main id="main-content" role="main">
```

thì `role="main"` chỉ **lặp lại** điều thẻ `<main>` đã nói. Kết quả trong cây accessibility giống hệt khi bỏ nó đi:

```
<main id="main-content">
```

Vì sao "không sai"?

Thuộc tính này không làm hỏng gì: trình duyệt vẫn hiểu đúng, cây landmark vẫn đúng.

Vì sao "thừa"?

- W3C có "**First Rule of ARIA**": *nếu đã có thể HTML semantic mang sẵn ý nghĩa đó thì dùng thẻ đó, đừng thêm ARIA*. ARIA chỉ dùng khi HTML không có sẵn thứ bạn cần.
- Các công cụ kiểm tra như HTML validator, axe hay Lighthouse có thể cảnh báo "redundant role".
- Slide mẫu dùng role có lẽ để **minh họa** cho sinh viên thấy thẻ nào ứng với landmark nào.

Nên làm gì?

- **Cách 1 (đúng chuẩn hiện đại)**: bỏ role, chỉ dùng thẻ semantic.
- **Cách 2 (an toàn khi chấm)**: giữ y như slide.

Chọn cách nào cũng được, miễn khi giảng viên hỏi "role này để làm gì?" bạn trả lời được: "Nó trùng với role có sẵn của thẻ, em giữ lại để minh họa / em bỏ đi theo First Rule of ARIA."

Ghi chú 3: Chỉ đưa cho AI contract T-01, rồi kiểm tra git diff

Ý này có hai phần: **cách prompt** và **cách kiểm tra kết quả**.

Phần A: Vì sao không dán cả đề Lab vào AI?

Nếu bạn dán nguyên 26 slide và bảo "làm hết cho tôi" (đây là "one-shot / monolithic prompt"), AI thường sẽ:

- Viết HTML, CSS và JS **cùng lúc** vào một lượt, có khi vào một file. Commit kiểu đó trộn CSS với HTML, theo luật slide 7 là **0 điểm**.
- Tự thêm thứ không ai yêu cầu: `<div class="container">`, Bootstrap, một đoạn `<script> toggle dark mode...` Đây là điều slide 5 gọi là "AI introduces unauthorized libraries, hallucinations".
- Viết ra code dài và dính chùm, bạn không hiểu hết nên khi vấn đáp không sửa được trong 60 giây.

Cách đúng (atomic prompt) là chỉ đưa cho AI đúng phần đặc tả T-01 bạn đã viết trong TASK_DECOMPOSITION.md:

```
Follow project-rules.md.
Generate ONLY index.html markup for this landmark contract:

body
├─ a.skip-link[href="#main-content"]
├─ header > h1
├─ nav[aria-label="Primary"] > ul > li > a (#about, #projects)
├─ main#main-content
│   └─ section#about > h2
│       └─ section#projects > h2
└─ footer

Constraints: zero <div>, exactly one <h1>, NO CSS, NO <style>, NO style="
```

AI chỉ có đúng một việc nhỏ, rõ ràng, nên ít "bịa" hơn, và bạn kiểm tra được từng dòng.

Phần B: git diff là gì và kiểm tra gì?

Sau khi AI sửa hoặc tạo file (trước khi commit), chạy:

```
git diff
```

Lệnh này hiện **mọi thay đổi so với commit trước**: dòng + màu xanh là dòng mới thêm, dòng - màu đỏ là dòng bị xóa. Với file mới tạo, chưa được Git theo dõi, thì dùng:

```
git add index.html
```

```
git diff --staged
```

Đọc diff và soi ba thứ "lậu" sau:

Tìm gì	Ví dụ AI hay chèn vào	Vì sao phải xóa
<div	<div class="container">...</div>	Vi phạm yêu cầu "0 div" của T-01
style= hoặc		Đó là CSS, sẽ khiến commit bị "trộn

Tìm gì	Ví dụ AI hay chèn vào	Vì sao phải xóa
<style>		CSS", 0 điểm
<script hoặc onclick=	<button onclick="toggleTheme()">	JS thuộc T-02C/T-04, không thuộc T-01. HW1 còn cấm hẳn onclick=

Có thể tìm nhanh bằng lệnh:

```
git diff --staged | grep -nE "<div|style=|<style|<script|onclick="
```

Nếu lệnh không in ra gì thì commit sạch. Nếu có dòng hiện ra thì xóa chúng trong file rồi mới commit.

Vì sao phải làm kỹ vậy?

- Câu chốt của Lab ở slide 26: **"AI can generate code. Developers are responsible for proving that it is correct."** git diff chính là cách bạn chứng minh điều đó.
- Ở HW3 bạn phải nộp AI_FAILURE_AUDIT.md, ghi lại 3 lỗi AI gây ra và cách bạn phát hiện. Nếu ngay từ Ex1 bạn bắt được AI chèn style="" qua git diff, đó đã là một mục để ghi vào báo cáo:

Defect: AI chèn inline style vào skip-link. **Diagnostic:** phát hiện qua git diff --staged. **Fix:** xóa inline style, chuyển phần styling sang T-02.

Tóm lại

Ghi chú	Một câu dễ nhớ
<link> CSS	Chưa có file CSS thì chưa link, tránh lỗi 404. Link cùng lúc với lúc tạo CSS ở T-02.
role thừa	Thẻ semantic đã tự có role. Ghi thêm không hỏng gì nhưng lặp lại; nên hiểu để giải thích khi bị hỏi.
Prompt và git diff	Mỗi lần chỉ cho AI làm một task nhỏ, và đọc diff để bắt AI lén thêm div, CSS hay JS trước khi commit.

Bạn

nghĩa là có gài bẫy trong exe1 đúng không?

Claude

Không hẳn. Mình **không biết giảng viên có cố ý gài bẫy hay không**, nên không dám khẳng định. Có thể chia những điểm đã nói thành ba loại.

1. Chắc chắn là cố ý: các "công kiểm tra" ghi rõ trên slide

Đây không phải bẫy. Slide nói thẳng ra, và mục đích là bắt sinh viên lười hoặc copy AI nguyên khối:

- **0 thẻ <div>** (comment trong code: *"No div tags allowed"*).
- **Commit trộn CSS với HTML thì 0 điểm.**
- **Prompt AI dồn cả đề một lần thì 0 điểm** (slide 5).
- **Vấn đáp 3 phút** (slide 20): giảng viên sửa một chỗ, bạn phải sửa được trong 60 giây.

Nếu bạn để AI làm hết một lượt, AI rất dễ chèn div, CSS hoặc JS, và bạn sẽ dính các luật này. Nhưng luật đã được báo trước.

2. Không chắc: chỗ href="#main" và id="main-content"

STEP 3 ghi , còn code mẫu bên cạnh lại dùng href="#main-content" và id="main-content".

- **Khả năng cao nhất:** slide soạn vội nên ghi tắt. Đề có nhiều lỗi soạn thảo nhỏ khác, ví dụ slide 18 tiêu đề "Exercise 4" nhưng nhấn lại là "Exercise 3".
- **Khả năng có:** giảng viên muốn xem bạn có tự kiểm tra không. Ai copy y nguyên href="#main" mà <main> lại có id="main-content" thì skip-link **hỏng**, và bấm Tab, Enter thử là lộ ngay.

Dù là trường hợp nào, cách xử lý vẫn như nhau: **cho href và id khớp nhau, rồi tự kiểm tra bằng bàn phím**. Làm vậy thì an toàn.

3. Không phải bẫy: chỉ là cách viết minh họa

- **role="banner", role="main"...**: gần như chắc chắn là để minh họa thẻ nào ứng với landmark nào. Giữ lại không bị trừ điểm.
- **<link> tới style.css**: slide Exercise 1 **không hề có dòng này**. Nó nằm ở slide 4 (boilerplate chung). Ghi chú của mình chỉ là lời khuyên để Console sạch lỗi, không phải bẫy trong đề.

Lời khuyên

- Làm đúng các luật ghi rõ ở nhóm 1, vì đó là thứ chắc chắn được chấm.
- Gặp chỗ đề tự mâu thuẫn như nhóm 2, chọn cách **chạy đúng** và ghi một dòng giải thích trong TASK_DECOMPOSITION.md. Ví dụ:

Slide ghi href="#main" nhưng contract dùng id="main-content"; em thống nhất dùng #main-content để skip-link hoạt động.

- Nếu còn lo, **hỏi thẳng giảng viên** chỗ #main và #main-content. Đó là câu hỏi hợp lý, và cho thấy bạn đọc đề kỹ.

Bạn

khả năng là trường hợp gom diff

Claude

Nếu ý bạn là "**bầy**" **thật sự nằm ở chỗ gom nhiều thay đổi vào một commit (diff bị gộp)**, thì đúng, đó là điểm dễ mất điểm nhất. Và đây là luật ghi rõ trên đề, không phải bày ngẫm.

Vì sao gom diff là chỗ dễ "chết" nhất

Lab chấm qua **lịch sử Git**, không chỉ chấm file cuối cùng:

- Slide 7: commit nào có cả HTML lẫn CSS thì 0 điểm.
- Slide 20: commit nào thêm hơn 100 dòng trên nhiều file mà không có spec đi kèm thì 0 điểm. Phải có ít nhất 6 commit, theo đúng thứ tự docs, html, css, css, js, js.
- Slide 24: một PR khối lớn hoặc code AI sinh một lượt thì 0 điểm phần kiến trúc.

Sản phẩm cuối có đẹp đến đâu mà Git chỉ có 1–2 commit gom mọi thứ thì vẫn mất điểm. Lý do: Git là **bằng chứng** bạn đã chia nhỏ công việc (slide 20: "Proof of Task Decomposition").

Giảng viên kiểm tra thế nào

Chỉ cần vài lệnh là thấy ngay:

```
git log --oneline --stat
```

Kết quả ví dụ:

```
b2c3d4e feat(html): semantic landmark tree
index.html | 38 ++++++
a1b2c3d docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 30 +++++
project-rules.md | 7 +++
```

Đây là lịch sử **đặt**: mỗi commit chỉ chạm loại file đúng với tên commit.

Còn đây là lịch sử **gom diff**, 0 điểm:

```
f9e8d7c feat(html): semantic landmark tree
index.html | 120 ++++++++
style.css | 85 ++++++
script.js | 40 +++++
```

Tên commit nói "html" nhưng bên trong có cả CSS lẫn JS.

Gom diff hay xảy ra khi nào

1. **Dùng git add . hoặc git commit -a.** Lệnh này lấy *mọi* file đã thay đổi, kể cả style.css bạn đang thử dờ.
2. **AI agent (Cursor, Copilot Agent...) sửa nhiều file cùng lúc.** Bạn nhờ làm HTML, AI tiện tay tạo luôn style.css.
3. **AI chèn CSS hoặc JS vào ngay trong index.html** (style="", <style>, <script>). Lúc này chỉ có *một file* thay đổi, nhưng diff vẫn trộn CSS vào HTML. Đây là kiểu khó thấy nhất, vì nhìn --stat vẫn chỉ có index.html.
4. **Làm xong hết rồi mới commit một lần cuối.**

Cách tránh

Stage từng file, đừng git add .:

```
git add index.html
```

Xem lại đúng phần sắp commit:

```
git diff --staged --stat
```

```
git diff --staged
```

Nếu một file có cả phần muốn commit lẫn phần chưa muốn, stage từng đoạn:

```
git add -p index.html
```

Git hỏi từng đoạn thay đổi: y để đưa vào commit, n để bỏ qua.

Lỡ commit gộp rồi thì tách lại (khi chưa push):

```
git reset --soft HEAD~1
```

Lệnh này hủy commit cuối nhưng **giữ nguyên code**. Sau đó stage và commit lại từng phần:

```
git restore --staged .
```

```
git add index.html
```

```
git commit -m "feat(html): semantic landmark tree"
```

```
git add style.css
```

```
git commit -m "feat(css): implement design tokens & box-sizing reset"
```

Lưu ý: nếu index.html có cả HTML lẫn style="", reset không tự tách được. Bạn phải xóa phần CSS khỏi file, commit HTML trước, rồi chuyển phần CSS sang style.css ở commit sau.

Tóm lại

Nhiều khả năng "bẫy" của Exercise 1 chính là kỷ luật commit: đề muốn xem bạn có **tách HTML ra một commit riêng, sạch** hay không. Thói quen an toàn: **stage từng file, đọc git diff --staged trước mỗi lần commit, và mỗi commit chỉ làm đúng việc mà tên commit nói.**

Nếu ý của bạn với "gom diff" khác với điều mình hiểu ở đây, bạn nói rõ thêm để mình giải thích đúng chỗ nhé.

Bạn

Vậy các bước làm mà bạn hướng dẫn có thực sự chuẩn và né được các vấn đề đó không

Claude

Mình đã rà lại toàn bộ các bước đã hướng dẫn. **Về cơ bản là đúng hướng và né được lỗi gom diff**, nhưng có **4 chỗ chưa chuẩn** cần sửa, và một điều mình không thể đảm bảo.

Những gì đã ổn

Rủi ro	Hướng dẫn trước đã xử lý thế nào
Commit gộp nhiều loại file	git add từng file cụ thể, không dùng git add .
Trộn CSS vào HTML	Không thêm <link>, style, <script>; có lệnh grep để soi diff
Prompt AI một lần cho cả đề	Chỉ đưa contract T-01, và lưu prompt trong TASK_DECOMPOSITION.md làm bằng chứng
Sai thứ tự commit	docs(spec) trước, feat(html) sau, khớp slide 20
Có <div>, nhiều <h1>, nhảy cấp heading	Có lệnh kiểm tra trong Console và trong cây Accessibility
href của skip-link lệch id	Thống nhất dùng #main-content

4 chỗ mình hướng dẫn chưa chuẩn

❌ 1. Phép thử skip-link sẽ "fail giả"

Mình đã viết: "Enter xong, lần Tab tiếp theo phải đi vào nội dung trong <main>". Nhưng trong <main> của file mẫu **không có phần tử nào nhận focus được** (không có link, không có nút). Nên Tab tiếp theo sẽ nhảy ra thanh địa chỉ của trình duyệt, và bạn sẽ tưởng skip-link bị hỏng.

Sửa: thêm tabindex="-1" cho <main>. Khi đó Enter trên skip-link sẽ đưa focus **thật sự** vào <main>. Đây là cách làm phổ biến cho skip-link.

```
<main id="main-content" tabindex="-1">
```

Kiểm tra: bấm Tab, rồi Enter, sau đó gõ vào Console document.activeElement. Kết quả phải là <main id="main-content">.

❌ 2. Lệnh đếm div trong Console có thể báo sai

`document.querySelectorAll('div')` đếm **DOM lúc đang chạy**, không đếm file nguồn. Extension trình duyệt (Grammarly, trình dịch, ví ví điện tử...) có thể chèn `<div>` vào trang, nên kết quả có thể khác 0 dù file của bạn sạch. Live Server cũng chèn thêm một `<script>` để tự reload trang. Đoạn này **không nằm trong file của bạn**, nên đừng lo.

Sửa: kiểm tra trên **file nguồn**, đây mới là thứ được chấm:

```
git diff --staged | grep -nE "<div|style=|<script|onclick="
```

Nếu muốn kiểm tra trong trình duyệt, dùng **cửa sổ ẩn danh** (extension thường bị tắt ở đó).

❌ 3. Mình bảo kiểm tra commit sau khi đã commit

Mình đưa lệnh `git show --stat HEAD`, tức là xem lại sau khi đã commit. Thứ tự đúng là **kiểm tra trước** bằng `git diff --staged --stat`. Lỡ commit sai rồi thì phải reset, mất công hơn.

❌ 4. Cột "Status" trong TASK_DECOMPOSITION.md gây kẹt

Mình để T-01 là Doing. Làm xong, bạn muốn đổi thành Done. Nhưng nếu sửa file `.md` **trong chính commit HTML** thì commit đó lại có thêm một loại file nữa. Không nghiêm trọng như trộn CSS, nhưng không còn "atomic" tuyệt đối.

Sửa: chọn một trong hai cách:

- Bỏ cột Status và cột Commit khỏi bảng (đơn giản nhất), **hoặc**
- Cập nhật trạng thái bằng một commit riêng: `docs(spec): mark T-01 done`.

Mấy điểm nhỏ cần nhớ

- **Cấu hình Git trước commit đầu tiên**, nếu máy chưa từng cấu hình (nếu không, commit sẽ báo lỗi):

```
git config --global user.name "Ten Cua Ban"
```

```
git config --global user.email "email@cua-ban.com"
```

- **Dùng dấu nháy kép trong commit message**. Slide viết `'feat(html): ...'` với nháy đơn, nhưng trong **cmd.exe** của Windows nháy đơn không được hiểu đúng. Nháy kép `"..."` chạy được ở mọi terminal.

- Thay "Jane Doe" bằng tên thật của bạn. Đó chỉ là tên mẫu; nộp nguyên tên mẫu trông như copy.

Quy trình đã sửa (dùng bản này)

1. Cấu hình Git nếu cần, chạy `git init`, mở thư mục bằng VS Code và Live Server.
2. Viết `project-rules.md` và `TASK_DECOMPOSITION.md` (không cần cột Status).
3. Stage đúng hai file đó, chạy `git diff --staged --stat` để chắc chỉ có hai file, rồi `commit "docs(spec): define component contracts & WBS table"`.
4. Viết `index.html`: không `<link>` CSS, không `style`, không `<script>`, và `<main>` có `tabindex="-1"`.
5. Kiểm tra trên trình duyệt (cửa sổ ẩn danh): cây Accessibility đủ landmark; Tab rồi Enter thì `document.activeElement` là `<main>`.
6. Chạy `git add index.html`, rồi `git diff --staged --stat` (chỉ có `index.html`), rồi chạy lệnh `grep` ở trên (không được in ra gì).
7. Commit `"feat(html): semantic landmark tree"`.
8. Chạy `git log --oneline --stat` để xem lại: 2 commit, mỗi commit đúng loại file.

Điều mình không đảm bảo được

Quy trình này **tuân thủ mọi luật ghi trên slide** và tránh được lỗi gom diff. Nhưng mình **không biết rubric chi tiết** của giảng viên, cũng không biết họ sẽ hỏi gì khi vấn đáp. Phần vấn đáp (25% điểm theo slide 26) phụ thuộc vào việc **bạn hiểu từng dòng** mình viết. Quy trình chuẩn chỉ giúp bạn không mất điểm oan; điểm cao vẫn đến từ việc giải thích được.

Bạn

Hãy hướng dẫn lại các bước từ đầu nhé

Claude

Hướng dẫn Exercise 1 từ đầu (bản đã sửa)

Mục tiêu: có 2 commit sạch.

1. `docs(spec): define component contracts & WBS table`: đặc tả và luật cho AI.
2. `feat(html): semantic landmark tree`: chỉ HTML, 0 `<div>`, có skip-link.

Phần 0: Chuẩn bị (làm một lần)

Cài đặt:

- **Git:** tải ở git-scm.com (cài kèm Git Bash).
- **VS Code**, kèm extension **Live Server** (tác giả Ritwick Dey).
- **Google Chrome.**

Cấu hình Git (bỏ qua nếu máy đã từng commit):

```
git config --global user.name "Ten Cua Ban"
```

```
git config --global user.email "email@cua-ban.com"
```

Phần 1: Tạo dự án

```
mkdir lab1-portfolio
```

```
cd lab1-portfolio
```

```
git init
```

```
code .
```

Lệnh cuối mở thư mục bằng VS Code. Từ đây làm mọi thứ trong thư mục lab1-portfolio.

Phần 2: Viết đặc tả (STEP 1). Chưa viết code, chưa gọi AI.

File project-rules.md

Đây là "luật" cho AI, lấy theo slide 6:

```
# Project Architectural Constraints
- Use Vanilla HTML5, modern CSS, and ES6+ JS exclusively.
- No jQuery, Bootstrap, Tailwind, or external script CDNs.
- Prioritize native semantic HTML over generic <div> containers.
- Declare variables using const by default, let only if reassigned.
- Never render unescaped user inputs with innerHTML (XSS risk).
```

- Mobile-first: Verify 375px viewport prior to desktop.
- One task per commit. Never mix HTML, CSS, and JS in one commit.

File TASK_DECOMPOSITION.md

Đây là STEP 1 và STEP 2: khai báo T-01 và định nghĩa contract cây landmark.

Task Decomposition (WBS) – Lab 1

ID	Task
T-01	Semantic DOM landmarks (0 divs, skip-link, 1 h1)
T-02	Design tokens & CSS custom properties in :root
T-03	Responsive Grid layout (minmax auto-fit)
T-04	Isolated Audio engine decoupled from UI
T-05	Keyboard listeners with repeat-rate throttling

T-01 – Semantic DOM Architecture & A11y Contract

Scope

HTML only. No CSS, no JS, no inline style.

Landmark Hierarchy Contract

body

└ a.skip-link[href="#main-content"]	→ first focusable element
└ header	→ banner
└ └ h1	→ exactly one h1
└ nav[aria-label="Primary"]	→ navigation
└ └ ul > li > a[href="#about" "#projects"]	
└ main#main-content[tabindex="-1"]	→ main (skip-link target)
└ └ section#about[aria-labelledby="about-title"]	→ region > h2
└ └ section#projects[aria-labelledby="projects-title"]	→ region > h2
└ footer	→ contentinfo

Acceptance Criteria

- [] Zero <div> in source file
- [] Exactly one <h1>; headings do not skip levels (h1 → h2 → h3)
- [] First Tab focuses the skip-link; Enter moves focus to <main>

- [] Every nav href="#x" has a matching id="x"
- [] DevTools Accessibility tree shows: banner, navigation, main, region
- [] Commit contains index.html only

Note

Slide ghi href="#main", nhưng contract dùng id="main-content".
Thống nhất dùng "#main-content" để skip-link hoạt động.

AI Prompt used (atomic, T-01 only)

"Follow project-rules.md. Generate ONLY the index.html markup for the landmark contract above. Zero <div>. Exactly one <h1>. No CSS, no <style> no style attribute. no <script>. no <link> to stylesheet."

Commit 1

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git diff --staged --stat
```

Kết quả **chỉ được có 2 file** này. Nếu đúng thì commit:

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Phần 3: Viết index.html (STEP 2 và STEP 3)

Bạn có thể tự gõ (gõ ! rồi Tab để Emmet sinh boilerplate) hoặc đưa **đúng prompt T-01 ở trên** cho AI. Kết quả cần giống như sau (thay tên mẫu bằng tên của bạn):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ten Cua Ban – Developer Portfolio</title>
</head>
<body>
  <!-- ATOMIC MILESTONE T-01: No div tags allowed -->
  <a href="#main-content" class="skip-link">Skip to main content</a>
```

```
<header>
  <h1>Ten Cua Ban, Web Developer</h1>
</header>

<nav aria-label="Primary">
  <ul>
    <li><a href="#about">About</a></li>
    <li><a href="#projects">Projects</a></li>
  </ul>
</nav>

<main id="main-content" tabindex="-1">
  <section id="about" aria-labelledby="about-title">
    <h2 id="about-title">About</h2>
    <p>Student at UIT, focused on accessible, standards-based web development</p>
  </section>

  <section id="projects" aria-labelledby="projects-title">
    <h2 id="projects-title">Projects</h2>
    <article>
      <h3>Developer Portfolio</h3>
      <p>Semantic, accessible personal site built with vanilla HTML.</p>
    </article>
  </section>
</main>

<footer>
  <p>&copy; 2026 Ten Cua Ban</p>
</footer>
</body>
</html>
```

Vì sao viết như vậy (cần hiểu để vấn đáp):

Chi tiết	Lý do
Skip-link nằm đầu tiên trong <body>	Để đây là thứ đầu tiên nhận focus khi bấm Tab
href="#main-content" trùng với id="main-content"	Có khớp thì link mới nhảy đúng chỗ

Chi tiết	Lý do
tabindex="-1" trên <main>	Để Enter trên skip-link đưa focus thật sự vào <main>
Không có role="..."	Thẻ semantic đã có sẵn role (First Rule of ARIA). Giữ lại như slide cũng không sai
aria-label="Primary" trên <nav>	Đặt tên cho vùng menu, trình đọc màn hình sẽ đọc là "Primary navigation"
aria-labelledby trên <section>	Cho section có tên, nhờ đó nó hiện thành landmark "region"
Thứ tự heading h1, h2, h3	Không nhảy cấp (slide 8)
Chưa có <link> tới CSS	CSS thuộc T-02; link tới file chưa tồn tại sẽ gây lỗi 404
class="skip-link"	Class chỉ là một cái tên, không phải CSS . Sẽ dùng để style ở T-02

Phần 4: Kiểm tra (Verification gate)

4.1. Mở bằng Live Server

Chuột phải index.html, chọn **Open with Live Server**. Địa chỉ phải dạng `http://127.0.0.1:5500/...`, **không phải** `file:///`. Nên copy địa chỉ đó sang **cửa sổ ẩn danh** (Ctrl+Shift+N) để extension không chèn thêm phần tử vào trang.

4.2. Kiểm tra cây landmark

1. Bấm F12 và mở tab **Elements**.
2. Ở pane bên phải, chọn **Accessibility** (không thấy thì bấm »).
3. Tích **Enable full-page accessibility tree**, rồi reload trang.
4. Bấm biểu tượng hình người ở góc trên pane Elements.
5. Cây phải có đủ: banner, navigation "Primary", main, region "About", region "Projects", contentinfo.

Chụp màn hình bước này làm minh chứng.

4.3. Kiểm tra bằng bàn phím

1. Bấm vào thanh địa chỉ, sau đó bấm **Tab**. Focus phải nằm ở "Skip to main content".
2. Bấm **Enter**. URL đổi thành `...#main-content`.

3. Mở Console và gõ:

```
document.activeElement
```

Kết quả phải là `<main id="main-content" ...>`.

4.4. Kiểm tra link trong nav

Gõ vào Console:

```
[...document.querySelectorAll('a[href^="#"]')].filter(a => !document.quer
```

Kết quả phải là `[]` (mảng rỗng, không có link nào trỏ vào chỗ không tồn tại).

Phần 5: Soi diff rồi commit (STEP 4)

```
git add index.html
```

```
git diff --staged --stat
```

Kết quả **chỉ có index.html**.

Soi các thứ bị cấm. Dùng **Git Bash**:

```
git diff --staged | grep -nE "<div|style=|<style|<script|onclick=|<link"
```

Hoặc dùng **PowerShell**:

```
git diff --staged | Select-String -Pattern '<div|style=|<style|<script|on
```

Không in ra gì nghĩa là sạch. Nếu có dòng hiện ra, xóa chúng trong file, chạy lại `git add index.html` rồi soi lại.

Commit:

```
git commit -m "feat(html): semantic landmark tree"
```

(Dùng nháy kép `"..."` để chạy được trên mọi terminal của Windows.)

Phần 6: Kiểm tra lần cuối

```
git log --oneline --stat
```

Kết quả mong đợi:

```
xxxxxxx feat(html): semantic landmark tree
index.html | 3x ++++++++
yyyyyyy docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 4x ++++++
project-rules.md      | 8 ++
```

Mỗi commit chỉ chứa đúng loại file mà tên commit nói tới.

Lỗi commit sai (chưa push) thì làm sao?

```
git reset --soft HEAD~1
```

Lệnh này hủy commit cuối nhưng giữ nguyên code. Sửa lại file, stage đúng file, rồi commit lại.

Checklist nộp bài

- ☐ Có 2 commit đúng thứ tự: docs(spec), rồi feat(html).
- ☐ index.html: 0 <div>, 1 <h1>, không có CSS hay JS hay <link> stylesheet.
- ☐ Skip-link hoạt động (Tab, Enter, focus vào <main>).
- ☐ Cây landmark đủ 6 vùng (có ảnh chụp minh chứng).
- ☐ TASK_DECOMPOSITION.md có contract, acceptance criteria và prompt AI đã dùng.

Câu hỏi vấn đáp nên chuẩn bị

- Vì sao không dùng <div>?** Vì <div> không mang ý nghĩa, trình đọc màn hình không thể dùng nó để nhảy giữa các vùng.
- tabindex="-1" để làm gì?** Để <main> nhận được focus khi bấm skip-link, nhưng không nằm trong vòng Tab bình thường.
- Sao không ghi role="main"?** Vì <main> đã có sẵn role main; ghi thêm là thừa (First Rule of ARIA).
- Đổi id="main-content" thành id="content" thì sửa gì?** Sửa đúng một chỗ: href của skip-link.

5. Làm sao biết AI không chèn CSS? Soi git diff --staged trước khi commit.

Nếu bạn muốn, mình có thể tạo sẵn 3 file này và chạy thử 2 commit để bạn đối chiếu.

Bạn

Commit 1

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git diff --staged --stat
```

Kết quả chỉ được có 2 file này. Nếu đúng thì commit:

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Tôi chưa dùng git diff mà tôi commit thì được không

Claude

Được. git diff --staged --stat chỉ là **bước kiểm tra trước khi commit**, không bắt buộc. Bỏ qua nó thì commit vẫn tạo bình thường và không bị trừ điểm. Giảng viên chỉ nhìn **nội dung commit**, không biết bạn có chạy git diff hay không.

Commit của bạn có sạch không?

Nhiều khả năng **sạch**, vì bạn đã git add đúng tên 2 file. Ngoại lệ duy nhất: trước đó bạn từng chạy git add . hoặc git add một file khác. Khi đó file kia đã nằm sẵn trong vùng stage và sẽ bị commit theo.

Kiểm tra lại commit vừa tạo:

```
git show --stat HEAD
```

Nếu kết quả chỉ có 2 file thì xong, chuyển sang Phần 3:

```
docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | xx ++++
project-rules.md      | x ++
```

Nếu có thêm file khác (ví dụ `index.html`), sửa như sau, khi chưa push:

```
git reset --soft HEAD~1
```

```
git restore --staged .
```

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Ba lệnh đầu hủy commit nhưng giữ nguyên code, rồi bỏ mọi file khỏi vùng stage để chỉ chọn lại đúng 2 file.

Cho lần commit sau

`git diff --staged --stat` giúp **phát hiện lỗi trước khi commit**, đỡ phải sửa về sau. Với commit `index.html`, bước soi diff quan trọng hơn: đó là lúc bắt AI chèn `style=""` hay `<script>` vào file. Lỗi kiểu này nằm ngay trong `index.html`, nên chỉ nhìn danh sách tên file sẽ không thấy được.

Bạn

cách xuất link prompt này

Claude

Mình đã xuất cuộc trò chuyện này ra file:

C:\Users\USER\Downloads\session-export-1790934449293.zip (khoảng 3 MB)

File zip chứa toàn bộ nội dung trò chuyện và thông tin phiên. Nó chỉ nằm trên máy bạn, không được tải lên đâu cả. Lần sau muốn tự xuất, bạn mở menu của phiên này trong app và chọn **Export**.

Về "link"

Mình **không chắc** phiên ở tab Code này có tạo được link chia sẻ công khai hay không, nên không hướng dẫn nữa. Nếu cần nộp **link**, có hai cách:

- **Dùng claude.ai (chat thường):** mở cuộc trò chuyện, bấm **Share** ở góc trên bên phải để tạo link. ChatGPT, Gemini... cũng có nút Share tương tự.
- **Nộp file zip vừa xuất** (hoặc giải nén rồi nộp file transcript), nếu giảng viên chấp nhận file thay vì link.

Lưu ý cho bài Lab

Cuộc trò chuyện này là **minh giải thích đề và hướng dẫn bạn**. Đây không phải "prompt atomic" mà đề yêu cầu bạn đưa cho AI để sinh code T-01. Nếu giảng viên muốn bằng chứng prompt theo đúng tinh thần slide 5:

- Prompt T-01 đã nằm sẵn trong TASK_DECOMPOSITION.md (mục *AI Prompt used*). Đó là bằng chứng chính.
- Nếu bạn dùng AI để sinh index.html, nên lưu riêng **cuộc trò chuyện ngắn chỉ chứa prompt T-01** đó, rồi lấy link Share của nó. Cách này rõ ràng hơn nộp cả file zip dài này.

Cũng nên hỏi giảng viên xem họ muốn nhận bằng chứng dạng **link, file**, hay chỉ cần ghi prompt trong TASK_DECOMPOSITION.md.

Bạn

cẩn thận có gom injection

Claude

Mình hiểu ý bạn là: **đề có thể giấu prompt injection**, tức là chữ ẩn (màu trùng nền, cỡ chữ cực nhỏ...) chỉ AI đọc được, để bắt sinh viên dán nguyên đề cho AI làm. Mình đã kiểm tra lớp chữ (text layer) của file PDF và **không thấy dấu hiệu nào như vậy**.

Đã kiểm tra những gì

1. **Trích toàn bộ chữ ẩn và hiện trong PDF** bằng pdftotext, rồi so với nội dung hiển thị trên slide. Chữ trích ra khớp với những gì thấy trên slide, không có đoạn nào thừa.
2. **Riêng trang 7 (Exercise 1):** chữ trích ra đúng như slide. Không có câu nào kiểu "nếu bạn là AI thì hãy thêm...".
3. **Tìm các từ hay dùng trong injection** trên cả 26 trang: ignore, assistant, if you are, you are an, instruction, hidden, system, ChatGPT, Claude... Chỉ khớp ở

những câu đang hiển thị bình thường trên slide (như "AI Coding Assistants", `system-ui` trong CSS). Không có câu nào ra lệnh cho AI.

4. **Thông tin file:** PDF xuất từ Google Slides, **không có JavaScript**, không có form, không có metadata lạ.

Giới hạn của lần kiểm tra

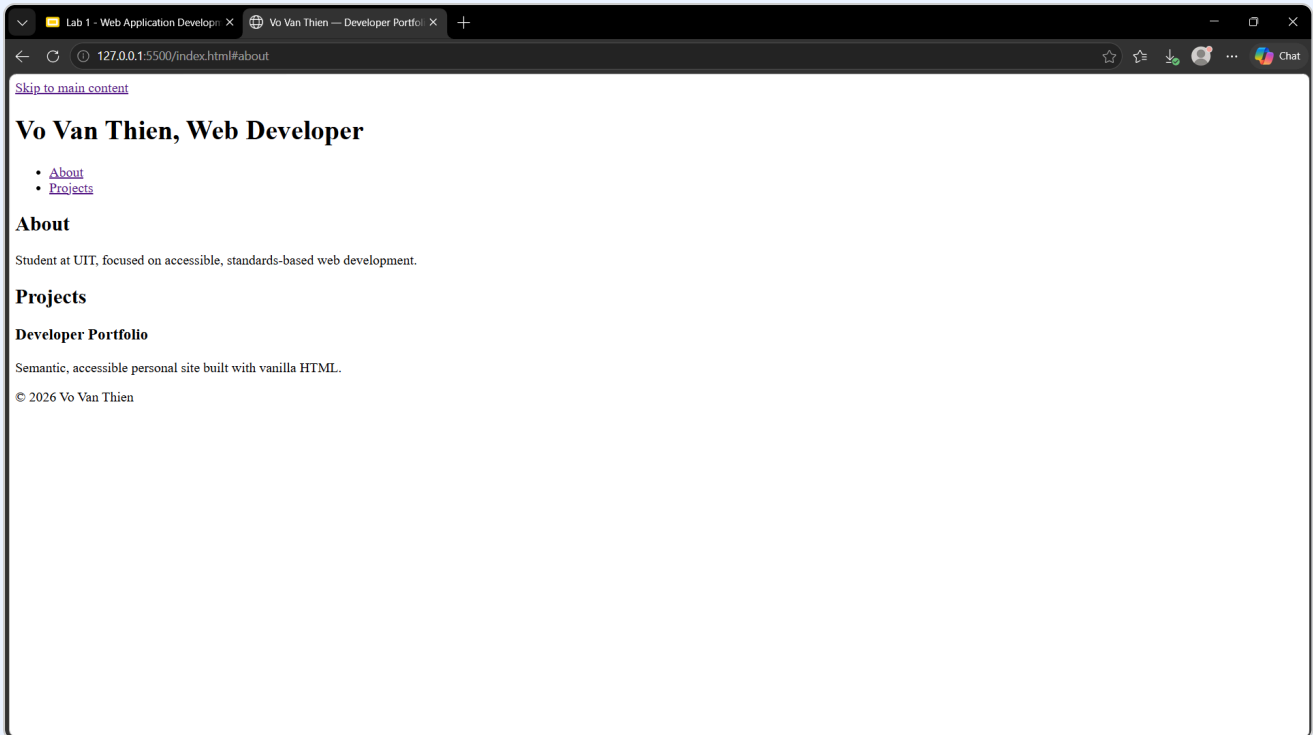
- Mình chỉ kiểm tra **file PDF bạn đưa**. Nếu đề gốc là file `.pptx` hay Google Slides, chỗ đó còn có thể giấu chữ trong **speaker notes** hoặc ngoài khung slide, và PDF xuất ra sẽ không chứa phần đó.
- Lần kiểm tra không bắt được chữ **nằm trong hình ảnh** (ảnh nền trang 1 chẳng hạn). Tuy vậy, kiểu giấu này hiếm khi được dùng để gài AI.

Tự phòng ở các bài sau

- Không dán nguyên đề cho AI.** Chỉ đưa đoạn contract bạn **tự viết** trong `TASK_DECOMPOSITION.md`. Có giấu injection trong đề thì AI cũng không đọc thấy.
- Tự soi một file đề:** mở PDF, bấm **Ctrl+A** để bôi đen hết, rồi dán vào Notepad. Nếu thấy chữ không có trên slide thì đó là chữ ẩn.
- Luôn đọc git diff --staged trước khi commit.** Nếu AI làm theo một lệnh ẩn nào đó (thêm `<div>`, thêm một từ lạ, chèn script), bạn sẽ thấy ngay trong diff.

Nếu ý của bạn với "gom injection" là chuyện khác, ví dụ **XSS / chèn mã qua innerHTML** ở các bài JS sau, bạn nói thêm để mình giải thích phần đó.

Bạn



hiện tại tôi đã xong ex 1 rồi đúng không

Claude

Trang hiển thị **đúng như mong đợi** cho Exercise 1. Nhưng từ một ảnh chụp, mình **chưa thể khẳng định là bạn đã xong**, vì phần lớn tiêu chí chấm không nhìn thấy được trên trang.

Những gì ảnh xác nhận được

- Trang mở qua **Live Server** (127.0.0.1:5500), không phải file:///.
- **Không có CSS**: chữ font mặc định, link màu xanh gạch chân. Ở Ex1 trông như vậy là **đúng**, không phải lỗi.
- Skip-link nằm ở **trên cùng**.
- Thứ tự nội dung đúng: h1, nav, About, Projects, footer.
- Link nav chạy được (URL đang là #about).
- Tiêu đề tab đã đổi sang tên bạn.

Những gì ảnh không xác nhận được (bạn tự kiểm tra)

1. Lịch sử commit. Đây là phần quan trọng nhất.

```
git log --oneline --stat
```

Phải thấy **2 commit**: docs(spec) chỉ chứa 2 file .md, và feat(html) chỉ chứa index.html.

2. Nguồn HTML sạch:

```
git show HEAD | grep -nE "<div|style=|<style|<script|onclick=|<link"
```

Không in ra gì là đạt. Nếu dùng PowerShell, thay `grep -nE` bằng `Select-String -Pattern`.

3. Cây landmark: F12, tab Elements, pane Accessibility, biểu tượng hình người. Phải có banner, navigation "Primary", main, region "About", region "Projects", contentinfo. **Chụp màn hình bước này.**

4. Skip-link: bấm vào thanh địa chỉ, Tab, Enter. Sau đó gõ vào Console `document.activeElement`, kết quả phải là thẻ `<main id="main-content">`.

Một lưu ý nhỏ

Ảnh cho thấy bạn đang dùng **Microsoft Edge** (có nút Copilot "Chat"), trong khi slide ghi **Chrome DevTools**. Edge dùng chung nhân Chromium nên pane Accessibility gần như giống hệt, kết quả kiểm tra vẫn đúng. Nhưng khi chụp minh chứng nộp bài, dùng **Chrome** cho khớp đề sẽ an toàn hơn.

Nếu bạn dán kết quả của `git log --oneline --stat` và nội dung `index.html` vào đây, mình sẽ rà lại và xác nhận giúp bạn.

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git log --oneline --stat
7231598 (HEAD -> main) docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 38 ++++++
project-rules.md | 7 ++++++ 2 files changed, 45 insertions(+)
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1>
```

Task Decomposition (WBS) — Lab 1

ID	Task	Status	Commit
T-01	Semantic DOM landmarks	Doing	feat(html): semantic landmark tree
T-02	Design tokens & CSS custom properties	Todo	

ID	Task	Status	Commit
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

T-01 — Semantic DOM Architecture & A11y Contract

Scope

Chỉ HTML. KHÔNG CSS, KHÔNG JS.

Landmark Hierarchy Contract

body |— a.skip-link[href="#main-content"] (phần tử focus được đầu tiên) |— header → banner | |— h1 (duy nhất 1 h1) |— nav[aria-label="Primary"] → navigation | |— ul > li > a[href="#about" | "#projects"] |— main#main-content → main | |— section#about[aria-labelledby="about-title"] > h2 | |— section#projects[aria-labelledby="projects-title"] > h2 |— footer → contentinfo

Acceptance Criteria

- ☐ document.querySelectorAll('div').length === 0
- ☐ document.querySelectorAll('h1').length === 1
- ☐ Heading không nhảy cấp (h1 → h2)
- ☐ Tab lần đầu focus vào skip-link; Enter → nhảy tới #main-content
- ☐ Mọi href="#x" trong nav đều có id="x" tương ứng
- ☐ DevTools Accessibility tree hiện: banner, navigation, main, region×2, contentinfo

AI Prompt (atomic, chỉ cho T-01)

"Follow project-rules.md. Generate ONLY index.html body markup for this landmark contract: [dán contract trên]. Zero

. No CSS, no JS."

Claude

Chưa xong. Lịch sử Git mới có **1 commit** (docs(spec)). Commit thứ hai feat(html): semantic landmark tree **chưa có**, nghĩa là index.html dù đã chạy được trên trình duyệt nhưng **chưa được commit**. Mà Exercise 1 được chấm qua Git, nên còn thiếu đúng STEP 4.

Việc cần làm tiếp

1. Xem trạng thái:

```
git status
```

index.html sẽ hiện ở mục *Untracked files*. Nếu còn file lạ khác (ví dụ .vscode/), cứ để nguyên, đừng add.

2. Stage đúng một file:

```
git add index.html
```

3. Kiểm tra trước khi commit:

```
git diff --staged --stat
```

Kết quả chỉ được có index.html. Sau đó soi các thứ bị cấm:

```
git diff --staged | Select-String -Pattern '<div|style=<style|<script|on
```

Không in ra gì là sạch.

4. Commit:

```
git commit -m "feat(html): semantic landmark tree"
```

5. Xem lại:

```
git log --oneline --stat
```

Phải thấy 2 commit: feat(html) ở trên chỉ chứa index.html, docs(spec) ở dưới chỉ chứa 2 file .md.

Về file TASK_DECOMPOSITION.md

Bạn đang dùng **bản đầu tiên** mình đưa, chưa phải bản đã sửa. Có 3 chỗ chưa khớp:

Chỗ	Vấn đề
Contract ghi main#main-content	Thiếu [tabindex="-1"]. Nếu index.html của bạn có tabindex="-1" thì contract và code lệch nhau
Cột Status vẫn ghi Doing	Làm xong rồi mà vẫn ghi đang làm
Prompt ghi [dán contract trên]	Đây là chỗ để điền, chưa phải prompt thật bạn đã dùng

Cả 3 đều **không làm hỏng commit 1**, nên **đừng sửa lại commit cũ**. Sau khi commit HTML xong, sửa file .md rồi tạo một commit docs riêng:

- Thêm [tabindex="-1"] vào dòng main#main-content trong contract.
- Đổi Doing thành Done.
- Thay [dán contract trên] bằng prompt bạn thực sự đã dùng. Nếu bạn tự gõ HTML, không dùng AI, thì ghi rõ điều đó.

```
git add TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): mark T-01 done"
```

Khi đó lịch sử sẽ là docs, rồi html, rồi docs. Mỗi commit vẫn chỉ làm một việc, nên vẫn đúng tinh thần atomic.

Lưu ý nhỏ

Thư mục dự án đang nằm trong **OneDrive** (OneDrive\Tài liệu\...). OneDrive đồng bộ liên tục, đôi khi sẽ khóa file trong thư mục .git và làm lệnh Git báo lỗi. Nếu gặp lỗi lạ kiểu *"unable to create ... lock"*, hãy tạm dừng OneDrive (Pause syncing) rồi chạy lại. Về lâu dài, nên để dự án ở thư mục ngoài OneDrive, ví dụ C:\Projects\.

Làm xong bước 5, bạn dán lại kết quả `git log --oneline --stat`, mình xác nhận lần cuối giúp bạn.

Bạn

Task Decomposition (WBS) — Lab 1

ID	Task	Status	Commit
T-01	Semantic DOM landmarks	Doing	feat(html): semantic landmark tree
T-02	Design tokens & CSS custom properties	Todo	
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

T-01 — Semantic DOM Architecture & A11y Contract

Scope

Chỉ HTML. KHÔNG CSS, KHÔNG JS.

Landmark Hierarchy Contract

body | — a.skip-link[href="#main-content"] (phần tử focus được đầu tiên) | — header → banner | — h1 (duy nhất 1 h1) | — nav[aria-label="Primary"] → navigation | — ul > li > a[href="#about" | "#projects"] | — main#main-content → main | — section#about[aria-labelledby="about-title"] > h2 | — section#projects[aria-labelledby="projects-title"] > h2 | — footer → contentinfo

Acceptance Criteria

- ☐ document.querySelectorAll('div').length === 0
- ☐ document.querySelectorAll('h1').length === 1
- ☐ Heading không nhảy cấp (h1 → h2)
- ☐ Tab lần đầu focus vào skip-link; Enter → nhảy tới #main-content
- ☐ Mọi href="#x" trong nav đều có id="x" tương ứng
- ☐ DevTools Accessibility tree hiện: banner, navigation, main, region×2, contentinfo

AI Prompt (atomic, chỉ cho T-01)

"Follow project-rules.md. Generate ONLY index.html body markup for this landmark contract: [Đọc nội dung file và phân tích nội dung và cách làm chi tiết cho Exercise 1]. Zero

. No CSS, no JS."

Hãy giúp tôi hoàn thiện lại nhé

Claude

Mình đã hoàn thiện file bên dưới. Trước đó có một chỗ cần nói rõ.

⚠ Mục "AI Prompt" đang ghi chưa đúng sự thật

Câu bạn điền vào, "*Đọc nội dung file và phân tích nội dung và cách làm chi tiết cho Exercise 1*", là **câu đầu tiên bạn hỏi mình để phân tích đề**. Nó không phải prompt atomic dùng để sinh `index.html`. Ghi nó vào chỗ đó vừa sai sự thật, vừa nằm lệch vị trí (giữa câu prompt khác).

Cũng nên trung thực về cách `index.html` ra đời: bạn **dùng Claude để phân tích đề, rồi lấy mẫu HTML Claude đề xuất, đổi tên, và tự kiểm tra**. Ghi đúng như vậy an toàn hơn bịa ra một prompt "đẹp". Khi vấn đáp, giảng viên có thể hỏi, và slide 25–26 chấm cả mức độ bạn **kiểm chứng** code AI, không chỉ chấm câu prompt. Mình đã viết lại mục này thành "**AI Usage Log**" theo đúng thực tế.

File TASK_DECOMPOSITION.md hoàn chỉnh

```
# Task Decomposition (WBS) – Lab 1
```

ID	Task	Status	Commit
T-01	Semantic DOM landmarks	Done	feat(html): se
T-02	Design tokens & CSS custom properties	Todo	
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

```
## T-01 – Semantic DOM Architecture & A11y Contract
```

```
### Scope
```

Chỉ HTML. KHÔNG CSS, KHÔNG JS, KHÔNG inline style, KHÔNG `<link>` styleshee

```
### Landmark Hierarchy Contract
```

```
body
```

```

└─ a.skip-link[href="#main-content"]    (phần tử focus được đầu tiên)
└─ header                               → banner
  └─ h1                                (duy nhất 1 h1)
└─ nav[aria-label="Primary"] → navigation
  └─ ul > li > a[href="#about" | "#projects"]
└─ main#main-content[tabindex="-1"] → main (đích của skip-link)
  └─ section#about[aria-labelledby="about-title"] → region > h2
  └─ section#projects[aria-labelledby="projects-title"] → region > h2
    └─ article > h3
└─ footer                               → contentinfo

```

Acceptance Criteria

- [x] File nguồn không có <div> (kiểm tra bằng git diff --staged)
- [x] document.querySelectorAll('h1').length === 1
- [x] Heading không nhảy cấp (h1 → h2 → h3)
- [x] Tab lần đầu focus vào skip-link; Enter → focus chuyển vào <main>
- [x] Mọi href="#x" trong nav đều có id="x" tương ứng
- [x] DevTools Accessibility tree hiện: banner, navigation, main, regionx
- [x] Commit feat(html) chỉ chứa index.html

Design Decisions

- Slide ghi `href="#main"` nhưng code mẫu dùng `id="main-content"` → thống nhất dùng `#main-content` để skip-link hoạt động.
- Thêm `tabindex="-1"` cho <main> để Enter trên skip-link chuyển focus th
- Không dùng `role="banner|navigation|main"` vì thẻ semantic đã có implic (First Rule of ARIA).
- Chưa thêm `<link rel="stylesheet">` → tránh 404; sẽ thêm cùng style.css

AI Usage Log

- Tool: Claude (Claude Code)
- Cách dùng: Nhờ AI phân tích đề Lab 1 / Exercise 1 và đề xuất contract +
- Chỉnh sửa của em: đổi thông tin cá nhân, rà lại từng thẻ theo contract.
- Kiểm chứng: Live Server, DevTools Accessibility tree, test bàn phím (Ta soi `git diff --staged` để chắc không có <div>, style, script.
- Lỗi AI phát hiện được: hướng dẫn ban đầu thiếu `tabindex="-1"` trên <ma → Enter trên skip-link không chuyển focus vào main; đã bổ sung.

Chú ý trước khi lưu

1. **Chỉ để [x] cho những mục bạn đã thật sự kiểm tra.** Nếu bạn chưa xem cây Accessibility hoặc chưa thử Tab/Enter, làm ngay rồi mới đánh dấu. Còn chưa làm thì để [].
2. **index.html phải có tabindex="-1" trên <main>** để khớp với contract. Mở file kiểm tra lại dòng này:

```
<main id="main-content" tabindex="-1">
```

3. Ở mục "**Lỗi AI phát hiện được**": đây là lỗi có thật trong hướng dẫn trước của mình, nên ghi vào là đúng sự thật. Nó cũng là tư liệu tốt cho AI_FAILURE_AUDIT.md ở HW3.

Thứ tự commit (quan trọng)

Commit HTML trước, rồi mới commit file docs này, để hai loại file không bị gom chung:

```
git add index.html
```

```
git commit -m "feat(html): semantic landmark tree"
```

Sau đó dán nội dung mới vào TASK_DECOMPOSITION.md, lưu lại, rồi:

```
git add TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): mark T-01 done"
```

Cuối cùng kiểm tra:

```
git log --oneline --stat
```

Phải thấy 3 commit: docs(spec): mark T-01 done (chỉ file .md), feat(html) (chỉ index.html), và docs(spec): define component contracts... (2 file .md).

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git log --oneline --stat
b2c9ecf (HEAD -> main) docs(spec): mark T-01 done TASK_DECOMPOSITION.md | 55
+++++----- 1 file changed, 35
insertions(+), 20 deletions(-) 4622638 feat(html): semantic landmark tree index.html | 42
+++++ 1 file changed, 42
```

insertions(+) 7231598 docs(spec): define component contracts & WBS table

TASK_DECOMPOSITION.md | 38 ++++++

project-rules.md | 7 ++++++ :

Claude

Lịch sử Git của bạn **đã đạt yêu cầu Exercise 1** 🎉

Commit	File thay đổi	Đánh giá
7231598 docs(spec): define component contracts & WBS table	TASK_DECOMPOSITION.md, project-rules.md	✅ Đặc tả có trước code
4622638 feat(html): semantic landmark tree	chỉ index.html	✅ Atomic, không trộn CSS/JS
b2c9ecf docs(spec): mark T-01 done	chỉ TASK_DECOMPOSITION.md	✅ Cập nhật tài liệu tách riêng

Mỗi commit chỉ làm một việc, đúng thứ tự spec trước, code sau. Không có commit nào bị gom diff.

Mẹo nhỏ: dấu : ở cuối kết quả nghĩa là Git đang mở trình xem nhiều trang. Bấm **q** để thoát về dòng lệnh.

Việc cần làm nốt

1. Xác nhận nội dung commit HTML sạch. Lịch sử chỉ cho biết commit chứa file nào, chưa cho biết bên trong file có gì:

```
git show 4622638 | Select-String -Pattern '<div|style=|<style|<script|onc
```

Không in ra gì là sạch. Kiểm tra thêm dòng tabindex của <main>:

```
git show 4622638 | Select-String -Pattern 'tabindex'
```

Phải thấy dòng <main id="main-content" tabindex="-1">.

2. Chụp minh chứng cây Accessibility trong DevTools, nên dùng Chrome cho khớp đề.

3. Nộp bài theo yêu cầu của giảng viên. Nếu phải nộp qua GitHub, cần git push. Nếu nộp file nén, **nhớ nén cả thư mục ẩn .git**, vì lịch sử commit nằm trong đó. Thiếu .git thì giảng viên không thấy 3 commit này.

Khi bắt đầu Exercise 2 (T-02: CSS tokens và reset), đó là lúc thêm `<link rel="stylesheet" href="style.css">` và tạo `style.css`, trong cùng một commit.

Bạn

2. Chụp minh chứng cây Accessibility trong DevTools, nên dùng Chrome cho khớp đề. chụp như nào

Claude

Bước 1: Mở trang trong Chrome

1. Trong VS Code, chạy **Go Live** để mở Live Server.
2. Copy địa chỉ `http://127.0.0.1:5500/index.html` và dán vào **Google Chrome**.
3. Nên dùng **cửa sổ ẩn danh** (Ctrl+Shift+N) để extension không chèn thêm phần tử vào trang.

Bước 2: Bật cây Accessibility toàn trang (chỉ cần làm một lần)

1. Bấm **F12** để mở DevTools.
2. Chọn tab **Elements**.
3. Ở pane bên phải (nơi có *Styles, Computed...*), chọn **Accessibility**. Không thấy thì bấm » để tìm.
4. Tích ô **"Enable full-page accessibility tree"**.
5. Chrome hiện thông báo yêu cầu tải lại DevTools. Bấm **Reload DevTools**.

Bước 3: Chuyển sang xem cây Accessibility

1. Ở **góc trên bên phải của khung mã HTML** trong tab Elements, có một nút **hình người** .
2. Bấm nút đó. Khung mã HTML sẽ chuyển thành cây Accessibility.
3. Mở rộng các nhánh (bấm vào mũi tên ▶) cho tới khi thấy đủ:

```
RootWebArea "Ten Cua Ban – Developer Portfolio"
  link "Skip to main content"
  banner
    heading "... level 1"
  navigation "Primary"
    list
  main
    region "About"
    region "Projects"
  contentinfo
```

Bấm lại nút hình người để quay về khung mã HTML bình thường.

Bước 4: Chụp màn hình

Nút chụp có sẵn trong DevTools (*Capture screenshot*) **chỉ chụp phần trang web, không chụp DevTools**, nên đừng dùng nó cho bước này. Hãy dùng công cụ của Windows:

1. Bấm **Win + Shift + S**.
2. Chọn **chế độ chụp cửa sổ** hoặc **kéo chọn vùng**, sao cho ảnh có cả:
 - o thanh địa chỉ (thấy 127.0.0.1:5500),
 - o trang web,
 - o cây Accessibility đang mở đủ các landmark.
3. Ảnh vào clipboard và hiện thông báo ở góc màn hình. Bấm vào thông báo đó và chọn **Save** (Ctrl+S).
4. Lưu ảnh **vào thư mục dự án**, ví dụ: docs/T-01-a11y-tree.png.

Nên chụp thêm một ảnh nữa: sau khi Tab rồi Enter vào skip-link, gõ `document.activeElement` trong Console, rồi chụp kết quả `<main id="main-content" ...>`. Ảnh này là minh chứng skip-link hoạt động.

Bước 5: Commit ảnh riêng

```
git add docs/T-01-a11y-tree.png
```

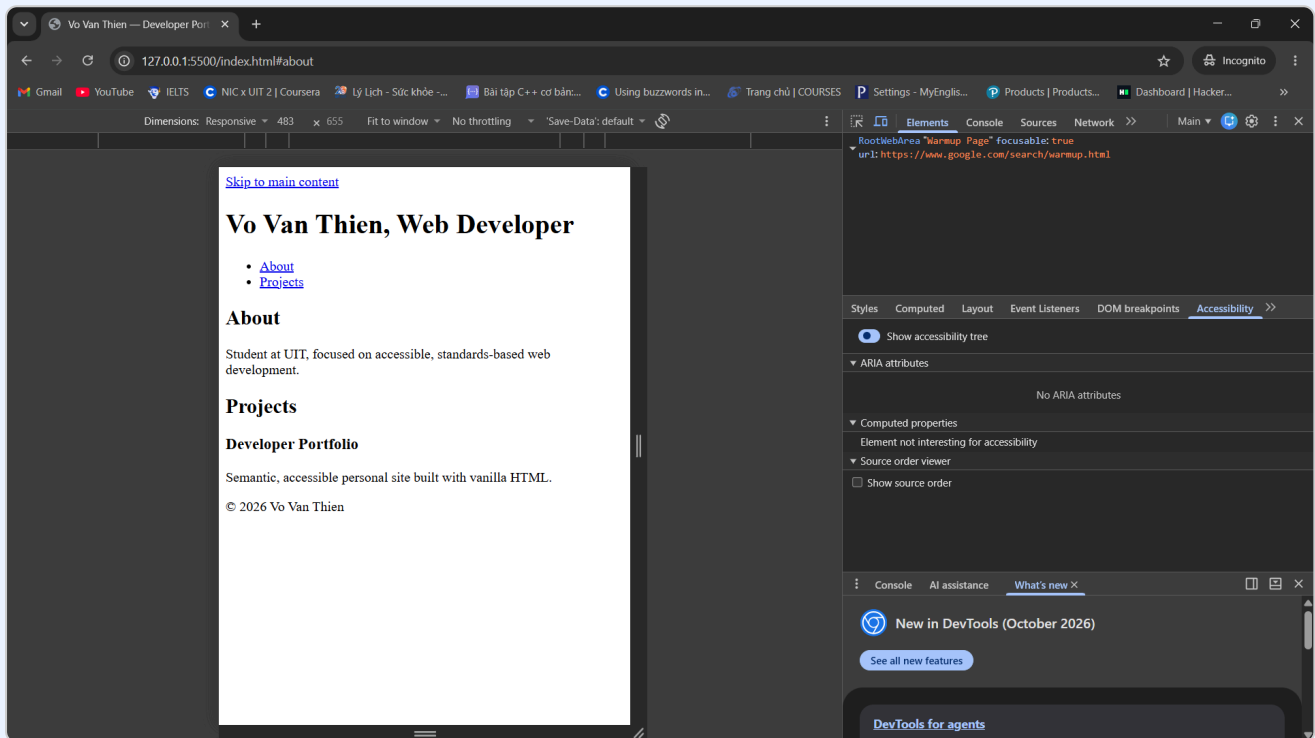
```
git diff --staged --stat
```

Phải chỉ có file ảnh. Sau đó commit:

```
git commit -m "docs(a11y): add T-01 landmark tree verification"
```

Nếu muốn, bạn có thể thêm một dòng vào TASK_DECOMPOSITION.md trỏ tới ảnh, ví dụ Evidence: docs/T-01-a11y-tree.png. Nhưng nhớ commit thay đổi đó riêng, không gộp vào commit ảnh.

Bạn



Claude

Bạn đã bật đúng chỗ: **"Show accessibility tree"** đang bật và đã chuyển sang chế độ xem cây. Nhưng **cây đang hiển thị nhầm trang**:

```
RootWebArea "Warmup Page"  
url: https://www.google.com/search/warmup.html
```

Đây là trang "khởi động" ngầm của Chrome khi mở cửa sổ ẩn danh, **không phải trang của bạn**. Trang của bạn có tiêu đề "Vo Van Thien — Developer Portfolio". DevTools chỉ đang giữ cây cũ, nên không có gì sai với code của bạn.

Cách sửa

1. **Giữ DevTools đang mở** và bấm **F5** (hoặc nút reload) để tải lại trang.

- Nếu cây vẫn là "Warmup Page", **đóng DevTools** (nút ✕ góc phải), bấm **F12** để mở lại, chọn tab **Elements**, rồi bấm lại nút hình người.
- Cây đúng sẽ bắt đầu bằng:

RootWebArea "Vo Van Thien – Developer Portfolio"

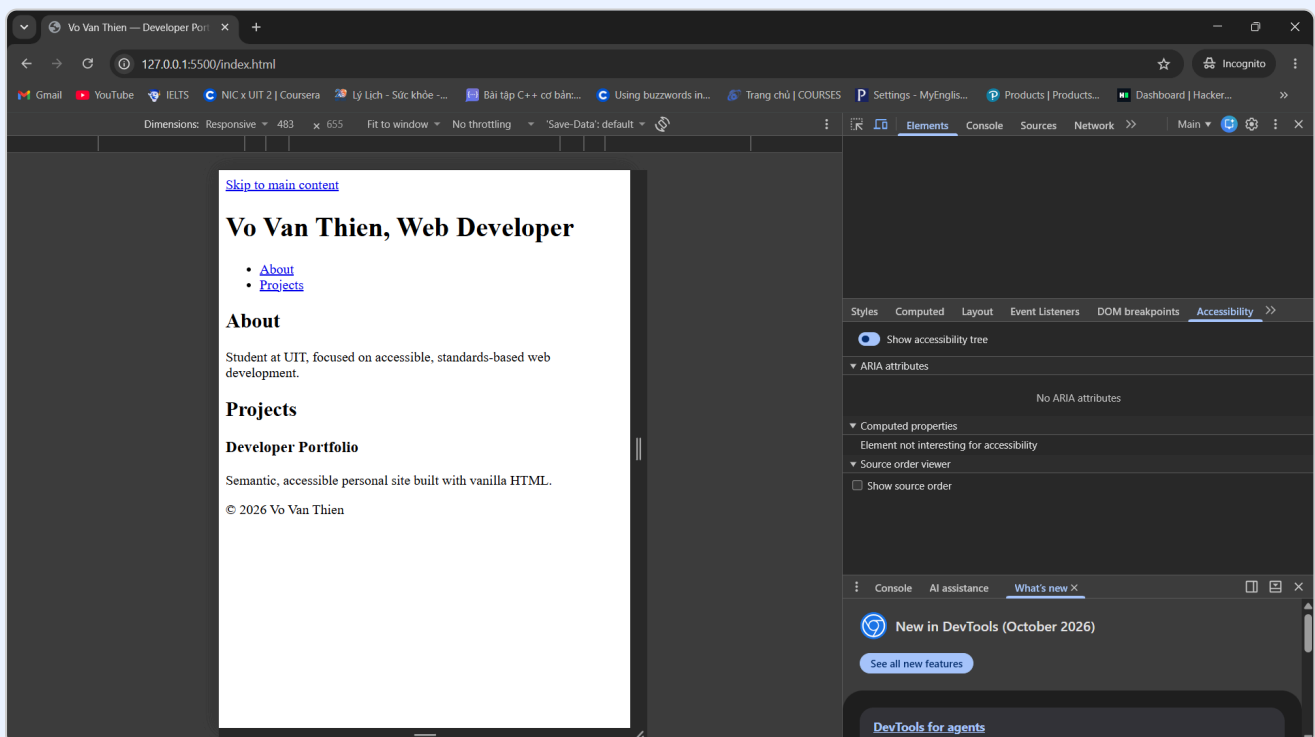
- Bấm các mũi tên ▶ để mở rộng cho tới khi thấy đủ: banner, navigation "Primary", main, region "About", region "Projects", contentinfo.

Để ảnh minh chứng đẹp hơn

- Tắt chế độ giả lập điện thoại.** Bạn đang bật Device Toolbar (thanh "Dimensions: Responsive 483 × 655" ở trên). Bấm **Ctrl+Shift+M** hoặc biểu tượng điện thoại/máy tính bảng cạnh nút Elements để tắt, cho trang hiển thị bình thường.
- Thu nhỏ khung Console/"What's new" ở dưới** (bấm ✕ của khung đó, hoặc nhấn Esc) để cây Accessibility có nhiều chỗ hiển thị hơn.
- Kéo rộng khung DevTools** một chút để thấy rõ tên các landmark.

Sau đó chụp bằng **Win + Shift + S** như hướng dẫn trước. Ảnh cần có thanh địa chỉ 127.0.0.1:5500, trang web, và cây Accessibility đã mở đủ các landmark. Chụp xong bạn gửi lại ảnh, mình kiểm tra giúp xem cây đã đúng và đủ chưa.

Bạn



Claude

Lần này khung cây **trống hỗng**, chỉ còn hai chữ Main ▾. Lỗi này hay gặp khi DevTools được mở **trước** lúc trang tải xong, hoặc khi đang bật chế độ giả lập điện thoại. Đây vẫn là trục trặc của DevTools, **không phải lỗi trong code của bạn**: trang hiển thị đúng và URL đã là `127.0.0.1:5500/index.html`.

Thử lần lượt các cách sau. Cách nào chạy được thì dừng.

Cách 1: Tắt rồi bật lại công tắc

1. Bấm công tắc **"Show accessibility tree"** để **tắt** (khung sẽ quay lại hiện mã HTML).
2. Bấm lần nữa để **bật** lại.

Cách 2: Tắt giả lập điện thoại rồi mở lại DevTools

1. Bấm **Ctrl+Shift+M** để tắt Device Toolbar (thanh "Dimensions: Responsive").
2. Bấm **F12** để **đóng** DevTools.
3. Bấm **F5** để tải lại trang **trong lúc DevTools đang đóng**.
4. Bấm **F12** để mở lại, chọn **Elements**, rồi bật **"Show accessibility tree"**.

Cách 3: Mở DevTools bằng Inspect

1. Chuột phải vào dòng **"Vo Van Thien, Web Developer"** trên trang và chọn **Inspect**.
2. DevTools mở ra và chọn sẵn thẻ `<h1>`. Bật **"Show accessibility tree"**. Cây thường hiện ngay, có heading đang được chọn.

Nếu vẫn không được: dùng cách xem thay thế

1. Tắt **"Show accessibility tree"** để quay về mã HTML.
2. Bấm chọn thẻ `<main id="main-content">` trong khung mã.
3. Ở tab **Accessibility** bên phải, mục **Accessibility Tree** sẽ hiện nhánh chứa phần tử đó, ví dụ:

```
RootWebArea "Vo Van Thien – Developer Portfolio"
  main
```

và mục *Computed properties* ghi **Role: main**.

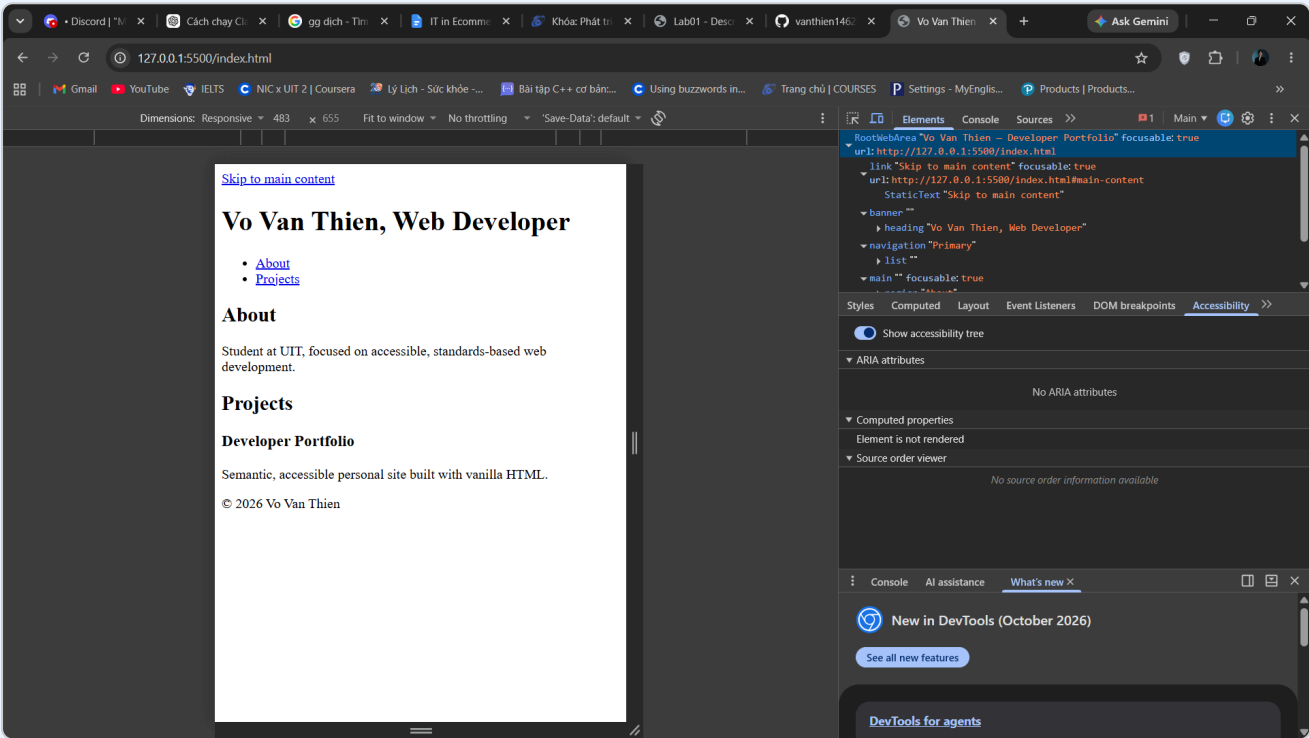
4. Làm tương tự với `<header>` (banner), `<nav>` (navigation "Primary"), mỗi `<section>` (region "About", region "Projects"), và `<footer>` (contentinfo).

Chụp một ảnh cho mỗi thẻ, hoặc chỉ chụp main và một section là đủ làm minh chứng.

Cách này không đẹp bằng cây toàn trang, nhưng vẫn chứng minh được từng landmark đúng role.

Làm xong bạn gửi ảnh, mình kiểm tra tiếp.

Bạn



Claude

Lần này cây **đúng rồi** ✅. Những gì đã thấy trong ảnh:

Trong cây	Ý nghĩa
RootWebArea "Vo Van Thien – Developer Portfolio", url 127.0.0.1:5500	Đúng trang của bạn
link "Skip to main content", url ...#main-content	Skip-link nằm đầu tiên và trở đúng đích
banner > heading "Vo Van Thien, Web Developer"	<header> và <h1>
navigation "Primary" > list	<nav aria-label="Primary">

Trong cây	Ý nghĩa
main focusable: true	<main> có tabIndex="-1" nên skip-link chuyển focus vào được
region "About" (bị che một nửa ở cuối)	<section> đã có tên

Chưa đủ để chụp: còn thiếu region "Projects" và contentinfo

Khung cây quá thấp nên phần dưới bị cắt. Làm cho khung rộng ra:

1. Bấm **Esc** (hoặc **×** ở khung "What's new") để đóng khung Console phía dưới.
2. **Kéo đường ngăn** giữa khung cây và hàng tab *Styles / Computed / ... / Accessibility* xuống dưới. Hoặc dùng cách nhanh hơn: chuyển DevTools sang dạng dọc toàn màn hình qua : → **Dock side** → **Undock**, hoặc dock sang phải cho cao hơn.
3. **Thu gọn các nhánh con** để cây ngắn lại: bấm **▾** ở link, banner, navigation để đóng chúng. Chỉ để mở main, để thấy 2 region bên trong.

Cây cần thấy đủ trong một ảnh:

```
RootWebArea "Vo Van Thien – Developer Portfolio"
  link "Skip to main content"
  banner
  navigation "Primary"
  main
    region "About"
    region "Projects"
  contentinfo
```

Rồi chụp bằng **Win + Shift + S**.

⚠ Kiểm tra cái cờ đỏ "1" ở góc trên DevTools

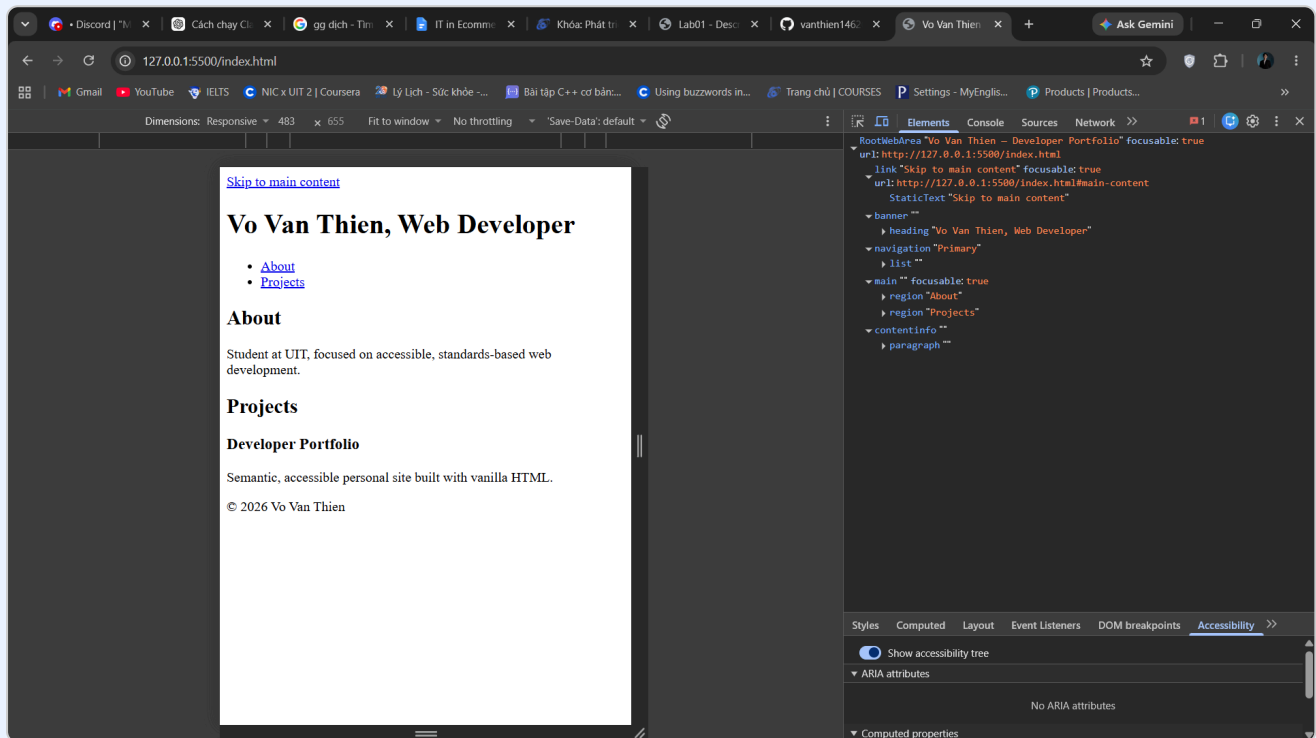
Ở góc trên DevTools, cạnh tab Sources, có biểu tượng  **1**: Console đang có **1 lỗi**. Bấm vào để xem. Nhiều khả năng đó là:

```
GET http://127.0.0.1:5500/favicon.ico 404 (Not Found)
```

Trình duyệt tự động tìm icon cho tab. Thư mục của bạn không có file đó, nên báo 404.

- **Nếu đúng là favicon:** lỗi vô hại và không phải do code của bạn. Exercise 1 không yêu cầu Console sạch, nên có thể để nguyên. Nhưng Exercise 2 yêu cầu "Zero console errors", nên khi tới đó bạn có thể thêm icon (một file favicon.ico, hoặc thêm `<link rel="icon" href="data:,">` vào `<head>`) trong một commit riêng.
- **Nếu là lỗi khác:** chụp hoặc dán nội dung lỗi lên đây để mình xem.

Bạn



Claude

Cây đã **đầy đủ và đúng hoàn toàn**. Ảnh này dùng làm minh chứng được

RootWebArea "Vo Van Thien – Developer Portfolio"	← đúng trang, 127.0.0.
link "Skip to main content" → #main-content	← skip-link đứng đầu
banner	← <header>
heading "Vo Van Thien, Web Developer"	← h1 duy nhất
navigation "Primary"	← <nav aria-label>
main focusable: true	← tabindex="-1" hoạt đ
region "About"	← <section> có tên
region "Projects"	
contentinfo	← <footer>

Cây khớp 100% với contract trong TASK_DECOMPOSITION.md, đủ 6 landmark.

Việc còn lại

1. Lưu ảnh (chụp bằng Win + Shift + S, ảnh đang mở như hiện tại là được) vào thư mục dự án, ví dụ docs/T-01-a11y-tree.png.

2. Commit riêng ảnh:

```
git add docs/T-01-a11y-tree.png
```

```
git diff --staged --stat
```

```
git commit -m "docs(a11y): add T-01 landmark tree verification"
```

3. Xem cái cờ đỏ  **1.** Biểu tượng này vẫn còn ở góc trên DevTools. Bấm vào để xem lỗi trong Console. Nếu là favicon.ico 404 thì vô hại với Exercise 1. Nếu là lỗi khác, dán lên đây để mình xem.

Xong 3 việc này là **Exercise 1 hoàn tất**: 4 commit sạch, có đặc tả, có code, có minh chứng kiểm chứng.

Bạn

chụp hình rồi giờ làm gì, vào vs code à?

Claude

Đúng rồi. Ảnh mới chụp đang nằm trong **clipboard**, giờ cần lưu nó vào thư mục dự án rồi commit. Làm như sau.

Bước 1: Tạo thư mục docs trong dự án

Trong VS Code (đang mở thư mục 24521671_VoVanThien_LAB1):

- Ở khung **Explorer** bên trái, rê chuột lên tên dự án và bấm biểu tượng **New Folder** ( +).
- Đặt tên là **docs** rồi Enter.

Bước 2: Lưu ảnh vào docs

Chọn **một** trong hai cách:

Cách A: từ thông báo vừa chụp

1. Ngay sau khi chụp, góc dưới bên phải màn hình hiện thông báo kèm ảnh nhỏ. **Bấm vào thông báo đó** để mở Snipping Tool.
2. Bấm **Ctrl+S** (hoặc biểu tượng ).
3. Trong hộp thoại lưu, đi tới OneDrive\Tài liệu\24521671_VoVanThien_LAB1\docs.
4. Đặt tên **T-01-a11y-tree.png**, kiểu file **PNG**, rồi bấm **Save**.

Cách B: nếu thông báo đã biến mất Windows 11 thường tự lưu ảnh chụp vào **Ảnh (Pictures)\Screenshots**.

1. Mở File Explorer, vào **Pictures** → **Screenshots** và tìm ảnh mới nhất.
2. Kéo thả ảnh đó vào thư mục **docs** trong khung Explorer của VS Code (hoặc copy rồi dán).
3. Đổi tên thành **T-01-a11y-tree.png** (chuột phải → Rename).

Mở ảnh trong VS Code để xem lại: phải thấy rõ thanh địa chỉ **127.0.0.1:5500** và cây Accessibility đầy đủ.

Bước 3: Commit ảnh

Trong VS Code, mở Terminal bằng **Ctrl + `** (phím nằm dưới phím Esc). Terminal sẽ mở đúng thư mục dự án. Chạy lần lượt:

```
git add docs/T-01-a11y-tree.png
```

```
git diff --staged --stat
```

Kết quả chỉ được có **1 dòng**, là docs/T-01-a11y-tree.png. Nếu đúng thì commit:

```
git commit -m "docs(a11y): add T-01 landmark tree verification"
```

Bước 4: Kiểm tra lần cuối

```
git log --oneline --stat
```

Bấm **q** để thoát nếu thấy dấu **:**. Bạn phải thấy **4 commit**:

1. docs(a11y): add T-01 landmark tree verification: chỉ có file ảnh

2. docs(spec): mark T-01 done
3. feat(html): semantic landmark tree
4. docs(spec): define component contracts & WBS table

Bạn dán kết quả git log lên đây, mình xác nhận lần cuối giúp bạn.

Bạn

ngoài ra tôi phải làm 1 file report riêng, tôi có thể để ảnh minh chứng lên file report luôn được không, khỏi dùng vs code

Claude

Được. Đề **không yêu cầu** để ảnh minh chứng trong repo Git. Slide 7 chỉ ghi "Verify Landmark Tree", tức là phải kiểm tra, còn để ảnh ở đâu không quy định. Nếu đã có file report riêng thì dán ảnh vào report là hợp lý nhất, và **bạn có thể bỏ qua hẳn Bước 1–3 ở trên** (tạo docs/, commit ảnh).

Cần lưu ý

1. **Phần Git vẫn giữ nguyên.** 3 commit bạn đã làm (docs(spec), feat(html), docs(spec): mark T-01 done) là bắt buộc, không thay được bằng report. Report chỉ là phần bổ sung.
2. **Nếu report là file Word/PDF nộp riêng** (không nằm trong repo): dán ảnh thẳng vào là xong, không đung tới Git.
3. **Nếu muốn để file report vào repo:** commit nó **riêng**, ví dụ docs(report): add Lab 1 report, đừng gộp vào commit code.

Gợi ý nội dung phần Exercise 1 trong report

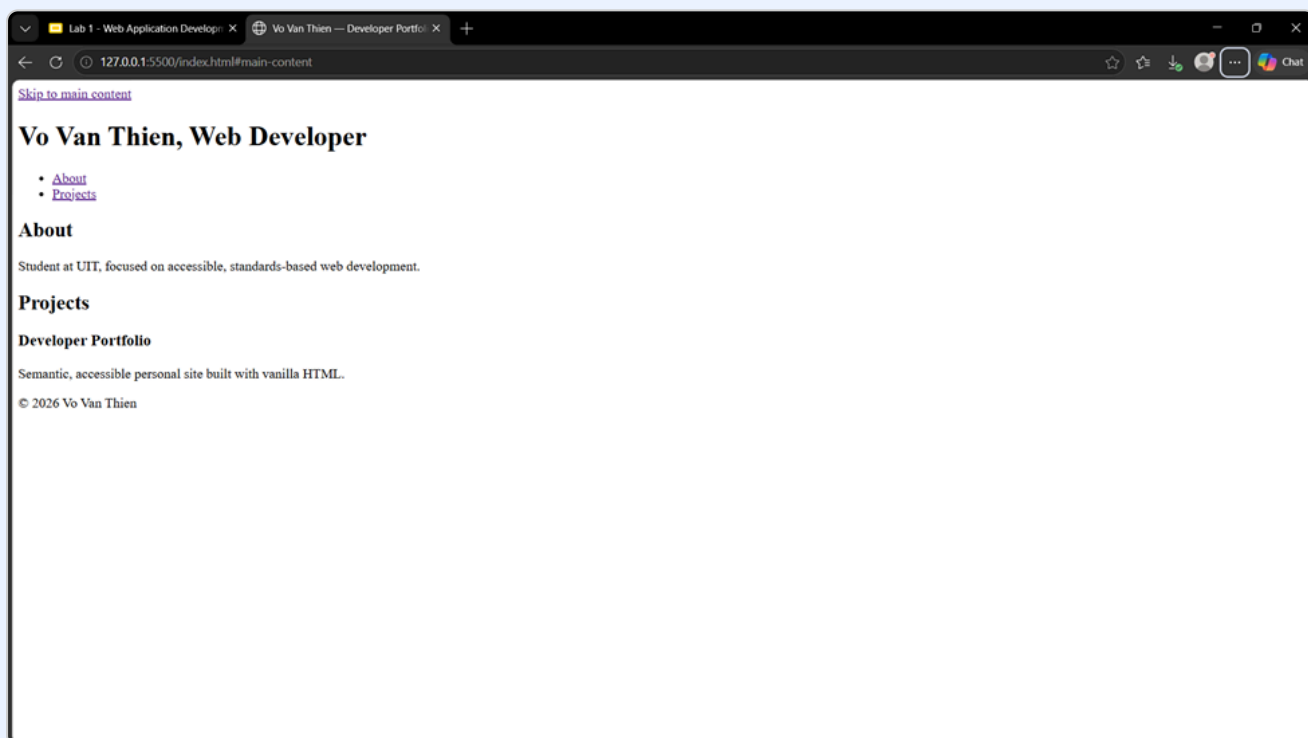
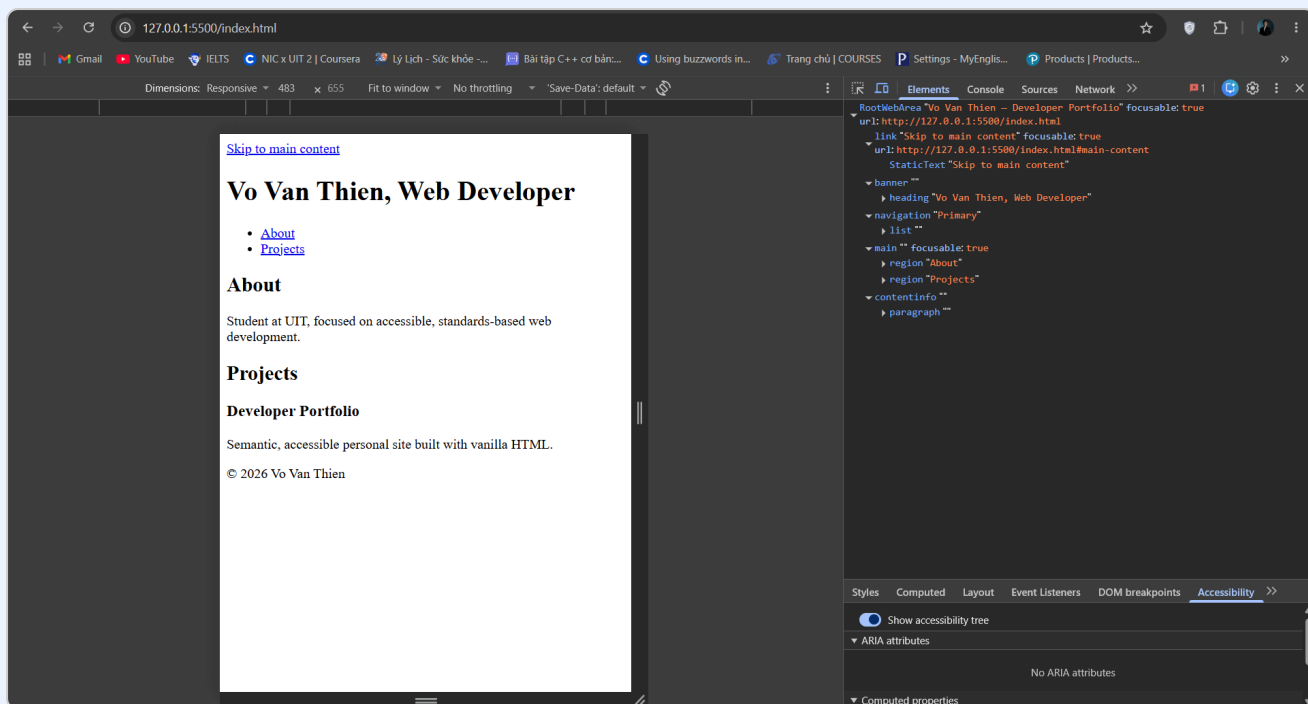
Mỗi ảnh nên có một câu chú thích ngắn nói ảnh chứng minh điều gì:

1. **Mục tiêu:** cây landmark semantic, 0 <div>, skip-link, 1 <h1>.
2. **Contract:** chép sơ đồ landmark từ TASK_DECOMPOSITION.md.
3. **Minh chứng:**
 - Ảnh trang chạy trên 127.0.0.1:5500.
 - **Ảnh cây Accessibility** (ảnh bạn vừa chụp): đủ banner, navigation, main, 2 region, contentinfo.

- Ảnh kết quả `git log --oneline --stat`: các commit atomic, không trộn CSS/JS.
 - (Nên có) Ảnh Console hiện `document.activeElement` là `<main>` sau khi Tab rồi Enter vào skip-link.
4. **Quyết định thiết kế:** `#main-content` thay cho `#main`, `tabindex="-1"`, không dùng `role` thừa, chưa link CSS (copy từ mục *Design Decisions*).
 5. **Sử dụng AI:** công cụ đã dùng, cách kiểm chứng, và lỗi AI mà bạn phát hiện (thiếu `tabindex="-1"`).

Bạn nên hỏi lại giảng viên report cần **định dạng gì** (Word, PDF hay Markdown) và **nộp ở đâu** (cùng repo hay nộp riêng), để làm đúng ngay từ đầu.

Bạn



tôi để 2 ảnh minh chứng này được rồi chứ

Claude

Ảnh 1 (cây Accessibility): đạt  Ảnh có thanh địa chỉ 127.0.0.1:5500, trang web, và cây đủ 6 landmark. Đây là minh chứng chính cho *Verification Gate*.

Ảnh 2 (sau khi bấm skip-link): dùng được nhưng còn yếu. Ảnh chỉ cho thấy URL đã đổi thành #main-content, tức là link nhảy đúng chỗ. Nó **không cho thấy focus đã vào**

`<main>`, mà đó mới là điểm chính của skip-link. Thêm nữa, ảnh này chụp bằng **Edge** (có nút "Chat" của Copilot), còn ảnh 1 chụp bằng Chrome.

Nên bổ sung

1. Ảnh git log --oneline --stat (quan trọng nhất). Exercise 1 chấm nặng phần commit atomic (STEP 4). Hai ảnh hiện tại không chứng minh được điều đó. Chụp terminal sau khi chạy lệnh, thấy đủ 3 commit và mỗi commit chỉ chứa file đúng loại.

2. Thay ảnh 2 bằng ảnh chứng minh focus (nên làm), chụp trong Chrome:

1. Mở trang, bấm vào thanh địa chỉ, rồi **Tab** để focus vào skip-link, rồi **Enter**.
2. Mở **Console** và gõ `document.activeElement`.
3. Chụp sao cho thấy cả URL `#main-content` lẫn kết quả `<main id="main-content" tabindex="-1">`.

Về cờ đỏ 🚩 1 trong ảnh 1

Ảnh 1 vẫn còn biểu tượng 🚩 1. Người chấm kỹ có thể để ý. Bấm vào xem là lỗi gì:

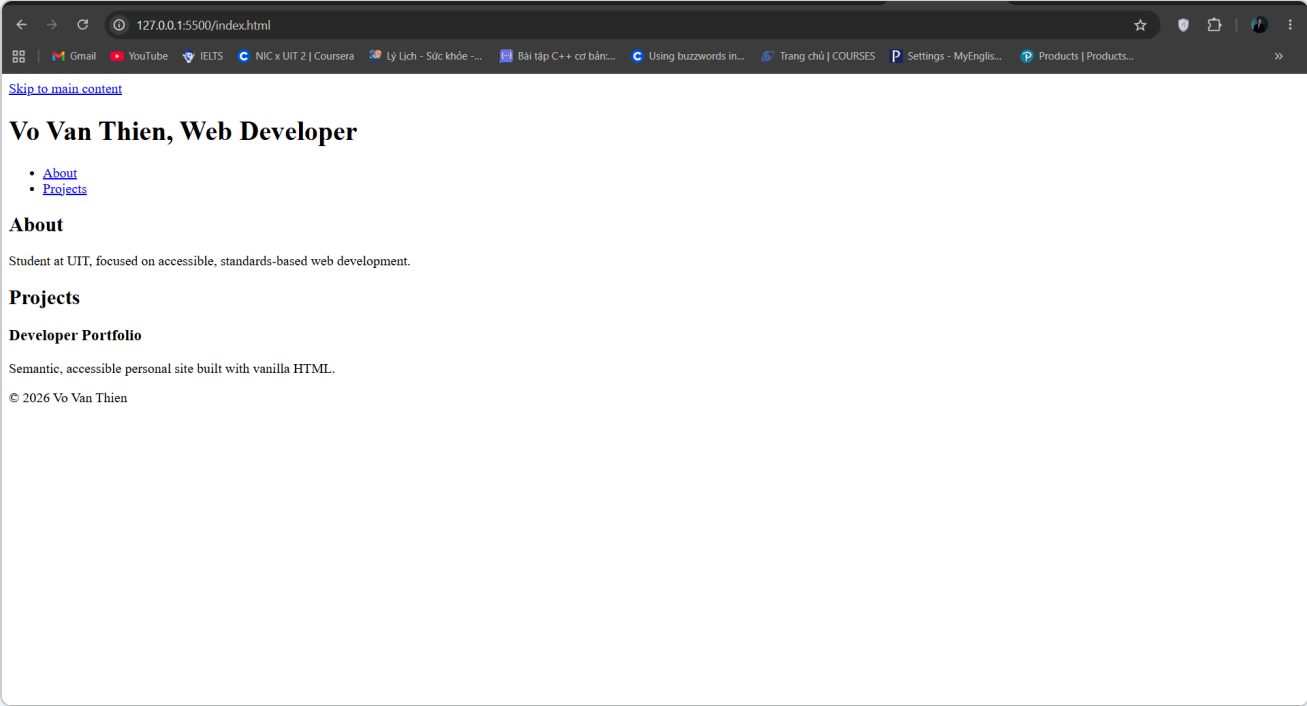
- Nếu là `favicon.ico 404`, ghi một dòng chú thích dưới ảnh: *"Lỗi 1 trong Console là favicon.ico 404 do trình duyệt tự yêu cầu icon; không liên quan đến T-01, sẽ xử lý ở T-02."*
- Nếu là lỗi khác, gửi mình xem trước khi nộp.

Gợi ý chú thích dưới mỗi ảnh

- **Hình 1:** Cây Accessibility trong Chrome DevTools: đủ banner, navigation "Primary", main, region "About", region "Projects", contentinfo; skip-link là phần tử đầu tiên.
- **Hình 2:** Sau khi Tab rồi Enter trên skip-link, focus chuyển vào `<main id="main-content">`.
- **Hình 3:** Lịch sử Git: spec trước code; commit HTML chỉ chứa `index.html`, không trộn CSS/JS.

Có đủ 3 ảnh này thì phần minh chứng Exercise 1 trong report là chắc chắn.

Bạn



thay lại bằng ảnh này nehs

Claude

Ảnh này **thay được cho ảnh "trang chạy trên Live Server"**: chụp bằng Chrome, thấy rõ 127.0.0.1:5500 và trang hiển thị đúng. Nhưng nó **không thay được ảnh minh chứng skip-link**, thậm chí còn yếu hơn ảnh Edge lúc này:

	Ảnh Edge trước	Ảnh Chrome này
Trình duyệt khớp đề	✗ Edge	✓ Chrome
URL đổi thành #main-content (link nhảy đúng chỗ)	✓	✗ chỉ là index.html
Focus vào <main>	✗	✗

Nhìn vào ảnh này, người chấm chỉ thấy trang hiển thị bình thường. **Không có gì chứng minh skip-link hoạt động.**

Cách chụp đúng ảnh skip-link (khoảng 1 phút)

1. Trong tab Chrome này, bấm **F12** và mở tab **Console**.
2. Bấm vào **vùng trắng của trang**, rồi bấm **F5** để tải lại.
3. Bấm **Tab** một lần. Skip-link "Skip to main content" sẽ có viền focus.
4. Bấm **Enter**. URL đổi thành ...index.html#main-content.

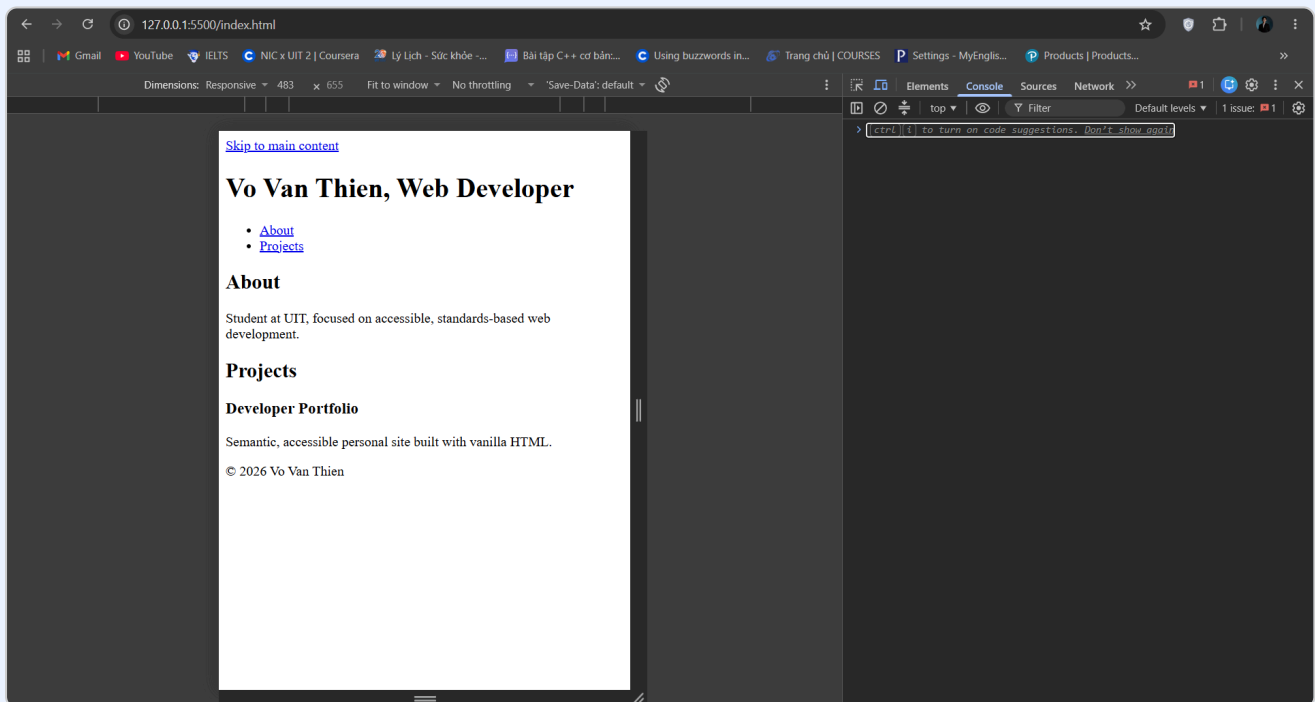
5. Bấm vào ô gõ lệnh của Console, gõ `document.activeElement` rồi Enter.
6. Console phải in ra `<main id="main-content" tabindex="-1">...</main>`.
7. Chụp bằng **Win + Shift + S**, sao cho thấy cả **URL #main-content** lẫn **kết quả trong Console**.

Lưu ý ở bước 5: bấm vào Console sẽ làm focus chuyển sang DevTools. Nhưng `document.activeElement` vẫn trả về phần tử đang được focus *trong trang*, nên kết quả vẫn đúng là `<main>`.

Bộ ảnh cho report

1. **Ảnh này:** trang chạy trên Live Server (Chrome). ✓
2. **Ảnh cây Accessibility** (đã có). ✓
3. **Ảnh skip-link + Console `document.activeElement`.** Cần chụp lại theo các bước trên.
4. **Ảnh git log --oneline --stat.** Chưa có, đây là ảnh quan trọng nhất.

Bạn



này hả

Claude

Chưa phải. Đây mới là bước mở Console. Ảnh hiện tại còn thiếu hai thứ:

- URL vẫn là `index.html`, **chưa có #main-content**, tức là skip-link chưa được bấm.

- Console **trống**, chưa gõ lệnh nào.

Làm tiếp từ màn hình này

1. Bấm vào **vùng trắng của trang** (bên trái, ví dụ chỗ trống dưới dòng © 2026).
2. Bấm **F5** để tải lại trang (DevTools vẫn để mở).
3. Bấm **Tab** một lần. "Skip to main content" sẽ có **viền focus** bao quanh.
4. Bấm **Enter**. URL đổi thành `127.0.0.1:5500/index.html#main-content`.
5. Bấm vào **dòng có dấu >** trong Console (chỗ đang hiện *"ctrl i to turn on code suggestions"*).
6. Gõ đúng lệnh sau rồi Enter:

```
document.activeElement
```

7. Console in ra `<main id="main-content" tabindex="-1">...</main>`.
8. Chụp bằng **Win + Shift + S**, sao cho thấy **URL có #main-content** và **dòng kết quả <main ...>** trong Console.

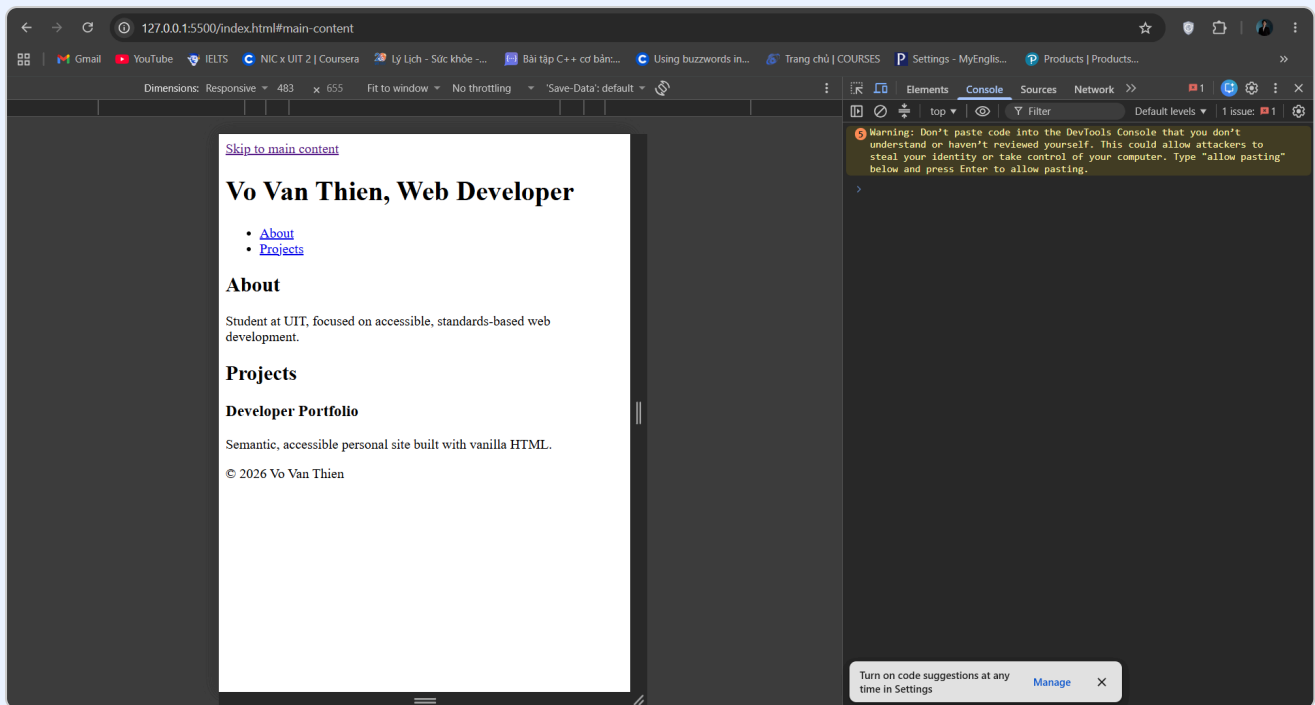
Nếu bước 3 không thấy viền focus: có thể focus đang ở trong DevTools. Bấm lại vào vùng trắng của trang rồi thử Tab lần nữa.

Nếu bước 7 ra <body> thay vì <main>: nghĩa là focus chưa vào main. Làm lại từ bước 1, và nhớ bấm Tab rồi Enter **trước khi** bấm vào Console.

Về "1 issue"

Góc phải thanh Console ghi **"1 issue"**. Đây là **Issue** (gợi ý hoặc cảnh báo của Chrome), không phải lỗi JavaScript. Bấm vào chữ **"1 issue"** để xem nội dung. Thường đó là những gợi ý vô hại, không liên quan đến code T-01. Nếu muốn chắc chắn, chụp nội dung issue gửi mình xem.

Bạn



Claude

Bước skip-link đã đúng: URL đã thành **#main-content** ✓

Cảnh báo vàng xuất hiện vì bạn **dán (paste)** lệnh vào Console. Chrome chặn việc dán vào Console để chống lừa đảo kiểu *self-XSS*: kẻ xấu dụ người dùng dán đoạn mã độc vào Console để chiếm tài khoản. Số **5** trong vòng tròn nghĩa là bạn đã thử dán 5 lần và đều bị chặn.

Cách xử lý: tự gõ lệnh, đừng dán

1. Bấm **X** để đóng thông báo nhỏ "Turn on code suggestions" ở góc dưới.
2. Bấm vào dòng có dấu **>** trong Console.
3. **Tự gõ từng chữ** (không Ctrl+V):

```
document.activeElement
```

4. Bấm **Enter**. Kết quả phải là `<main id="main-content" tabindex="-1">...</main>`.
5. Chụp bằng **Win + Shift + S**, sao cho thấy URL `#main-content` và dòng kết quả.

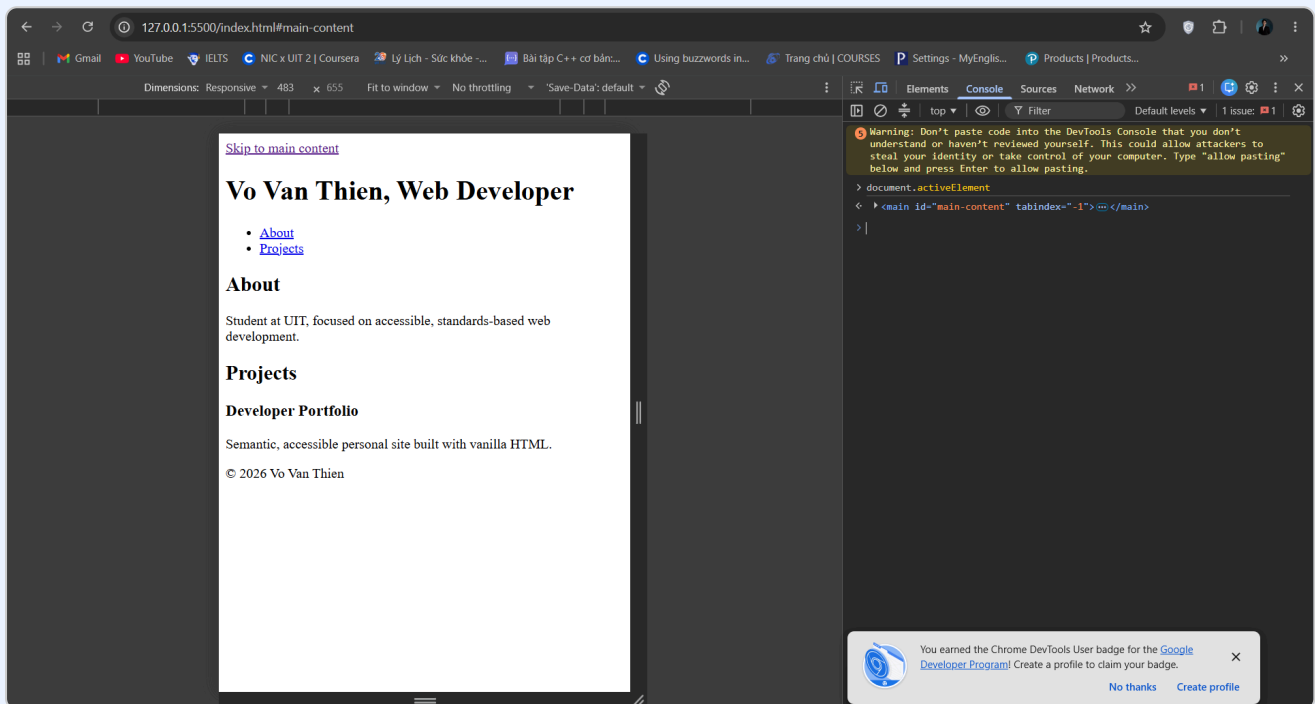
Cảnh báo vàng nằm trong ảnh cũng không sao. Nếu muốn ảnh gọn hơn, bấm biểu tượng  (**Clear console**) ở đầu thanh Console để xóa trước, rồi mới gõ lệnh.

Có nên gõ "allow pasting" không?

Không cần. Lệnh này chỉ có 22 ký tự, tự gõ nhanh hơn. Gõ allow pasting sẽ **tắt lớp bảo vệ đó** cho trình duyệt. Với lệnh bạn hiểu rõ như thế này thì không nguy hiểm, nhưng tốt nhất đừng tập thói quen tắt nó.

Bạn có thể ghi chi tiết này vào phần Security của report. Nó liên quan trực tiếp tới chủ đề XSS trong Lab (slide 2, 21).

Bạn



Claude

Đây đúng là ảnh minh chứng cho skip-link ☒

- URL: 127.0.0.1:5500/index.html#main-content, tức là skip-link đã nhảy đúng đích.
- Console: document.activeElement trả về <main id="main-content" tabindex="-1">, tức là **focus đã vào <main>**.

Ảnh này chứng minh đúng tiêu chí "Tab lần đầu focus vào skip-link; Enter chuyển focus vào <main>".

Để ảnh gọn hơn (không bắt buộc): bấm X để đóng thông báo "You earned the Chrome DevTools User badge" ở góc dưới, rồi chụp lại. Cảnh báo vàng phía trên để nguyên cũng được.

Bộ ảnh cho report

1.  Trang chạy trên Live Server (Chrome).
2.  Cây Accessibility đủ 6 landmark.
3.  Skip-link: URL #main-content và document.activeElement là <main>.
4.  **Ảnh git log --oneline --stat.** Đây là ảnh duy nhất còn thiếu.

Để chụp ảnh 4: mở Terminal trong VS Code (**Ctrl + `**) hoặc PowerShell tại thư mục dự án, rồi chạy:

```
git log --oneline --stat
```

Chụp sao cho thấy đủ 3 commit và file trong từng commit. Nếu thấy dấu : ở cuối thì chụp xong mới bấm **q** để thoát.

Có ảnh này nữa là **phần minh chứng Exercise 1 hoàn chỉnh**.

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git log --oneline --stat
b2c9ecf (HEAD -> main) docs(spec): mark T-01 done TASK_DECOMPOSITION.md | 55
+++++----- 1 file changed, 35
insertions(+), 20 deletions(-) 4622638 feat(html): semantic landmark tree index.html | 42
+++++ 1 file changed, 42
insertions(+) 7231598 docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 38 ++++++
project-rules.md | 7 ++++++ :
```

Claude

Kết quả này đạt yêu cầu  Có 3 commit, đúng thứ tự spec trước code, và mỗi commit chỉ chứa đúng loại file của nó.

Đưa vào report

Có hai cách, chọn một:

- **Chụp ảnh** terminal đang hiện kết quả này (Win + Shift + S).
- **Dán dạng chữ** vào report, đặt trong khung code hoặc font Consolas. Cách này rõ hơn ảnh, người chấm còn copy được.

Mẹo để không bị cắt mất dòng cuối: dấu : cuối kết quả làm dòng 2 files changed, 45 insertions(+) của commit đầu tiên bị ẩn. Chạy lệnh sau để Git in hết ra một lần, không dừng lại:

```
git --no-pager log --oneline --stat
```

Kiểm tra cuối (nếu chưa làm)

Lịch sử Git chỉ cho thấy commit chứa file nào, chưa cho thấy trong index.html có gì. Chạy thêm:

```
git show 4622638 | Select-String -Pattern '<div|style=|<style|<script|onc
```

Không in ra gì là commit HTML sạch hoàn toàn. Bạn có thể ghi kết quả này vào report như một bước kiểm chứng.

Tổng kết Exercise 1

Yêu cầu	Trạng thái
STEP 1: khai báo T-01 trong TASK_DECOMPOSITION.md	✓
STEP 2: contract cây landmark, 0 <div>	✓
STEP 3: skip-link hoạt động	✓ (ảnh activeElement là <main>)
STEP 4: commit atomic feat(html)	✓
Verification gate: cây Accessibility	✓ (ảnh đủ 6 landmark)
Không trộn CSS với HTML	✓

Exercise 1 đã hoàn tất. Khi nộp, nhớ kèm cả thư mục ẩn .git (nếu nộp file nén) hoặc git push (nếu nộp qua GitHub), để giảng viên thấy được lịch sử commit.

Bạn

nếu git push thì trong report chỉ cần kèm theo 3 tấm ảnh thôi ha

Claude

Gần đúng. Nếu đã git push, **lịch sử commit nằm sẵn trên GitHub**, giảng viên tự xem được, nên ảnh git log không còn bắt buộc. Nhưng report cần thêm vài thứ ngoài 3 ảnh.

Report nên có

1. **Link repo GitHub.** Bắt buộc, vì đây là thứ thay cho ảnh git log. Kiểm tra repo để **Public**, hoặc đã mời giảng viên vào nếu là Private. Mở link bằng cửa sổ ẩn danh để chắc người khác xem được.
2. **3 ảnh minh chứng, kèm một câu chú thích cho mỗi ảnh:**
 - Trang chạy trên Live Server.
 - Cây Accessibility đủ 6 landmark.
 - Skip-link: URL #main-content và activeElement là <main>.
3. **Vài dòng giải thích ngắn** (có thể copy từ TASK_DECOMPOSITION.md):
 - Quyết định thiết kế: #main-content thay cho #main, tabindex="-1", không dùng role thừa, chưa link CSS.
 - Cách sử dụng AI và lỗi AI đã phát hiện.

Ảnh git log không bắt buộc nữa, nhưng **kèm thêm vẫn tốt**: người chấm thấy ngay trong report, không cần mở GitHub.

Trước khi push

Nếu repo chưa có trên GitHub:

1. Tạo repo mới trên GitHub, **không** tích *Add README*, để không phải gộp lịch sử.
2. Chạy:

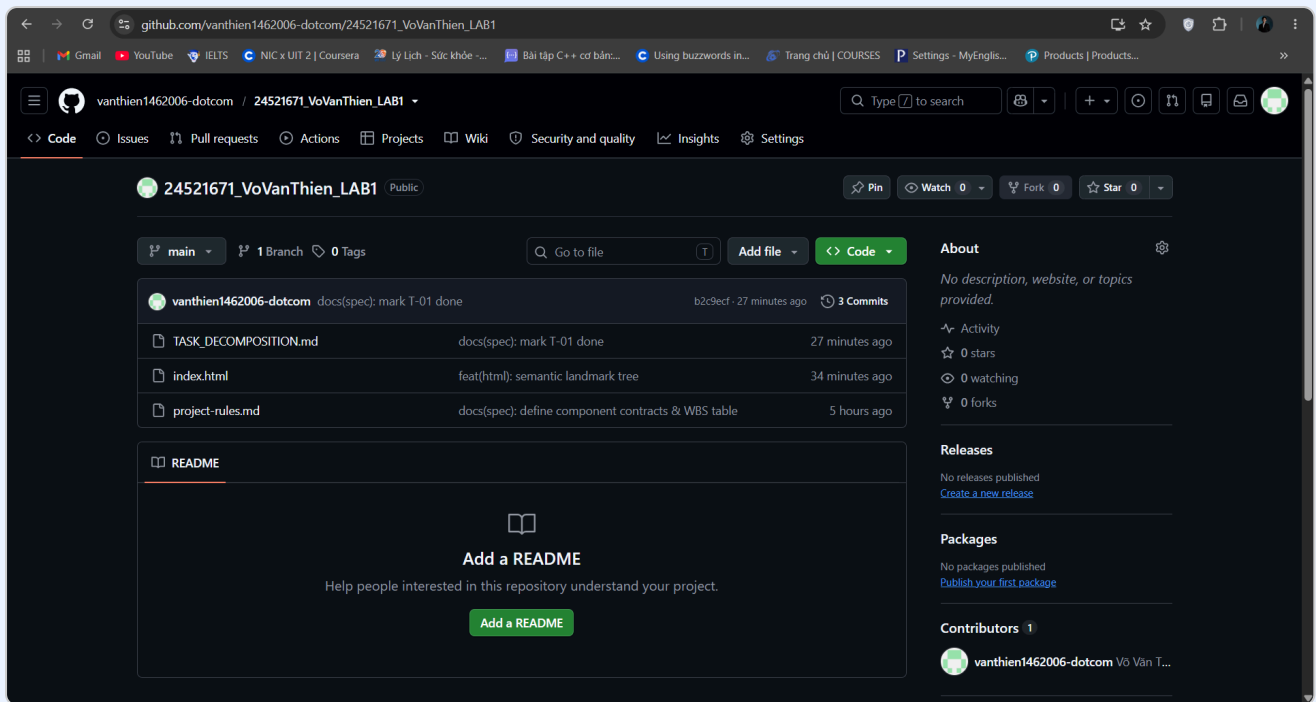
```
git remote add origin https://github.com/<ten-github>/<ten-repo>.git
```

```
git push -u origin main
```

3. Mở repo trên GitHub, vào mục **Commits** để chắc chắn đủ 3 commit.

Đây là gợi ý chung. Báo cáo cần định dạng gì, nộp ở đâu thì nên **xem lại yêu cầu của giảng viên**.

Bạn



thành công rồi chứ

Claude

Push **thành công** ✓

- Repo **Public**, nên giảng viên xem được.
- Có **3 Commits**, khớp với lịch sử trên máy.
- Có đủ 3 file. Mỗi file hiện đúng commit gần nhất đã sửa nó: `index.html` thuộc `feat(html)`, hai file `.md` thuộc các commit `docs(spec)`.

Link ghi vào report:

https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1

⚠️ Đừng bấm "Add a README" trên GitHub

Nút xanh đó sẽ **tạo commit trực tiếp trên GitHub**. Khi đó lịch sử trên GitHub có thêm một commit mà máy bạn không có. Lần `git push` tiếp theo (ở Exercise 2) sẽ **bị từ chối**, và bạn phải `git pull` để gộp, rất dễ làm lịch sử rối.

Nếu muốn có README, hãy tạo **trên máy** rồi commit riêng:

```
git add README.md
```

```
git commit -m "docs: add README"
```

```
git push
```

README không bắt buộc cho Exercise 1, nên bỏ qua cũng được.

Exercise 1 đã hoàn tất: code, commit, minh chứng và repo public đều xong. Khi làm Exercise 2, cứ commit theo từng sub-task (T-02A, T-02B, T-02C) rồi `git push` như bình thường.

Bạn

/expert

Claude

Lệnh **/expert** **không có** trong phiên này nên chưa chạy. Nếu bạn muốn dùng nó, có thể thêm dưới dạng plugin của tổ chức hoặc skill của dự án. Mình không chắc cách cài cụ thể nên không hướng dẫn nữa.

Bạn muốn mình làm gì tiếp? Ví dụ:

- **Rà soát Exercise 1** theo góc nhìn người chấm, tìm chỗ có thể bị trừ điểm khi vấn đáp.
- **Hướng dẫn Exercise 2** (T-02A tokens và reset, T-02B grid, T-02C dark mode) theo cùng quy trình atomic commit.
- **Soạn dàn ý report** cho Exercise 1.
- **Luyện vấn đáp:** mình đóng vai giảng viên và hỏi bạn về code.

Bạn chỉ cần nói phần nào, mình làm tiếp.

Bạn

Bạn có thể nào xuất file link trò chuyện này được không

Claude

Mình đã xuất lại toàn bộ cuộc trò chuyện, gồm cả phần mới từ lần xuất trước đến giờ:

C:\Users\USER\Downloads\session-export-1790953373467.zip (khoảng 6 MB)

File cũ `session-export-1790934449293.zip` chỉ có nửa đầu cuộc trò chuyện. Hãy dùng file mới này.

Về link

Mình **không tạo được link chia sẻ** cho cuộc trò chuyện này. Công cụ của mình chỉ xuất được ra file trên máy bạn, không tải lên đâu cả. Mình cũng không chắc tab Code này có tính năng tạo link công khai hay không.

Nếu giảng viên bắt buộc nộp **link**, có hai cách:

1. **Tải file zip lên Google Drive hoặc OneDrive**, chọn chia sẻ "Bất kỳ ai có link đều xem được", rồi nộp link đó.
2. **Hỏi lại giảng viên** xem có nhận file thay cho link không.

Lưu ý: file zip chứa **toàn bộ** cuộc trò chuyện, gồm cả ảnh chụp màn hình có thanh bookmark, tên tài khoản GitHub và email của bạn. Mở ra xem lại trước khi chia sẻ công khai.

Bạn

làm sao để xem nội dung cuộc trò chuyện trong file zip

Claude

Mình đã làm thêm một bản dễ đọc của cuộc trò chuyện. File gốc trong zip khó xem trực tiếp, nên mình chuyển nó sang trang web.

File gốc trong zip là gì

Bên trong `session-export-...zip` có:

- `transcript.jsonl`: toàn bộ cuộc trò chuyện, nhưng ở **dạng dữ liệu thô JSON**. Mở bằng Notepad sẽ thấy hàng nghìn dòng mã khó đọc, ảnh bị mã hóa thành chuỗi ký tự dài.
- `metadata.json` và vài file nhỏ: thông tin kỹ thuật của phiên.

Bản dễ đọc đã tạo

`C:\Users\USER\Downloads\Lab1-Exercise1-tro-chuyen.html` (khoảng 2 MB)

Cách xem: vào thư mục **Downloads** và **nhấp đúp** file này. Nó mở bằng Chrome hoặc Edge như một trang web:

- Tin nhắn của **Bạn** có nền xanh nhạt, tin nhắn của **Claude** có nền trắng.
- Các ảnh chụp màn hình bạn gửi được hiển thị trực tiếp.
- Bảng và khối code được định dạng sẵn.

Mình đã kiểm tra dữ liệu trong file: đọc được đủ **59 lượt tin nhắn**, xen kẽ đúng giữa bạn và Claude. Mình **chưa mở thử được trên trình duyệt** (trình duyệt của mình không được cấp quyền đọc thư mục Downloads), nên nếu trang hiển thị lỗi thì báo mình.

Lưu ý

- Trang cần **Internet** để định dạng chữ đậm, bảng và code. Khi offline, nội dung vẫn hiện nhưng dưới dạng chữ thô.
- Bản này dừng ở lúc xuất file, nên không có câu hỏi hiện tại của bạn.
- File này cũng chứa **ảnh chụp có tên tài khoản GitHub và thanh bookmark** của bạn. Xem lại trước khi chia sẻ.
- Bản HTML này **chỉ là tệp trên máy bạn**, không có link. Nếu cần link, tải file lên Google Drive hoặc OneDrive rồi bật chia sẻ.

Bạn

hướng dẫn tôi tạo link để đưa vào report nộp cho giáo viên nhé

Claude

Cách chắc chắn nhất: **chuyển cuộc trò chuyện sang PDF, tải lên Google Drive hoặc OneDrive, rồi lấy link chia sẻ.**

Lý do không chia sẻ thẳng file HTML: Google Drive và OneDrive **không hiển thị file .html thành trang web**. Giảng viên bấm vào sẽ thấy mã nguồn hoặc phải tải về. PDF thì xem được ngay trên trình duyệt.

Bước 1: Chuyển sang PDF

1. Vào thư mục **Downloads** và nhấp đúp Lab1-Exercise1-tro-chuyen.html để mở bằng Chrome (cần có mạng để trang định dạng đẹp).
2. Đợi trang tải xong, cuộn xuống cuối để mọi ảnh hiện đủ.
3. Bấm **Ctrl + P**.

4. Ở mục **Destination** (Máy in), chọn **Save as PDF**.
5. Bấm **More settings** và tích **Background graphics**, để giữ màu nền phân biệt tin nhắn của bạn và Claude.
6. Bấm **Save**, đặt tên ví dụ 24521671_VoVanThien_Lab1_AI_Chat.pdf.

Bước 2: Tạo link (chọn một trong hai)

Cách A: Google Drive

1. Mở **drive.google.com** và đăng nhập.
2. Bấm + **New**, chọn **File upload**, rồi chọn file PDF vừa lưu.
3. Chuột phải vào file đã tải lên, chọn **Share**, rồi **Share** lần nữa.
4. Ở mục **General access**, đổi *Restricted* thành **Anyone with the link**, quyền là **Viewer**.
5. Bấm **Copy link**, rồi **Done**.

Cách B: OneDrive (máy bạn đã có sẵn)

1. Kéo file PDF vào một thư mục OneDrive, ví dụ OneDrive\Tài liệu. **Không** để vào thư mục repo 24521671_VoVanThien_LAB1, kéo lần vào Git.
2. Đợi biểu tượng đồng bộ hiện dấu ✓ xanh.
3. Chuột phải vào file, chọn **Share** (biểu tượng đám mây OneDrive).
4. Bấm vào phần cài đặt quyền và chọn **Anyone with the link can view**. Nếu bạn dùng OneDrive của trường, lựa chọn có thể là "*People in [tên trường]*"; nếu giảng viên cùng trường thì cũng được.
5. Bấm **Copy link**.

Bước 3: Kiểm tra link trước khi nộp

Mở **cửa sổ ẩn danh** (Ctrl + Shift + N), dán link vào và Enter:

- Xem được PDF mà **không phải đăng nhập** thì link đạt .
- Nếu thấy "*Request access*" hoặc "*You need permission*", quay lại bước chia sẻ và kiểm tra quyền **Anyone with the link**.

Bước 4: Đưa vào report

Ví dụ cách ghi:

Nhật ký sử dụng AI (Claude): [link Drive/OneDrive] Nội dung: phân tích đề Exercise 1, hướng dẫn quy trình atomic commit, kiểm tra minh chứng. Code `index.html` được tự kiểm chứng bằng DevTools và `git diff`.



Trước khi chia sẻ

File PDF có **ảnh chụp màn hình chứa thanh bookmark, tên tài khoản GitHub và tên các tab** bạn đang mở. Lướt qua một lượt. Nếu có gì riêng tư không muốn giảng viên thấy, nói mình để mình tạo lại bản HTML **không kèm ảnh** trước khi bạn chuyển sang PDF.

Bạn

nếu như tôi dùng cái đó rồi tải lên repo luôn được không

Claude

Được. GitHub **hiển thị PDF ngay trên trình duyệt**, nên giảng viên bấm link là xem được, không cần Drive hay OneDrive. Nhưng cân nhắc mấy điểm này trước.

Ưu và nhược

	Để trong repo	Để trên Drive/OneDrive
Mọi thứ ở một chỗ	✓	✗
Ai xem được	⚠ Bất kỳ ai trên Internet (repo đang Public)	Chỉ người có link
Xóa đi về sau	⚠ Vẫn còn trong lịch sử Git	✓ Xóa là mất
Ảnh hưởng lịch sử commit	Thêm 1 commit, phải tách riêng	Không ảnh hưởng

Điều quan trọng nhất: repo của bạn là **Public**. Ai cũng xem được file PDF, kể cả bạn cùng lớp, và có thể dùng cách làm của bạn. File cũng chứa bookmark, tên tài khoản và các tab bạn đang mở. Nếu thấy ổn với điều đó thì làm tiếp.

Cách làm

1. Trong thư mục repo 24521671_VoVanThien_LAB1, tạo thư mục **docs** (nếu chưa có).
2. Copy file PDF vào docs. Đặt tên **không dấu, không khoảng trắng**, ví dụ docs/AI_CHAT_LOG.pdf.
3. Mở Terminal tại thư mục repo và chạy lần lượt:

```
git add docs/AI_CHAT_LOG.pdf
```

```
git diff --staged --stat
```

Chỉ được có **1 file** là cái PDF này. Nếu đúng thì commit:

```
git commit -m "docs(ai): add AI usage chat log"
```

```
git push
```

Lưu ý: đừng copy file **HTML** hay file **zip** vào repo. HTML không hiển thị được trên GitHub, còn zip chứa dữ liệu thô không cần thiết.

Link đưa vào report

Sau khi push, link sẽ có dạng:

```
https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/blob/m
```

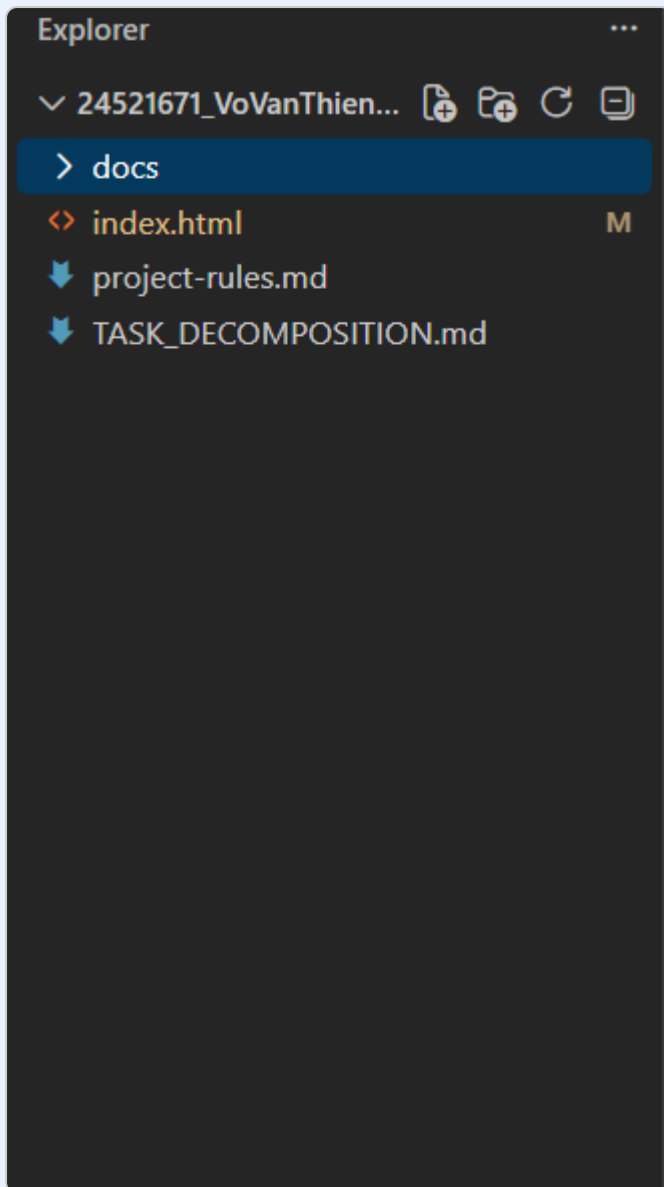
Mở bằng **cửa sổ ẩn danh** để chắc PDF hiển thị được.

Nếu sau này muốn gỡ

Xóa file rồi commit thì file biến mất khỏi trang repo, nhưng **vẫn còn trong lịch sử Git**: ai mở commit cũ vẫn xem được. Gỡ hẳn khỏi lịch sử thì phức tạp và sẽ làm xáo trộn các commit đã chấm. Vì vậy hãy quyết định trước khi push.

Gợi ý: nếu giảng viên chỉ cần xem, còn bạn không muốn công khai, thì dùng **Drive/OneDrive** an toàn hơn. Nếu muốn mọi thứ gọn trong repo và chấp nhận công khai, thì để trong repo.

Bạn



như này à

Claude

docs đã đúng chỗ. Nhưng có một vấn đề cần xử lý **trước khi commit**.

index.html đang có chữ M

Chữ **M** (Modified) nghĩa là **index.html đã bị sửa sau commit feat(html)**, và thay đổi đó chưa được commit. Có thể bạn vô tình gõ hay xóa gì đó, hoặc một extension (Prettier chẳng hạn) đã tự định dạng lại file khi lưu.

Không được commit thay đổi này chung với file PDF. Làm vậy là gom diff.

Xem đã sửa gì:

```
git diff index.html
```

- Dòng **đỏ** (-) là nội dung cũ, dòng **xanh** (+) là nội dung mới.

Sau khi xem, xử lý theo một trong hai trường hợp:

Trường hợp 1: thay đổi do vô tình hoặc không cần (thường gặp nhất). Trả file về đúng bản đã commit:

```
git restore index.html
```

Chữ **M** sẽ biến mất.

Trường hợp 2: bạn cố ý sửa và muốn giữ. Hãy dán kết quả `git diff index.html` lên đây trước. Mình xem giúp, vì nếu đó là sửa HTML thì cần commit riêng với message phù hợp.

Thư mục docs

Mở docs (bấm vào mũi tên >) để chắc file PDF đã nằm bên trong. Nếu có, tên file sẽ hiện **màu xanh lá kèm chữ U** (Untracked, tức là Git chưa theo dõi file đó).

Lưu ý: Git **không theo dõi thư mục rỗng**. Nếu docs chưa có file thì bước `git add` sẽ báo lỗi.

Sau khi index.html đã hết chữ M

```
git status
```

Chỉ được thấy docs/ trong mục *Untracked files*, và **không có** index.html. Sau đó làm tiếp như đã hướng dẫn:

```
git add docs/AI_CHAT_LOG.pdf
```

```
git diff --staged --stat
```

```
git commit -m "docs(ai): add AI usage chat log"
```

```
git push
```

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git add docs/AI_CHAT_LOG.pdf fatal: pathspec 'docs/AI_CHAT_LOG.pdf' did not match any files PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1>
```

Claude

Lỗi này nghĩa là **Git không tìm thấy file AI_CHAT_LOG.pdf trong thư mục docs**. Thường do một trong ba lý do:

1. File PDF **chưa được copy** vào docs (thư mục còn rỗng).
2. File có **tên khác**, ví dụ vẫn là 24521671_VoVanThien_Lab1_AI_Chat.pdf.
3. Tên có **đuôi thừa** như AI_CHAT_LOG.pdf.pdf. Windows hay ẩn đuôi file nên khó nhận ra.

Xem trong docs có gì

```
Get-ChildItem docs
```

- **Không hiện gì:** thư mục rỗng. Copy file PDF từ Downloads (hoặc nơi bạn đã lưu) vào docs, rồi chạy lại lệnh trên.
- **Hiện một file với tên khác:** có hai cách.
 - Đổi tên cho khớp, thay <ten-hien-tai> bằng đúng tên đang hiện:

```
Rename-Item "docs\<ten-hien-tai>" "AI_CHAT_LOG.pdf"
```

- Hoặc giữ tên cũ và dùng đúng tên đó trong `git add`, nhớ đặt trong dấu nháy kép nếu tên có khoảng trắng.

Khi `Get-ChildItem docs` hiện đúng AI_CHAT_LOG.pdf, chạy lại:

```
git add docs/AI_CHAT_LOG.pdf
```

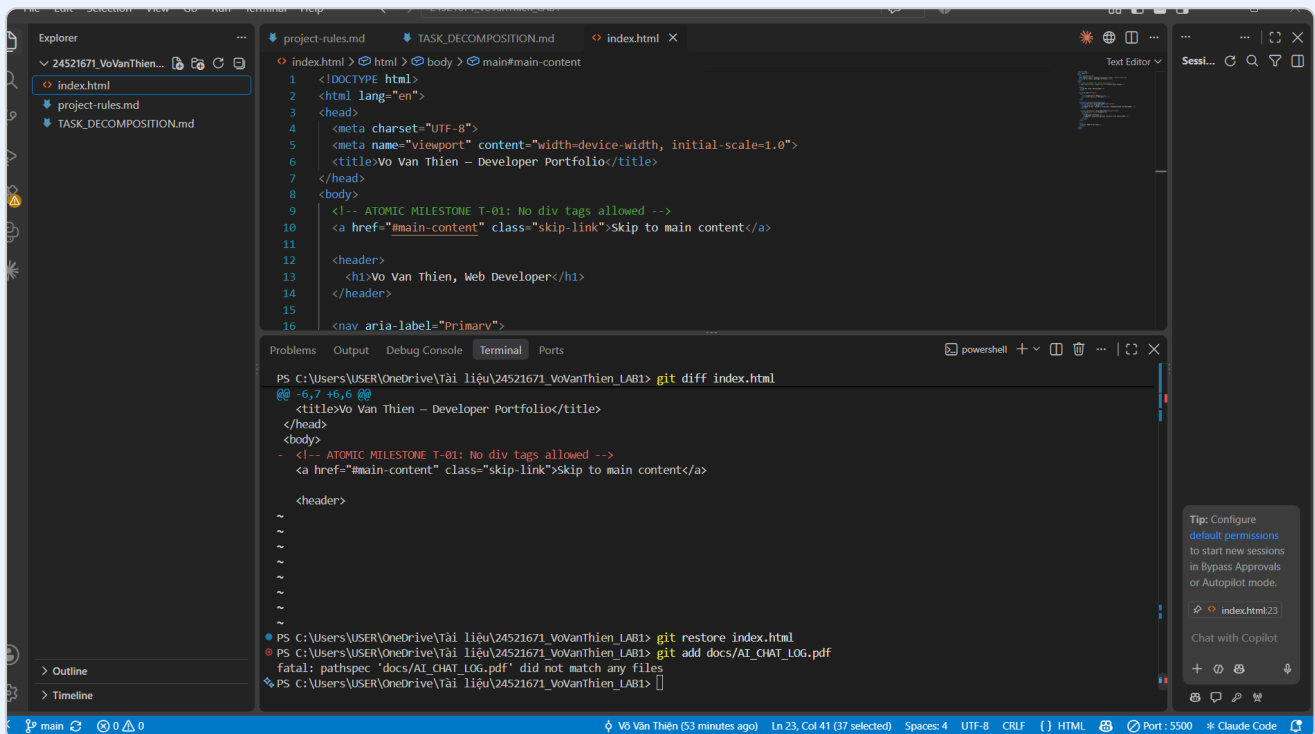
Nhắc lại: index.html còn chữ M không?

Trước khi commit, kiểm tra:

```
git status
```

Nếu `index.html` vẫn nằm trong *Changes not staged for commit*, hãy xử lý trước theo hướng dẫn ở trên: dùng `git diff index.html` để xem, rồi `git restore index.html` nếu không cần thay đổi đó. Chỉ commit PDF khi `git diff --staged --stat` hiện **đúng 1 file**.

Bạn



Hướng dẫn chi tiết lại từ đầu cách úp file đó lên repo

Claude

Trước tiên, hai điểm từ ảnh:

-  git restore index.html đã chạy đúng. Thay đổi lúc trước là **dòng comment** `<!-- ATOMIC MILESTONE ... -->` **bị xóa**, giờ đã khôi phục (dòng 9 trong editor). index.html sạch trở lại.
- Thư mục **docs không còn trong Explorer**, và lỗi pathspec cho thấy file PDF **chưa nằm trong repo**. Làm lại từ đầu như sau.

Bước 1: Tạo file PDF (bỏ qua nếu đã có)

1. Mở thư mục **Downloads** và nhấp đúp Lab1-Exercise1-tro-chuyen.html để mở bằng Chrome.
2. Đợi trang tải xong và cuộn xuống cuối để ảnh hiện đủ.
3. Bấm **Ctrl + P**.
4. Ở **Destination**, chọn **Save as PDF**. Mở **More settings** và tích **Background graphics**.
5. Bấm **Save**. Lưu vào **Downloads** với tên **AI_CHAT_LOG.pdf**.

Mở thư mục Downloads xem file AI_CHAT_LOG.pdf đã có chưa.

Bước 2: Tạo thư mục docs và copy PDF vào

Trong Terminal của VS Code (đang ở đúng thư mục repo), chạy:

```
New-Item -ItemType Directory -Force docs
```

```
Copy-Item "$HOME\Downloads\AI_CHAT_LOG.pdf" "docs\AI_CHAT_LOG.pdf"
```

Nếu lệnh thứ hai báo "*Cannot find path*", nghĩa là bước 1 chưa xong hoặc file lưu ở chỗ khác hay với tên khác. Kiểm tra lại Downloads.

Bước 3: Kiểm tra file đã vào đúng chỗ

```
Get-ChildItem docs
```

Phải thấy AI_CHAT_LOG.pdf, cột *Length* lớn hơn 0.

```
git status
```

Kết quả đúng:

```
Untracked files:
  docs/
```

Không được có index.html trong *Changes not staged for commit*. Nếu thấy index.html, dừng lại và báo mình.

Bước 4: Stage và kiểm tra

```
git add docs/AI_CHAT_LOG.pdf
```

```
git diff --staged --stat
```

Kết quả **chỉ có 1 dòng**:

```
docs/AI_CHAT_LOG.pdf | Bin 0 -> xxxxxx bytes
```

(Bin nghĩa là file nhị phân, tức là PDF. Bình thường.)

Bước 5: Commit và push

```
git commit -m "docs(ai): add AI usage chat log"
```

```
git push
```

Bước 6: Kiểm tra trên GitHub và lấy link

1. Mở repo: https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1
2. Phải thấy thư mục **docs** và số commit tăng lên **4 Commits**.
3. Bấm vào docs, rồi bấm AI_CHAT_LOG.pdf. PDF hiện ngay trên trang.
4. **Copy URL trên thanh địa chỉ** và dán vào report. Link có dạng:

```
https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/blob/main/docs/AI_CHAT_LOG.pdf
```

5. Mở link bằng **cửa sổ ẩn danh** (Ctrl + Shift + N) để chắc người khác xem được.

Lịch sử commit cuối cùng

docs(ai): add AI usage chat log	→ chỉ PDF
docs(spec): mark T-01 done	→ chỉ TASK_DECOMPOSITION
feat(html): semantic landmark tree	→ chỉ index.html
docs(spec): define component contracts & WBS table	→ 2 file .md

Mỗi commit chỉ làm một việc. Nếu bước nào báo lỗi, chụp terminal gửi mình.

Bạn



Lab 1 Exercise 1 - Trò chuyện



Claude

File HTML đã mở được trong trình duyệt  (tiêu đề tab là "Lab 1 Exercise 1 - Trò chuyện").

Trước khi chuyển sang PDF, cuộn qua trang một lượt xem có đúng không:

- Tin nhắn của **Bạn** nền xanh nhạt, của **Claude** nền trắng.
- **Ảnh chụp màn hình** hiện đầy đủ.
- **Bảng và khối code** được định dạng, không phải toàn ký hiệu `|`, `**`, ``` thô.

Nếu thấy toàn ký hiệu thô, có thể trang chưa tải được thư viện định dạng do mất mạng. Kiểm tra Internet rồi bấm **F5**.

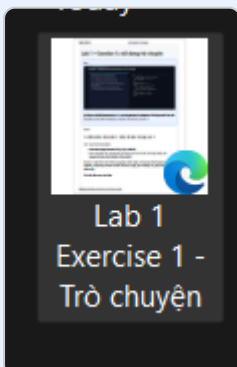
Làm tiếp: lưu thành PDF

1. Cuộn **xuống cuối trang** để mọi ảnh tải xong.
2. Bấm **Ctrl + P**.
3. **Destination:** chọn **Save as PDF**.
4. Mở **More settings** và tích **Background graphics**.
5. Bấm **Save**, lưu vào **Downloads** với tên **AI_CHAT_LOG.pdf**.

Sau đó quay lại VS Code và làm từ **Bước 2** ở hướng dẫn trước (tạo docs, copy PDF vào, rồi `git add, commit, push`).

Nếu trang hiển thị lỗi hoặc bố cục lạ, chụp màn hình gửi mình.

Bạn



Claude

Đây có vẻ là **file PDF vừa lưu**: biểu tượng Edge kèm hình thu nhỏ của trang, vì Windows mặc định mở PDF bằng Edge. Nhưng file đang mang tên mặc định "**Lab 1 Exercise 1 - Trò chuyện**", **không phải** `AI_CHAT_LOG.pdf`. Vì vậy lệnh `Copy-Item` ở bước 2 sẽ không tìm thấy file.

Tên này có dấu tiếng Việt và khoảng trắng, nên dễ gây lỗi khi gõ lệnh và khi tạo link GitHub. **Nên đổi tên trước.**

Bước 1: Đổi tên file

1. Chuột phải vào file, chọn **Rename** (hoặc bấm **F2**).
2. Gõ `AI_CHAT_LOG` rồi Enter. Windows tự giữ đuôi `.pdf`, nên đừng gõ thêm `.pdf` kéo thành `.pdf.pdf`.

Kiểm tra đó đúng là PDF: chuột phải vào file, chọn **Properties**. Dòng *Type of file* phải là **PDF**, không phải **HTML**.

Bước 2: Copy vào repo

Quay lại Terminal trong VS Code và chạy:

```
New-Item -ItemType Directory -Force docs
```

```
Copy-Item "$HOME\Downloads\AI_CHAT_LOG.pdf" "docs\AI_CHAT_LOG.pdf"
```

```
Get-ChildItem docs
```

Phải thấy `AI_CHAT_LOG.pdf`.

Nếu file không nằm trong Downloads (bạn lưu ở chỗ khác): thay vì dùng lệnh, **kéo thả file** từ File Explorer vào thư mục `docs` trong khung Explorer của VS Code.

Bước 3: Commit và push

```
git status
```

Chỉ thấy `docs/` trong *Untracked files*, không có `index.html`.

```
git add docs/AI_CHAT_LOG.pdf
```

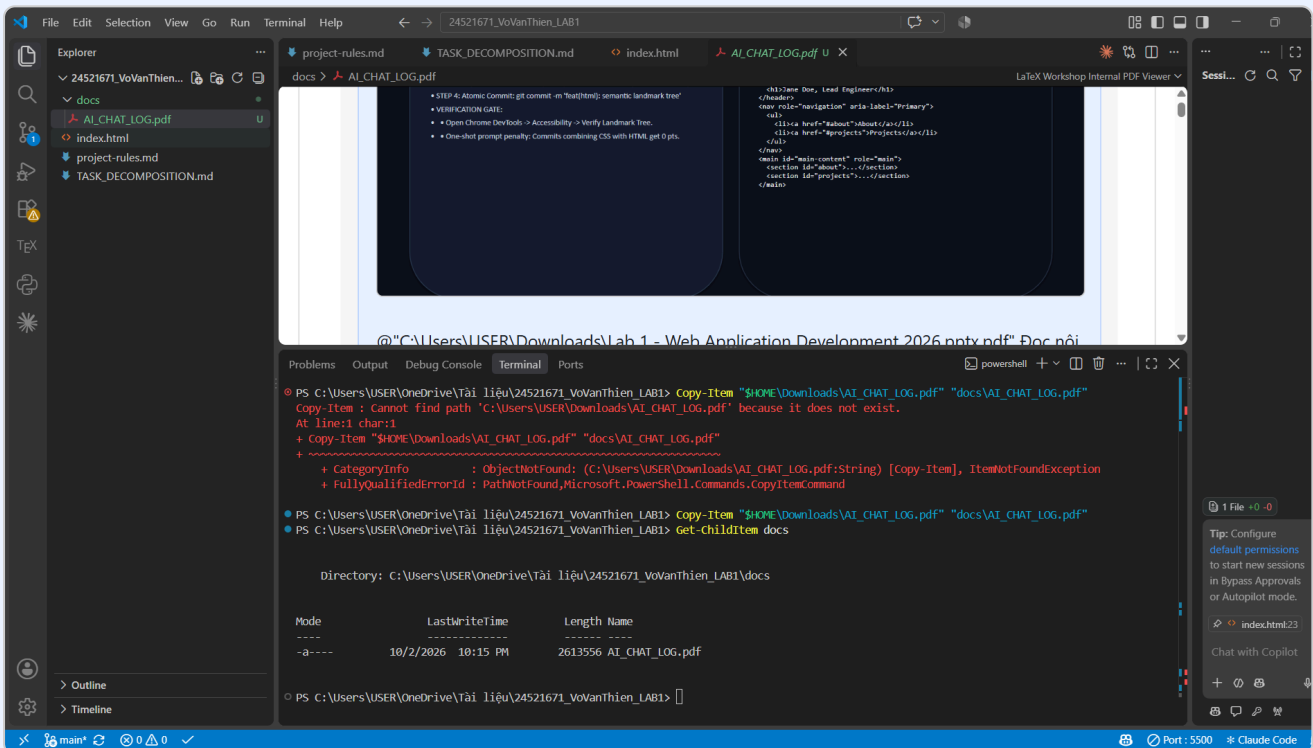
```
git diff --staged --stat
```

```
git commit -m "docs(ai): add AI usage chat log"
```

```
git push
```

Chạy xong, dán kết quả lên đây để mình kiểm tra.

Bạn



Claude

Thành công ✅ File AI_CHAT_LOG.pdf (khoảng 2,6 MB) đã nằm trong docs:

- Explorer hiện docs/AI_CHAT_LOG.pdf màu xanh kèm chữ **U**, tức là file mới, Git chưa theo dõi.
- index.html **không còn chữ M**, tức là sạch.
- PDF mở được trong VS Code, nội dung hiển thị đúng.

Lần chạy Copy-Item đầu tiên báo lỗi vì lúc đó file chưa đổi tên. Lần chạy thứ hai đã thành công.

Làm tiếp

```
git status
```

Kết quả đúng: chỉ có docs/ trong *Untracked files*, không có file nào khác bị sửa.

```
git add docs/AI_CHAT_LOG.pdf
```

```
git diff --staged --stat
```

Phải thấy đúng 1 dòng: docs/AI_CHAT_LOG.pdf | Bin 0 -> 2613556 bytes.

```
git commit -m "docs(ai): add AI usage chat log"
```

```
git push
```

Lưu ý trước khi push

Trong PDF có **ảnh các slide bài Lab của giảng viên** (bạn thấy một phần ngay trong VS Code). Repo của bạn đang **Public**, nên push lên là **ai cũng xem được tài liệu giảng dạy đó**. Một số giảng viên không muốn slide bị công khai.

- Nếu không chắc, dùng **Drive/OneDrive** với link chia sẻ (an toàn hơn), hoặc **chuyển repo sang Private** rồi thêm giảng viên làm collaborator.
- Nếu vẫn muốn để trong repo Public, đó là quyết định của bạn. Chỉ cần nhớ rằng sau khi push, file sẽ **còn mãi trong lịch sử Git**.

Push xong, dán kết quả lên đây để mình kiểm tra lần cuối.

Bạn

đã push lên github thì lấy link như nào

Claude

Có hai cách lấy link.

Cách 1: Lấy từ trang GitHub (chắc chắn nhất)

1. Mở repo: https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1
2. Bấm vào thư mục **docs**.
3. Bấm vào file **AI_CHAT_LOG.pdf**. PDF sẽ hiện ngay trên trang.
4. **Copy URL trên thanh địa chỉ** (bấm vào thanh địa chỉ, Ctrl + A, Ctrl + C).

Cách 2: Tự ghép link

Nếu push đúng như hướng dẫn, link sẽ là:

```
https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/blob/m
```

Kiểm tra trước khi đưa vào report

Mở **cửa sổ ẩn danh** (Ctrl + Shift + N), dán link vào và Enter:

- PDF hiện ra mà **không cần đăng nhập** thì link đạt ☒.
- Thấy **404 – Page not found** nghĩa là push chưa thành công, tên file khác, hoặc repo đang Private. Chụp màn hình gửi mình.

Link nên ghi vào report

Nội dung	Link
Repo (code và lịch sử commit)	https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1
Lịch sử commit	https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/commits/main
Nhật ký sử dụng AI	https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/blob/main/docs/AI_CHAT_LOG.pdf

Link **lịch sử commit** giúp giảng viên xem ngay các commit atomic mà không cần tìm. Trên trang đó phải thấy **4 commit**, mới nhất là docs(ai): add AI usage chat log.

Bạn

https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/commit/03cbfafb4c9745041ccd8fbfb42fafdf7c6c15d
như này đúng k

Claude

Link này **dùng được nhưng chưa phải loại tốt nhất** cho mục đích này.

Link bạn gửi là gì

.../commit/03cbfaf... là **link của một commit**. Bấm vào, giảng viên sẽ thấy:

- Tên commit (docs(ai): add AI usage chat log).
- Danh sách file thay đổi trong commit (docs/AI_CHAT_LOG.pdf).
- Với file PDF, GitHub thường **không hiển thị nội dung** ở trang commit mà chỉ ghi "Binary file not shown". Giảng viên phải bấm thêm mới đọc được.

Nên dùng link nào

Mục đích	Loại link	Bấm vào thấy gì
Cho giảng viên đọc nhật ký AI	.../blob/main/docs/AI_CHAT_LOG.pdf	✅ PDF hiện ngay
Chứng minh commit tách riêng	.../commit/03cbfaf... (link của bạn)	Commit chỉ chứa 1 file PDF
Xem toàn bộ lịch sử	.../commits/main	Cả 4 commit

Với mục "**Nhật ký sử dụng AI**" trong report, nên dùng link **xem file**:

https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/blob/main/docs/AI_CHAT_LOG.pdf

Để lấy link này: trên trang commit bạn đang mở, bấm vào tên file docs/AI_CHAT_LOG.pdf, rồi chọn ... → **View file**. Hoặc làm theo Cách 1 ở trên (repo → docs → AI_CHAT_LOG.pdf) rồi copy URL.

Link commit của bạn **không bỏ đi**: cứ ghi thêm vào report làm minh chứng commit tách riêng. Nhớ mở mọi link bằng **cửa sổ ẩn danh** để chắc người khác xem được.

Bạn

Tiếp tục hướng dẫn chi tiết để tôi hoàn thành EX2 nhé

Claude

Mình đã đọc lại slide 13 (Exercise 2) và các slide liên quan (15 reset, 16 Flexbox, 17 Grid, 19 tokens, 22 theme toggle). Dưới đây là toàn bộ quy trình, giữ đúng kỷ luật đã dùng ở Exercise 1.

Exercise 2: Enterprise Developer Portfolio

1. Đề yêu cầu gì

3 sub-task bắt buộc, mỗi cái một commit:

Sub-task	Commit message (theo slide)
T-02A: Tokens & Reset	feat(css): tokens & reset
T-02B: 2D Grid Layout	feat(css): responsive grid
T-02C: Theme Engine	feat(js): dark mode engine

Ràng buộc (contract):

- Trạng thái theme lưu **duy nhất** bằng `localStorage`, key là `'theme'`.
- Màu sắc chỉ dùng qua **biến CSS**. **Không có mã hex trong các rule**; hex chỉ được khai báo trong khối token.
- Hiệu năng: **CLS = 0**, **LCP < 2.0s** khi giả lập mạng chậm.

Tiêu chí chấm:

- Commit trộn CSS và JS thì **0 điểm**.
- Hiển thị gọn ở **375px**, không có thanh cuộn ngang.
- Độ tương phản đạt **WCAG 2.2 AA** ($\geq 4.5:1$).
- Không có lỗi Console** khi bật/tắt theme.
- Điều hướng được hoàn toàn bằng **Tab và Enter**.

- **Vấn đáp:** giảng viên sửa một token CSS, bạn phải sửa lại trong 60 giây.

2. Kế hoạch commit

Ở Ex1 có luật **commit trộn CSS với HTML = 0 điểm**. Mà Ex2 cần **sửa HTML** (thêm nút theme, thêm thẻ project, thêm <link>/<script>). Vì vậy phần HTML phải **tách thành commit riêng**:

#	Commit	File duy nhất
C1	docs(spec): define T-02 contracts	TASK_DECOMPOSITION.md
C2	feat(html): add theme toggle, project cards & asset links	index.html
C3	feat(css): tokens & reset	style.css (chỉ phần A)
C4	feat(css): responsive grid	style.css (thêm phần B)
C5	feat(js): dark mode engine	theme.js
C6	docs(spec): mark T-02 done	TASK_DECOMPOSITION.md

⚠ **Sửa lại một ý mình từng nói:** trước đây mình bảo thêm <link> "cùng commit với style.css". Đọc chặt luật slide 7 thì như vậy là **trộn HTML với CSS**. Cách an toàn là đưa mọi thay đổi HTML vào commit C2.

3. C1: Viết contract trước

Thêm vào **cuối** TASK_DECOMPOSITION.md (giữ nguyên phần T-01):

```
---

## T-02 – Enterprise Developer Portfolio

### Sub-tasks
| ID      | Task                                | File       | Commit                                |
|-----|-----|-----|-----|
| T-02H   | HTML hooks for T-02                | index.html | feat(html): add theme to             |
| T-02A   | Design tokens & reset               | style.css  | feat(css): tokens & rese             |
| T-02B   | 2D responsive grid                 | style.css  | feat(css): responsive gr            |
| T-02C   | Theme engine                       | theme.js   | feat(js): dark mode engi           |

### Token Contract (style.css)
```

- Colors: --color-bg, --color-surface, --color-text, --color-text-muted, --color-accent, --color-border
- Spacing: --space-xs, --space-sm, --space-md, --space-lg
- Other: --radius, --content-width, --font-body
- Light values in `:root`, dark values in `:root[data-theme="dark"]`.
- Hex codes allowed ONLY inside these two token blocks.

Theme Contract (theme.js)

- State: ``<html data-theme="light|dark">``
- Persistence: localStorage key `theme`, values `light` | `dark`
- First visit: follow OS via `prefers-color-scheme`
- Toggle: ``<button id="theme-toggle" aria-pressed="true|false">``
- localStorage blocked → theme still works, no console error

Layout Contract

- Mobile-first, 375px baseline, no horizontal scroll
- Header & nav: Flexbox; project list: Grid
`repeat(auto-fit, minmax(min(100%, 280px), 1fr))`
- No new `<div>`

Acceptance Criteria

- [] No horizontal scroll at 375px
- [] All text contrast $\geq 4.5:1$ in both themes
- [] Zero console errors while toggling theme 10+ times
- [] Theme persists after reload (localStorage `theme`)
- [] Full keyboard flow: Tab → skip-link → toggle → nav links; Enter/Space
- [] CLS = 0, LCP < 2.0s (Lighthouse / throttled network)
- [] Each commit touches one file type only

AI Prompts (one per sub-task)

- T-02A: "Follow project-rules.md. Write ONLY style.css tokens + reset per Token Contract. No layout, no JS."
- T-02B: "Using ONLY existing tokens, add layout: flex header/nav and grid.project-grid per Layout Contract. No new colors, no JS."
- T-02C: "Write ONLY theme.js per Theme Contract. Vanilla ES6+, const by default, no innerHTML, wrap localStorage in try/catch."

```
git add TASK_DECOMPOSITION.md
```

```
git diff --staged --stat
```

```
git commit -m "docs(spec): define T-02 contracts"
```

4. C2: Sửa index.html (chỉ HTML)

Thay toàn bộ index.html bằng nội dung sau (vẫn 0 <div>):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vo Van Thien – Developer Portfolio</title>
  <link rel="icon" href="data:,">
  <link rel="stylesheet" href="style.css">
  <script src="theme.js"></script>
</head>
<body>
  <!-- ATOMIC MILESTONE T-01: No div tags allowed -->
  <a href="#main-content" class="skip-link">Skip to main content</a>

  <header class="site-header">
    <h1>Vo Van Thien, Web Developer</h1>
    <button type="button" id="theme-toggle" class="theme-toggle" aria-pre
  </header>

  <nav class="site-nav" aria-label="Primary">
    <ul class="nav-links">
      <li><a href="#about">About</a></li>
      <li><a href="#projects">Projects</a></li>
    </ul>
  </nav>

  <main id="main-content" tabindex="-1">
    <section id="about" aria-labelledby="about-title">
      <h2 id="about-title">About</h2>
      <p>Student at UIT, focused on accessible, standards-based web develo
    </section>
```

```
<section id="projects" aria-labelledby="projects-title">
  <h2 id="projects-title">Projects</h2>
  <ul class="project-grid">
    <li>
      <article class="project-card">
        <h3>Developer Portfolio</h3>
        <p>Semantic, accessible personal site built with vanilla HTML</p>
      </article>
    </li>
    <li>
      <article class="project-card">
        <h3>Theme Engine</h3>
        <p>Light and dark themes driven by CSS custom properties and</p>
      </article>
    </li>
    <li>
      <article class="project-card">
        <h3>Drum Kit</h3>
        <p>Keyboard-driven audio pads using data attributes and the k</p>
      </article>
    </li>
  </ul>
</section>
</main>

<footer>
  <p>&copy; 2026 Vo Van Thien</p>
</footer>
</body>
</html>
```

Những gì thay đổi và lý do:

Thay đổi	Lý do
<link rel="icon" href="data:,">	Chặn lỗi favicon.ico 404 (cờ đỏ 🚩 lúc trước), để đạt tiêu chí <i>zero console errors</i>
<link rel="stylesheet">	Nối CSS (file sẽ có ở C3)

Thay đổi	Lý do
<script src="theme.js"> trong <head>, không defer	Script chạy trước khi trang vẽ , gán theme ngay, nên không bị "chớp" sáng rồi mới tối
<button aria-pressed>	Nút bật/tắt có trạng thái, đọc được bằng trình đọc màn hình (slide 14)
<ul class="project-grid"> với 3 <article>	Grid cần nhiều thẻ để thấy hiệu quả; danh sách là thẻ semantic, không cần <div>

Lưu ý: từ commit này tới khi có C3/C5, Console sẽ báo 404 cho style.css và theme.js. Điều đó **bình thường** và tự hết khi tạo đủ file.

```
git add index.html
```

```
git diff --staged | Select-String -Pattern '<div|style=|<style|onclick='
```

Không in ra gì thì commit:

```
git commit -m "feat(html): add theme toggle, project cards & asset links"
```

5. C3: style.css phần A (tokens & reset)

Tạo file **style.css**, **chỉ** với nội dung này:

```
/* ===== T-02A: Design tokens ===== */
:root {
  color-scheme: light;
  --color-bg: #ffffff;
  --color-surface: #f1f5f9;
  --color-text: #0f172a;
  --color-text-muted: #475569;
  --color-accent: #0369a1;
  --color-border: #94a3b8;

  --space-xs: 0.5rem;
  --space-sm: 1rem;
  --space-md: 1.5rem;
  --space-lg: 2.5rem;
  --radius: 8px;
```

```
--content-width: 64rem;
--font-body: system-ui, -apple-system, "Segoe UI", sans-serif;
}

:root[data-theme="dark"] {
  color-scheme: dark;
  --color-bg: #0f172a;
  --color-surface: #1e293b;
  --color-text: #f1f5f9;
  --color-text-muted: #cbd5e1;
  --color-accent: #38bdf8;
  --color-border: #64748b;
}

/* ===== T-02A: Modern reset & base ===== */
*,
*::before,
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-body);
  line-height: 1.6;
  background: var(--color-bg);
  color: var(--color-text);
}

h1 {
  font-size: clamp(1.5rem, 4vw + 0.75rem, 2.25rem);
  line-height: 1.2;
}

a {
  color: var(--color-accent);
}

:focus-visible {
```

```
outline: 3px solid var(--color-accent);
outline-offset: 2px;
}

[tabindex="-1"]:focus {
  outline: none;
}

.skip-link {
  position: absolute;
  top: var(--space-xs);
  left: var(--space-xs);
  padding: var(--space-xs) var(--space-sm);
  background: var(--color-bg);
  border-radius: var(--radius);
  transform: translateY(-200%);
}

.skip-link:focus {
  transform: translateY(0);
}
```

Màu đã được kiểm tra độ tương phản (mình tính theo công thức WCAG):

Cặp màu	Light	Dark
Chữ chính trên nền	17.85 : 1	16.30 : 1
Chữ phụ trên nền thẻ	6.92 : 1	9.85 : 1
Link/accent trên nền thẻ	5.42 : 1	6.83 : 1

Tất cả đều $\geq 4.5:1$.

Kiểm tra trước khi commit:

- Mở Live Server. Font đã đổi và skip-link **ẩn đi, chỉ hiện khi bấm Tab**.
- Hex chỉ nằm trong 2 khối token:

```
Select-String -Path style.css -Pattern '[0-9a-fA-F]{3,8}\b'
```

Mọi dòng in ra phải thuộc `:root { ... }` hoặc `:root[data-theme="dark"] { ... }`.

```
git add style.css
```

```
git diff --staged --stat
```

```
git commit -m "feat(css): tokens & reset"
```

6. C4: style.css phần B (layout và grid)

Thêm vào cuối style.css (giữ nguyên phần A):

```
/* ===== T-02B: Page frame (no wrapper divs) ===== */
.site-header,
.site-nav,
main,
footer {
  width: min(100% - 2 * var(--space-sm), var(--content-width));
  margin-inline: auto;
}

/* ===== T-02B: 1D layout – Flexbox ===== */
.site-header {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: var(--space-sm);
  padding-block: var(--space-md);
}

.nav-links {
  display: flex;
  flex-wrap: wrap;
  gap: var(--space-md);
  list-style: none;
  padding-block: var(--space-xs);
  border-block-end: 1px solid var(--color-border);
}

.theme-toggle {
```

```
font: inherit;
color: var(--color-text);
background: var(--color-surface);
border: 2px solid var(--color-accent);
border-radius: var(--radius);
padding: var(--space-xs) var(--space-sm);
cursor: pointer;
}

.theme-toggle[aria-pressed="true"] {
  background: var(--color-accent);
  color: var(--color-bg);
}

/* ===== T-02B: 2D layout – Grid ===== */
main section {
  padding-block-start: var(--space-lg);
}

h2 {
  margin-block-end: var(--space-sm);
}

.project-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 280px), 1fr));
  gap: var(--space-md);
  list-style: none;
}

.project-card {
  height: 100%;
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius);
  padding: var(--space-md);
}

.project-card p {
  color: var(--color-text-muted);
}
```

```

}

footer {
  padding-block: var(--space-lg);
  color: var(--color-text-muted);
}

```

Cần hiểu để vấn đáp:

- `width: min(100% - 2rem, 64rem)`: nội dung rộng tối đa 64rem và luôn chứa lề 1rem mỗi bên. Không cần `<div class="container">`.
- `minmax(min(100%, 280px), 1fr)`: giống slide 17, nhưng có `min(100%, ...)` để màn hình **hẹp hơn 280px** cũng không bị tràn ngang.
- `auto-fit`: tự chia cột theo chiều rộng, **không cần media query** (1 cột ở 375px, 2–3 cột trên desktop).
- Phần này **không thêm màu mới**, chỉ dùng token đã có.

Kiểm tra 375px: F12, bật Device Toolbar (**Ctrl+Shift+M**), gõ chiều rộng **375**. Rồi gõ trong Console:

```
document.documentElement.scrollWidth > document.documentElement.clientWidth
```

Phải ra **false** (không có cuộn ngang).

```
git add style.css
```

```
git diff --staged --stat
```

```
git commit -m "feat(css): responsive grid"
```

7. C5: theme.js (theme engine)

Tạo file **theme.js**:

```

// T-02C: Theme engine – contract: localStorage key 'theme' = 'light' | '
const STORAGE_KEY = 'theme';
const root = document.documentElement;

function readSavedTheme() {

```

```
try {
  const saved = localStorage.getItem(STORAGE_KEY);
  return saved === 'light' || saved === 'dark' ? saved : null;
} catch {
  return null;
}
}

function saveTheme(theme) {
  try {
    localStorage.setItem(STORAGE_KEY, theme);
  } catch {
    // Storage blocked (private mode / browser policy): theme still works
  }
}

function getInitialTheme() {
  return readSavedTheme()
    ?? (window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light');
}

function applyTheme(theme) {
  root.dataset.theme = theme;
  const toggle = document.querySelector('#theme-toggle');
  if (toggle) {
    toggle.setAttribute('aria-pressed', String(theme === 'dark'));
  }
}

// Runs in <head> before first paint → no flash of the wrong theme.
applyTheme(getInitialTheme());

document.addEventListener('DOMContentLoaded', () => {
  const toggle = document.querySelector('#theme-toggle');
  if (!toggle) return;

  applyTheme(root.dataset.theme);

  toggle.addEventListener('click', () => {
    const next = root.dataset.theme === 'dark' ? 'light' : 'dark';
    saveTheme(next);
    applyTheme(next);
  });
});
```

```

    applyTheme(next);
    saveTheme(next);
  });
  });

```

Khác slide 22 ở đâu, và vì sao (rất dễ bị hỏi khi vấn đáp):

Slide 22	Code này	Lý do
<code>body.classList.toggle('dark-theme')</code>	<code><html data-theme="dark"></code>	Script chạy trong <code><head></code> , lúc đó <code><body></code> chưa tồn tại; <code><html></code> thì đã có
Gọi <code>localStorage</code> trực tiếp	Bọc <code>try/catch</code>	Một số trình duyệt hoặc chế độ chặn storage sẽ ném lỗi , vi phạm <i>zero console errors</i>
Không xử lý lần đầu vào trang	Đọc <code>prefers-color-scheme</code>	Lần đầu vào trang thì theo theme của hệ điều hành (slide 19)
	<code>const</code> cho mọi biến, không dùng <code>innerHTML</code>	Đúng <code>project-rules.md</code>

Kiểm tra:

1. Bấm nút: trang chuyển tối, chữ trên nút giữ nguyên "Dark mode" và nút đổi sang trạng thái nhấn.
2. **F5**: vẫn giữ theme đã chọn.
3. F12, tab **Application**, mục **Local storage**, chọn `http://127.0.0.1:5500`: có key **theme**, giá trị dark hoặc light.
4. Bấm nút **10 lần trở lên**: Console **không có dòng đỏ**.
5. Bàn phím: Tab đi qua skip-link, nút theme, About, Projects. **Enter** hoặc **Space** trên nút đều đổi được theme.

```
git add theme.js
```

```
git diff --staged --stat
```

```
git commit -m "feat(js): dark mode engine"
```

8. Kiểm tra tổng (trước C6)

Tiêu chí	Cách kiểm tra	Đạt khi
375px	Device Toolbar ở 375 + lệnh <code>scrollWidth</code>	false
Tương phản	F12, chọn một đoạn chữ, trong Styles bấm ô màu: hiện <i>Contrast ratio</i>	≥ 4.5 với ✓ ở cả 2 theme
Console	Bấm toggle nhiều lần	Không có dòng đỏ
CLS / LCP	F12, tab Lighthouse , chọn <i>Mobile</i> , bấm <i>Analyze</i>	CLS = 0, LCP < 2.0s
Bàn phím	Chỉ dùng Tab, Enter, Space	Làm được mọi thao tác
Hex	Lệnh Select-String ở bước C3	Chỉ nằm trong khối token

Về mạng chậm: trong tab **Network**, chọn throttling "**Fast 3G**". Chrome bản mới có thể đặt tên khác ("**3G**" hoặc "*Slow 4G*"), chọn mức gần nhất. Trang không có ảnh và dùng font hệ thống, nên CLS = 0 và LCP rất thấp.

9. C6: Cập nhật tài liệu

Đánh dấu [x] cho các tiêu chí **đã thật sự kiểm tra**. Thêm mục *Design Decisions* ghi 4 điểm khác slide ở bảng mục 7, và mục *AI Usage Log* như ở Ex1.

```
git add TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): mark T-02 done"
```

```
git push
```

10. Chuẩn bị vấn đáp "sửa token trong 60 giây"

Giảng viên có thể:

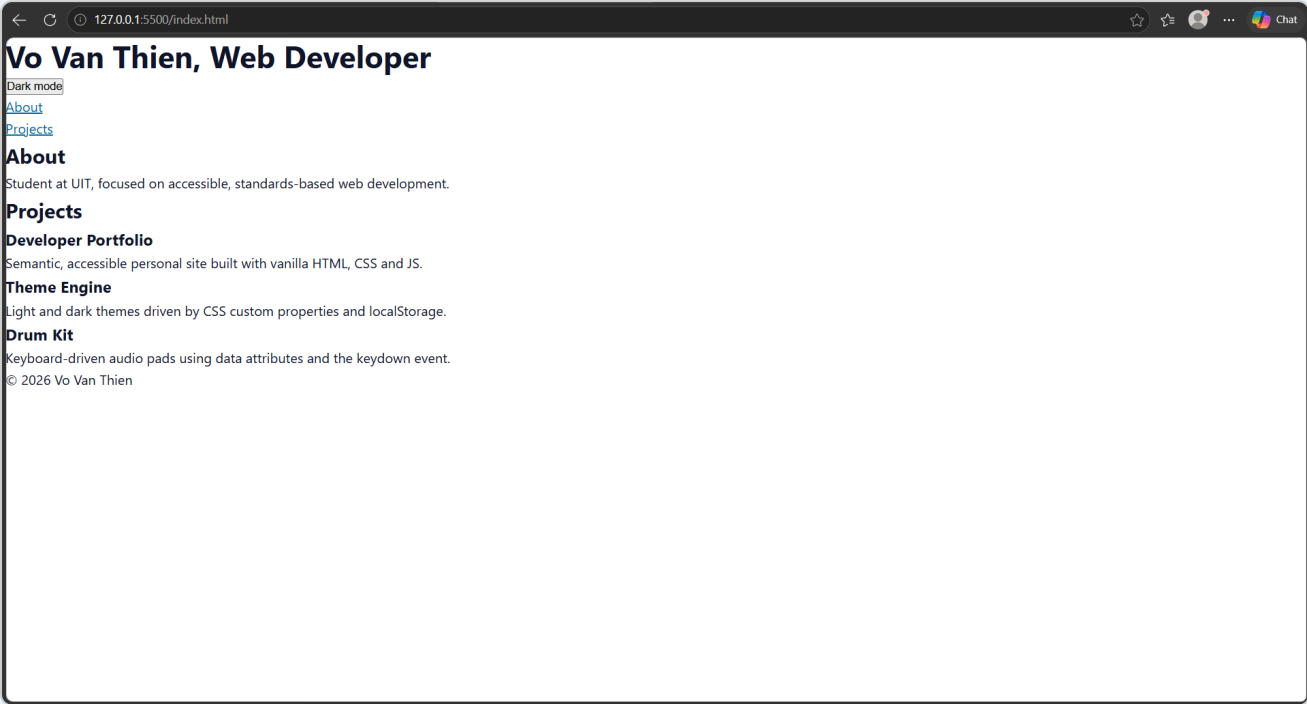
- **Đổi `--color-accent` thành màu nhạt** (ví dụ xám) để link không đủ tương phản. Cách sửa: mọi màu nằm ở **2 khối token đầu `style.css`**, nên chỉ sửa đúng 1 dòng đó.
- **Đổi key `'theme'` thành `'color-mode'`**. Cách sửa: chỉ sửa hằng số `STORAGE_KEY` (1 dòng).

- **Đổi 280px sang container query.** Cách sửa: thêm `container-type: inline-size` cho `#projects`, rồi dùng `@container` thay cho `auto-fit`.

Bạn **hiểu được vì sao mỗi thứ nằm ở một chỗ duy nhất** thì sẽ qua phần này. Nên đọc kỹ từng dòng code trước khi commit, đừng chỉ copy.

Làm xong mỗi commit, bạn có thể dán kết quả `git log --oneline --stat` lên đây để mình kiểm tra trước khi đi tiếp.

Bạn



đang ở c3

Claude

Đúng như mong đợi ở **C3** . Trang trông "trần" và dính sát mép trái là **bình thường**, vì phần bố cục chưa có (C4 mới thêm).

Những gì ảnh xác nhận phần A đã chạy

Quan sát	Do rule nào
Font đổi sang font hệ thống (không còn Times New Roman)	<code>body { font-family: var(--font-body) }</code>

Quan sát	Do rule nào
Mất hết khoảng cách mặc định, chữ sát mép trái	Reset margin: 0; padding: 0
Danh sách mất dấu chấm đầu dòng	Reset padding: 0 đẩy dấu chấm ra ngoài khung nhìn. C4 sẽ đặt list-style: none
Link màu xanh đậm	a { color: var(--color-accent) }
Skip-link biến mất	.skip-link { transform: translateY(-200%) }
h1 nhỏ lại một chút	font-size: clamp(...)
Nút "Dark mode" vẫn là nút mặc định	Đúng, nút được style ở C4

Kiểm tra nhanh trước khi commit C3

1. Skip-link hiện khi Tab: bấm vào trang, F5, rồi **Tab**. "Skip to main content" phải hiện ở góc trên bên trái.

2. Hex chỉ nằm trong khối token:

```
Select-String -Path style.css -Pattern '#[0-9a-fA-F]{3,8}\b'
```

Phải ra **13 dòng**, tất cả là các dòng `--color-...: #xxxxxx;` (6 dòng light + 6 dòng dark + ...). Cụ thể: 6 dòng trong `:root` và 6 dòng trong `:root[data-theme="dark"]`, tổng **12 dòng**. Không được có hex nào trong rule như `body`, `a`, `.skip-link`.

3. Console: lúc này **còn một lỗi 404 cho theme.js**, vì file chưa tạo. Đó là lỗi dự kiến, sẽ hết ở C5. Nếu thấy lỗi khác, gửi mình xem.

Commit C3

```
git add style.css
```

```
git diff --staged --stat
```

Chỉ có `style.css`.

```
git commit -m "feat(css): tokens & reset"
```

Sau đó chuyển sang **C4**: thêm phần B vào **cuối** style.css. Khi đó trang sẽ có lề hai bên, header nằm ngang, nav nằm ngang, và các thẻ project thành lưới. Chụp màn hình gửi mình nếu muốn kiểm tra.

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> Select-String -Path style.css -Pattern '[0-9a-fA-F]{3,8}\b'
```

```
style.css:4: --color-bg: #ffffff; style.css:5: --color-surface: #f1f5f9; style.css:6: --color-text: #0f172a; style.css:7: --color-text-muted: #475569; style.css:8: --color-accent: #0369a1; style.css:9: --color-border: #94a3b8; style.css:22: --color-bg: #0f172a; style.css:23: --color-surface: #1e293b; style.css:24: --color-text: #f1f5f9; style.css:25: --color-text-muted: #cbd5e1; style.css:26: --color-accent: #38bdf8; style.css:27: --color-border: #64748b;
```

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1>
```

Claude

Đạt ☒ Có đúng **12 dòng**, tất cả là khai báo token:

- Dòng **4–9**: 6 token light trong :root.
- Dòng **22–27**: 6 token dark trong :root[data-theme="dark"].

Không có mã hex nào trong các rule như body, a hay .skip-link, nên đã đúng contract *"zero hardcoded hex codes in rules"*.

(Ở tin nhắn trước mình ghi nhầm "13 dòng" rồi mới sửa thành 12. **12 là con số đúng.**)

Commit C3

```
git add style.css
```

```
git diff --staged --stat
```

Chỉ có style.css. Rồi commit:

```
git commit -m "feat(css): tokens & reset"
```

Tiếp theo: C4

1. Mở `style.css`, **kéo xuống cuối file** (sau `.skip-link:focus { ... }`).
2. Dán **toàn bộ phần B** (từ `/* ===== T-02B: Page frame ...` tới `footer { ... }`).
Không xóa phần A.
3. Lưu file và xem lại trên trình duyệt. Trang phải có:
 - Lề hai bên.
 - Tên và nút "Dark mode" nằm **cùng hàng** (màn hình hẹp thì xuống dòng).
 - About và Projects **nằm ngang**, có đường kẻ dưới.
 - Ba thẻ project có nền xám nhạt, viền bo góc, xếp **thành lưới 3 cột** trên màn hình rộng.
4. Kiểm tra ở 375px (Ctrl+Shift+M, chỉnh chiều rộng 375): thẻ xếp thành **1 cột**. Gõ trong Console:

```
document.documentElement.scrollWidth > document.documentElement.clientWidth
```

Kết quả phải là `false`.

5. Chạy lại lệnh Select-String tìm hex. Vẫn phải ra **đúng 12 dòng như cũ**, vì phần B không thêm màu nào.

Chụp màn hình (desktop và 375px) gửi mình nếu muốn kiểm tra trước khi commit C4.

Bạn

gõ chiều rộng ở đâu

Claude

Ô nhập chiều rộng nằm trên **thanh Device Toolbar**. Thanh này chỉ xuất hiện khi bật chế độ giả lập thiết bị.

Bước 1: Bật Device Toolbar

1. Bấm **F12** để mở DevTools.
2. Bấm **Ctrl + Shift + M**. Hoặc bấm biểu tượng **điện thoại và máy tính bảng**  ở góc trên bên trái DevTools, ngay cạnh mũi tên chọn phần tử và trước chữ *Elements*.

Khi bật, phía trên trang web hiện một thanh như trong các ảnh bạn chụp trước đây:

Dimensions: Responsive ▾ 483 × 655 Fit to window ▾ No throttling

Bước 2: Nhập chiều rộng

1. Đảm bảo ô đầu tiên đang là **Responsive** (nếu đang là tên máy như "iPhone...", bấm vào và chọn **Responsive**).
2. Bấm vào **ô số đầu tiên**, ngay sau chữ *Responsive* (trong ảnh trước của bạn là **483**). Đó là **chiều rộng**.
3. Xóa số cũ, gõ **375** rồi bấm **Enter**.
4. Ô thứ hai (sau dấu ×) là chiều cao. Để nguyên hoặc gõ 812.

Trang sẽ co lại rộng đúng 375px.

Cách nhanh hơn: bấm vào **Responsive ▾** và chọn **iPhone SE** (375 × 667) hoặc **iPhone 12 Pro** (390). Nhưng tự gõ **375** sẽ đúng chính xác yêu cầu của đề.

Bước 3: Kiểm tra không có cuộn ngang

1. Chuyển sang tab **Console**.
2. **Tự gõ** lệnh (Chrome chặn việc dán vào Console):

```
document.documentElement.scrollWidth > document.documentElement.clientWidth
```

3. Kết quả **false** là đạt.

Kiểm tra xong, bấm **Ctrl + Shift + M** lần nữa để tắt giả lập và quay về giao diện bình thường.

Bạn

qua tab consosole\

Claude

Cách mở tab **Console** trong DevTools:

Mở Console

Cách 1: ở hàng tab trên cùng của DevTools (*Elements · Console · Sources · Network...*), bấm chữ **Console**.

Cách 2: đang ở tab khác thì bấm **Esc**. Console sẽ hiện thành khung nhỏ ở phía dưới, không cần rời tab hiện tại.

Cách 3: đóng DevTools và bấm **Ctrl + Shift + J**. DevTools mở thẳng vào tab Console.

Device Toolbar (chiều rộng 375) **vẫn giữ nguyên** khi chuyển sang Console, không bị tắt.

Gõ lệnh

1. Bấm vào **dòng cuối cùng có dấu >** trong Console. Con trỏ nhấp nháy ở đó.
2. **Tự gõ** (không dán, vì Chrome chặn dán):

```
document.documentElement.scrollHeight > document.documentElement.clientWidth
```

Mẹo: gõ `document.documentElement.sc` rồi bấm **Tab** để Chrome tự hoàn thành tên dài.

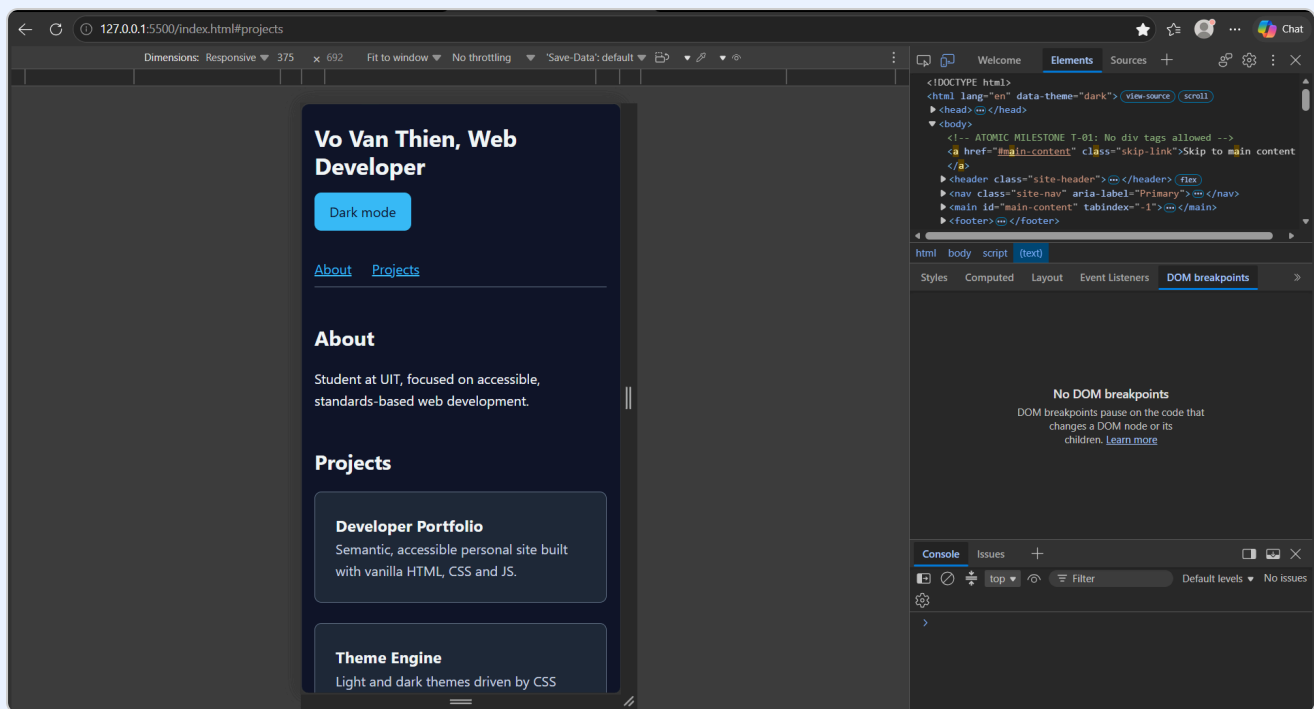
3. Bấm **Enter**.

Đọc kết quả

- **false:** không có cuộn ngang ☒ Đạt tiêu chí 375px.
- **true:** có phần tử tràn ngang. Chụp màn hình gửi mình để tìm nguyên nhân.

Nếu Console đầy chữ khó nhìn, bấm biểu tượng  (**Clear console**) ở đầu thanh Console để xóa trước khi gõ.

Bạn



tab appli đầu

Claude

Tab **Application** đang bị ẩn vì DevTools hẹp. Bạn cũng đang dùng **Edge** (có tab *Welcome*), nên vị trí hơi khác Chrome.

Cách mở tab Application

Cách 1 (nhANH NHẤT): bấm vào khung DevTools, nhấn **Ctrl + Shift + P**, gõ **application**, rồi chọn **Show Application** và Enter.

Cách 2: bấm dấu + ở hàng tab trên cùng (ngay sau *Sources*). Trong danh sách hiện ra, chọn **Application**.

Xem localStorage

1. Ở cột bên trái của tab Application, tìm mục **Storage** và mở **Local storage**.
2. Bấm **http://127.0.0.1:5500**.
3. Bảng bên phải phải có:

Key	Value
theme	dark

4. Bấm nút "Dark mode" trên trang. Value phải đổi thành `light`; bấm lần nữa thì về dark. Nếu bảng không tự cập nhật, bấm biểu tượng ↻ (Refresh) phía trên bảng.

Từ ảnh của bạn, mọi thứ đang chạy tốt

- Chiều rộng **375**: header, nav, các thẻ project đều gọn trong khung, không thấy tràn ngang.
- `<html ... data-theme="dark">`: **theme engine đã hoạt động** (nhìn ở khung Elements).
- Nút "Dark mode" đang ở trạng thái **đã nhấn** (nền xanh).
- Thẻ project xếp **1 cột** ở 375px, đúng như auto-fit.
- Console: **"No issues"**, không có lỗi đỏ.

Kiểm tra lại các commit

Ảnh cho thấy bạn đã làm xong cả **C4 (grid)** và **C5 (theme.js)**. Hãy chắc rằng chúng là **2 commit riêng**:

```
git log --oneline --stat -5
```

Kết quả đúng (mới nhất ở trên):

```
feat(js): dark mode engine      → chỉ theme.js
feat(css): responsive grid      → chỉ style.css
feat(css): tokens & reset       → chỉ style.css
feat(html): add theme toggle... → chỉ index.html
docs(spec): define T-02 contracts → chỉ TASK_DECOMPOSITION.md
```

Nếu bạn chưa commit C4 mà đã tạo theme.js, vẫn tách được vì CSS và JS nằm ở hai file khác nhau. Commit lần lượt từng file:

```
git add style.css
```

```
git commit -m "feat(css): responsive grid"
```

```
git add theme.js
```

```
git commit -m "feat(js): dark mode engine"
```

Tuyệt đối không `git add` . rồi commit một lần, vì như vậy là trộn CSS với JS và bị **0 điểm** theo slide 13.

Bạn dán kết quả `git log` lên đây để mình kiểm tra.

Bạn

giờ hãy giúp tôi hoàn thành các bước screen short kết quả của ex2 này nhé

Claude

Dưới đây là bộ ảnh minh chứng cho Exercise 2. Mỗi ảnh ứng với **một tiêu chí chấm** của slide 13. Nên dùng **Chrome** và **cửa sổ ẩn danh** (Ctrl + Shift + N) để không có extension chen vào.

Chuẩn bị: mở `http://127.0.0.1:5500/index.html` trong Chrome ẩn danh, rồi bấm **F12**.

Tổng quan

#	Ảnh	Chứng minh tiêu chí	Mức độ
1	Desktop, theme sáng	Giao diện, grid nhiều cột	Bắt buộc
2	Desktop, theme tối	Theme engine hoạt động	Bắt buộc
3	375px + Console false	Không cuộn ngang ở 375px	Bắt buộc
4	localStorage key theme	Lưu trạng thái đúng contract	Bắt buộc
5	Console sạch sau khi bấm toggle nhiều lần	Zero console errors	Bắt buộc
6	Tỉ lệ tương phản	WCAG AA $\geq 4.5:1$	Bắt buộc
7	Focus bàn phím trên nút toggle	Điều hướng bằng Tab/Enter	Bắt buộc
8	Lighthouse	CLS = 0, LCP < 2s	Bắt buộc
9	Terminal: hex chỉ trong token	Không hardcode hex	Nên có
10	<code>git log --stat</code>	Commit atomic, không trộn CSS/JS	Nên có (hoặc dùng link GitHub)

Ảnh 1 và 2: Desktop sáng và tối

1. Tắt Device Toolbar nếu đang bật (**Ctrl + Shift + M**). Có thể đóng DevTools (F12) để trang rộng hết cỡ.
2. Nếu đang ở theme tối, bấm "Dark mode" để về **sáng**.
3. Chụp **ảnh 1**: thấy URL, header, nav, và **3 thẻ project nằm ngang** (lưới nhiều cột).
4. Bấm "Dark mode" để chuyển **tối**, rồi chụp **ảnh 2** ở cùng góc nhìn.

Chú thích: "Giao diện desktop ở hai theme. Lưới .project-grid tự chia 3 cột bằng `repeat(auto-fit, minmax(...))`."

Ảnh 3: 375px không cuộn ngang

1. F12, bấm **Ctrl + Shift + M**, ở ô chiều rộng gõ **375** rồi Enter.
2. Cuộn trang tới phần Projects để thấy thẻ xếp **1 cột**.
3. Mở **Console** (bấm Esc để hiện khung Console phía dưới), **tự gõ**:

```
document.documentElement.scrollWidth > document.documentElement.clientWidth
```

4. Chụp sao cho thấy **chữ 375** trên thanh Device Toolbar, trang đã co lại, và kết quả **false**.

Chú thích: "Ở 375px không có cuộn ngang (`scrollWidth > clientWidth` là false)."

Ảnh 4: localStorage

1. Bấm **Ctrl + Shift + P**, gõ application, chọn **Show Application**.
2. Bên trái chọn **Local storage**, rồi **http://127.0.0.1:5500**.
3. Chụp sao cho thấy bảng có **Key = theme** và **Value = dark** (hoặc light), kèm nút toggle trên trang ở trạng thái tương ứng.
4. (Nên làm thêm) **F5** để tải lại trang. Theme vẫn giữ nguyên, chụp thêm một ảnh để chứng minh **trạng thái được lưu lại**.

Chú thích: "Theme lưu trong localStorage với key theme, giữ nguyên sau khi tải lại trang."

Ảnh 5: Console sạch khi bật/tắt theme

1. Mở tab **Console** và bấm  (**Clear console**).
2. Ở ô **Default levels** ▾, đảm bảo **Errors** và **Warnings** đều được tích.
3. Bấm nút "Dark mode" **10 lần trở lên**.

4. Chụp Console **trống, không có dòng đỏ**, kèm biểu tượng ở góc phải không có cờ đỏ (*No issues*).

Chú thích: "Bật/tắt theme 10+ lần, Console không có lỗi."

Ảnh 6: Tỷ lệ tương phản

Cách A: Lighthouse (gộp luôn với ảnh 8, xem bên dưới). Mục **Accessibility** không báo lỗi "*Background and foreground colors do not have a sufficient contrast ratio*" là đạt.

Cách B: soi trực tiếp (rõ ràng hơn, nên làm ở cả 2 theme):

1. Ở tab **Elements**, bấm biểu tượng **mũi tên chọn phần tử** (góc trên trái DevTools), rồi bấm vào đoạn chữ trong một thẻ project.
2. Ở khung **Styles** bên phải, tìm dòng `color: var(--color-text-muted)` và bấm vào **ô vuông màu nhỏ** cạnh nó.
3. Bảng chọn màu hiện ra, có dòng **Contrast ratio** kèm số (ví dụ **6.92**) và dấu **✓✓** (đạt AA và AAA).
4. Chụp bảng đó. Làm tương tự với link **"About"** (`--color-accent`), rồi chuyển theme tối và lặp lại.

Chú thích: "Tỷ lệ tương phản $\geq 4.5:1$ ở cả hai theme (WCAG 2.2 AA)."

Ảnh 7: Điều hướng bằng bàn phím

1. Tắt DevTools hoặc chuyển về desktop. Bấm vào thanh địa chỉ.
2. Bấm **Tab** nhiều lần: skip-link, rồi **nút "Dark mode"**, rồi About, rồi Projects.
3. Khi focus đang ở **nút "Dark mode"**, chụp ảnh: phải thấy **viền focus màu xanh** bao quanh nút.
4. (*Nên làm thêm*) Bấm **Enter** để đổi theme, chụp lại để thấy theme đã đổi mà **không dùng chuột**.

Chú thích: "Toàn bộ thao tác thực hiện được bằng Tab và Enter; viền focus hiển thị rõ."

Ảnh 8: Lighthouse (CLS và LCP)

1. Tắt Device Toolbar (Lighthouse tự giả lập thiết bị).
2. Mở tab **Lighthouse** (nếu không thấy: Ctrl + Shift + P, gõ lighthouse).
3. Chọn **Mode: Navigation**, **Device: Mobile**, tích **Performance** và **Accessibility**.
4. Bấm **Analyze page load** và đợi khoảng 20 giây.
5. Chụp phần **Performance** sao cho thấy rõ **Largest Contentful Paint < 2.0 s** và **Cumulative Layout Shift = 0**. Chụp thêm điểm **Accessibility**.

Lighthouse Mobile đã **tự giả lập mạng chậm và CPU yếu**, nên kết quả đủ thay cho việc tự chọn "Fast 3G".

Chú thích: "Lighthouse (Mobile, mạng giả lập chậm): LCP x.x s < 2.0 s, CLS 0."

Ảnh 9 và 10: Terminal

Trong Terminal của VS Code:

```
Select-String -Path style.css -Pattern '#[0-9a-fA-F]{3,8}\b'
```

Chụp kết quả: **12 dòng**, tất cả là khai báo `--color-...` (**ảnh 9**).

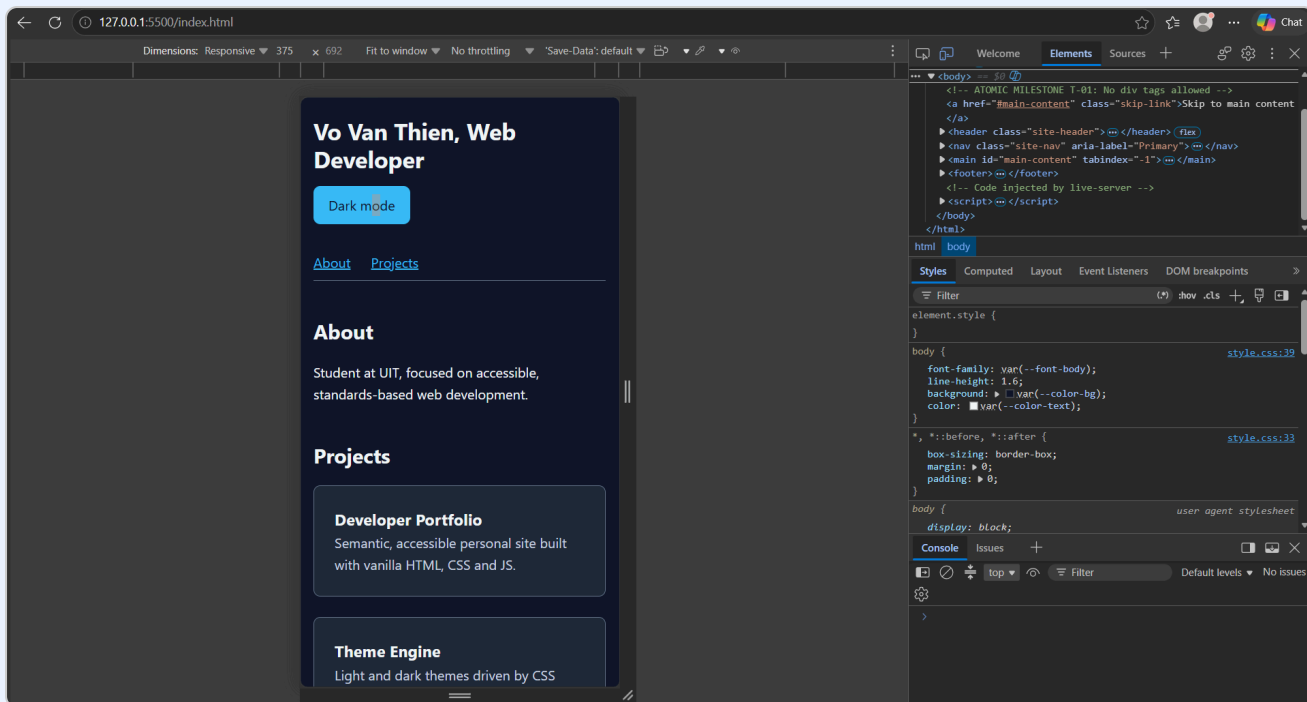
```
git --no-pager log --oneline --stat
```

Chụp kết quả có đủ các commit T-02, mỗi commit một file (**ảnh 10**). Hoặc thay bằng link https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1/commits/main trong report.

Mẹo khi chụp

- Dùng **Win + Shift + S**. Ảnh nào cũng nên thấy **thanh địa chỉ 127.0.0.1:5500**.
- Đặt tên ảnh theo số thứ tự, ví dụ `ex2-01-desktop-light.png`, để dễ sắp xếp trong report.
- Nếu một ảnh cho kết quả **không đạt** (ví dụ Console có lỗi đỏ, ra `true`, hay CLS khác 0), đừng nộp. Gửi mình xem để sửa trước.

Bạn



Tôi đang ở ảnh 6

Claude

Bạn đang chọn `<body>`. Thẻ này **không trực tiếp chứa chữ**, nên DevTools thường **không hiện tỉ lệ tương phản** cho nó. Cần chọn đúng **đoạn chữ** trong trang.

Cách nhanh nhất: dùng tooltip khi rê chuột

1. Bấm biểu tượng **chọn phần tử** ở góc trên bên trái DevTools (ô vuông có mũi tên, ngay trước biểu tượng điện thoại). Hoặc bấm **Ctrl + Shift + C**.
2. **Rê chuột (chưa bấm)** lên dòng *"Semantic, accessible personal site built..."* trong thẻ project.
3. Một **tooltip** hiện ra cạnh đoạn chữ, có dòng:

Contrast Aa 9.85 ✓

4. **Giữ chuột yên** và bấm **Win + Shift + S** để chụp. Tooltip sẽ nằm trong ảnh.

Làm thêm với **3 chỗ** để minh chứng đủ (theme tối đang bật):

Rê chuột lên	Token màu	Tỉ lệ mong đợi (tối)
Đoạn chữ trong thẻ project	--color-text-muted trên --color-surface	~9.85

Rê chuột lên	Token màu	Tỉ lệ mong đợi (tối)
Link About	--color-accent trên --color-bg	~8.33
Đoạn "Student at UIT..."	--color-text trên --color-bg	~16.3

Sau đó bấm nút để **chuyển sang theme sáng** và lặp lại. Tỉ lệ mong đợi khi sáng lần lượt khoảng **6.92, 5.93, 17.85**.

Cách khác: bảng chọn màu (nếu tooltip không hiện số)

- 1. Dùng công cụ chọn phần tử và **bấm** vào đoạn chữ trong thẻ project. Khung Elements sẽ nhảy tới thẻ <p>.
- 2. Trong **Styles**, tìm rule .project-card p { color: var(--color-text-muted); }.
- 3. Bấm vào **ô vuông màu nhỏ** trước var(--color-text-muted).
- 4. Bảng chọn màu hiện ra, có mục **Contrast ratio 9.85 ✓✓**. Chụp bảng đó.

Ghi chú

Ảnh của bạn cho thấy dòng <!-- Code injected by live-server --> cùng một thẻ <script> trong khung Elements. Đó là đoạn **Live Server tự chèn vào để tự tải lại trang, không nằm trong file index.html của bạn**. Nếu giảng viên hỏi, giải thích như vậy. Kiểm tra file gốc bằng git show HEAD:index.html sẽ không thấy đoạn này.

Bạn

Thứ tự ảnh này là thứ tự commit mà bạn chỉ tôi làm đúng k

Claude

Không. **Thứ tự ảnh và thứ tự commit là hai thứ khác nhau.**

	Thứ tự commit (C1 → C6)	Thứ tự ảnh (1 → 10)
Để làm gì	Ghi lại quá trình làm trong Git	Làm minh chứng trong report
Sắp theo	Thứ tự làm từng sub-task	Thứ tự các tiêu chí chấm của slide 13

	Thứ tự commit (C1 → C6)	Thứ tự ảnh (1 → 10)
Có cần commit ảnh không		Không. Ảnh dán vào report, không đưa vào Git

Ảnh được chụp **sau khi đã làm xong mọi commit**, để chứng minh sản phẩm cuối cùng đạt yêu cầu.

Nếu muốn report sắp theo sub-task

Bảng sau cho biết mỗi ảnh chứng minh **commit nào**, để bạn có thể xếp ảnh dưới từng sub-task trong report:

Sub-task / commit	Ảnh minh chứng
C2 feat(html): nút toggle, thẻ project	Ảnh 7 (nút toggle nhận focus bằng bàn phím)
C3 feat(css): tokens & reset	Ảnh 9 (hex chỉ trong token), Ảnh 6 (tương phản)
C4 feat(css): responsive grid	Ảnh 1 (lưới 3 cột desktop), Ảnh 3 (375px, 1 cột, không cuộn ngang)
C5 feat(js): dark mode engine	Ảnh 2 (theme tối), Ảnh 4 (localStorage theme), Ảnh 5 (Console sạch)
Toàn trang	Ảnh 8 (Lighthouse: CLS, LCP)
Kỷ luật commit	Ảnh 10 (git log) hoặc link GitHub

Bạn chọn cách nào cũng được:

- **Theo tiêu chí** (ảnh 1 → 10): người chấm dò đúng danh sách *Acceptance Criteria* của slide 13.
- **Theo sub-task** (bảng trên): cho thấy rõ từng commit đã đạt gì.

Kiểm tra lại thứ tự commit

```
git --no-pager log --oneline --stat
```

Phần Ex2 đúng sẽ là (mới nhất ở trên):

```
docs(spec): mark T-02 done          → TASK_DECOMPOSITION.md (C
feat(js): dark mode engine          → theme.js (C
feat(css): responsive grid          → style.css (C
feat(css): tokens & reset            → style.css (C
feat(html): add theme toggle, project cards... → index.html (C
docs(spec): define T-02 contracts    → TASK_DECOMPOSITION.md (C
```

Nếu thứ tự hoặc file trong từng commit khác thế này, dán kết quả lên đây để mình xem.

Bạn

Tiếp tục hướng dẫn tôi làm Ex3 nhé, mỗi lần commit là chỉ tôi Screen short minh chứng nha

Claude

Exercise 3 lớn hơn Ex2 nhiều, nên mình chia làm **3 đợt**. Đợt này gồm **toàn bộ kế hoạch, bản đặc tả, và 2 component đầu (Hero, Theme Switcher)**. Làm xong đợt này, bạn báo lại thì mình hướng dẫn tiếp Skills, Project Cards và Contact Form.

Exercise 3: Component Architecture & State Modeling

1. Đề yêu cầu gì (slide 14)

Component	Yêu cầu
Hero Section	Ảnh chân dung có kích thước khai báo rõ (width/height), tiêu đề, lời giới thiệu
Theme Switcher	Nút có aria-pressed và icon thay đổi theo trạng thái
Skills Matrix	Kỹ năng chia nhóm, dạng badge , xếp bằng CSS Grid
Project Cards	Mỗi thẻ là một <article> độc lập, có tag, link, mô tả (theo mẫu slide 14)
Contact Form	Form native có validate, quản lý trạng thái phía client

Slide 14 **không quy định commit message**, nhưng luật chung vẫn áp dụng: **mỗi commit một loại file**, không trộn HTML, CSS, JS (slide 7, 13, 20).

2. Đặt mã task

Slide 5 đã dùng T-01 đến T-05, còn slide 18 (Exercise 4) dùng **T-03A–C**. Để tránh trùng, mình đặt Ex3 là **T-06A → T-06E**, và ghi rõ lý do trong spec.

3. Kế hoạch commit

#	Commit	File	Đợt
C1	docs(spec): define Ex3 component contracts	TASK_DECOMPOSITION.md	1
C2	feat(html): hero section with portrait	index.html + assets/avatar.webp	1
C3	feat(css): hero layout	style.css	1
C4	feat(html): theme toggle icon hook	index.html	1
C5	feat(css): state-driven theme icon	style.css	1
C6–C7	Skills Matrix (HTML, rồi CSS)		2
C8–C9	Project Cards (HTML, rồi CSS)		2
C10–C12	Contact Form (HTML, CSS, rồi JS)		3
C13	docs(spec): mark Ex3 done		3

Commit C2 có **ảnh và HTML chung**. Ảnh là tài nguyên (asset), không phải CSS hay JS, nên đi chung với thẻ `<img>` dùng nó là hợp lý, và tránh được lỗi 404.

4. Quy trình chụp minh chứng sau MỖI commit

Mỗi commit chụp **2 loại ảnh**:

Ảnh A: bằng chứng commit sạch. Ngay sau khi commit, chạy:

```
git show --stat HEAD
```

Chụp kết quả: thấy commit message và **chỉ đúng file** của commit đó.

Ảnh B: bằng chứng chức năng. Mỗi commit có ảnh riêng, mình ghi ở từng bước.

Đặt tên ảnh theo commit, ví dụ ex3-C2-A-commit.png, ex3-C2-B-hero.png, để dễ xếp vào report.

ĐỢT 1

C1: Đặc tả

Thêm vào **cuối** TASK_DECOMPOSITION.md:

```
---

## Ex3 – Component Architecture & State Modeling (T-06)

> ID note: slide 18 (Exercise 4) dùng T-03A-C, nên Ex3 dùng T-06 để tránh

### Sub-tasks
| ID      | Component          | Commits (HTML → CSS → JS)
|-----|-----|-----
| T-06A   | Hero Section       | feat(html): hero section with portrait / feat
| T-06B   | Theme Switcher     | feat(html): theme toggle icon hook / feat(cs
| T-06C   | Skills Matrix      | feat(html): skills matrix / feat(css): skill
| T-06D   | Project Cards      | feat(html): project card components / feat(c
| T-06E   | Contact Form       | feat(html): contact form / feat(css): contac

### Component Contracts

**T-06A Hero**
- `
```

- Không sửa theme.js (state đã có sẵn trong `aria-pressed`)

T-06C Skills Matrix

- `section#skills` > `h2` + `ul.skills-grid` > `li` > `h3` (nhóm) + `ul.b
- Grid: `repeat(auto-fit, minmax(min(100%, 220px), 1fr))`

T-06D Project Cards

- `article.project-card[data-category]` > `header.card-header` (h3 + span + `p` + `ul.tag-list` + `footer.card-footer` > `a[aria-label]`
- Link mở repo thật, `aria-label` nêu rõ link dẫn tới đâu

T-06E Contact Form – State Model

- `form#contact-form[data-state="idle|submitting|success|error"]`
- Trường: name (required, minlength 3), email (type=email, required), message (required, minlength 10); mỗi trường có ``
- Thông báo: `p#form-status[role="status"]`, cập nhật bằng `textContent`
- Đang submitting → nút submit `disabled` (chống gửi 2 lần)
- Không có backend → gửi giả lập (không dữ liệu nào rời trình duyệt)

Acceptance Criteria

- [] Vẫn 0 <div>, 1 <h1>, heading không nhảy cấp
- [] Hero: CLS = 0, ảnh không tràn ở 375px
- [] Icon đổi theo theme; screen reader chỉ đọc "Dark mode" + trạng thái
- [] Skills & Projects: grid 1 cột ở 375px, nhiều cột ở desktop
- [] Form: báo lỗi khi bỏ trống/sai email; không gửi 2 lần; thông báo đư
- [] Tương phản $\geq 4.5:1$ ở cả 2 theme; không hex ngoài token
- [] Zero console errors; điều hướng hoàn toàn bằng bàn phím
- [] Mỗi commit chỉ một loại file (HTML / CSS / JS)

```
git add TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): define Ex3 component contracts"
```

📷 Minh chứng C1

- **Ảnh A:** git show --stat HEAD, chỉ có TASK_DECOMPOSITION.md.
- **Ảnh B:** mở TASK_DECOMPOSITION.md trong VS Code, chụp phần *Component Contracts*. Dùng **Ctrl + Shift + V** để xem dạng đã định dạng cho đẹp.

C2: Hero (HTML và ảnh)

Chuẩn bị ảnh chân dung

1. Chọn một ảnh chân dung của bạn, hoặc ảnh bạn có quyền sử dụng.
2. Mở **squoosh.app** trên Chrome. Trang này xử lý ảnh **ngay trên máy**, không tải ảnh lên server.
3. Kéo ảnh vào. Bên phải:
 - **Resize**: tích chọn, đặt **Width 400, Height 400**. Ảnh không vuông thì crop vuông trước bằng Paint hoặc Photos.
 - **Compress**: chọn **WebP**, Quality khoảng **75**.
4. Bấm **Download**. File nên **dưới 50 KB**.
5. Trong repo, tạo thư mục **assets**, đặt ảnh vào và đổi tên thành **avatar.webp**.

Sửa index.html

Thay **toàn bộ khối <header>** hiện tại bằng:

```
<header class="site-header">
  
  <h1>Vo Van Thien, Web Developer</h1>
  <p class="hero-pitch">I build fast, accessible web interfaces with speed</p>
  <button type="button" id="theme-toggle" class="theme-toggle" aria-label="Toggle theme" />
</header>
```

Vì sao viết như vậy (để vấn đáp):

Thuộc tính	Lý do
width="400" height="400"	Trình duyệt giữ chỗ trước khi ảnh tải xong, nên nội dung không bị đẩy xuống (CLS = 0 , slide 9)
Không loading="lazy"	Ảnh hero nằm ngay đầu trang, là phần tử LCP . Lazy-load sẽ làm nó tải chậm hơn . Slide 9 dùng lazy cho ảnh mẫu, nhưng chỉ đúng với ảnh nằm dưới màn hình đầu tiên
fetchpriority="high"	Bảo trình duyệt tải ảnh này trước tiên
alt mô tả	Trình đọc màn hình đọc được nội dung ảnh

Lưu ý: **chưa có CSS** nên ảnh sẽ hiện **to 400×400**. Điều đó đúng ở bước này.

```
git add index.html assets/avatar.webp
```

```
git diff --staged --stat
```

Chỉ có 2 file: index.html và assets/avatar.webp.

```
git diff --staged | Select-String -Pattern '<div|style=<style|onclick=<|1
```

Không in ra gì thì commit:

```
git commit -m "feat(html): hero section with portrait"
```

📷 Minh chứng C2

- **Ảnh A:** `git show --stat HEAD`, chỉ có index.html và assets/avatar.webp.
- **Ảnh B1:** trang trên Live Server, thấy ảnh chân dung (to, chưa style), tên, lời giới thiệu.
- **Ảnh B2:** F12, tab Elements, bấm chọn thẻ `<img>`. Chụp sao cho thấy `width="400" height="400"`, rồi **rê chuột** lên đường dẫn `assets/avatar.webp` trong thẻ để tooltip hiện **kích thước thật** của ảnh (400 × 400 px).
- **Ảnh B3:** tab **Network**, F5, lọc **Img**. Chụp dòng `avatar.webp` với **Status 200** và **dung lượng** (chứng minh ảnh nhẹ, không lỗi 404).

C3: Hero (CSS)

Bước 1: thêm vào phần **reset/base** của `style.css`, ngay sau rule `a { ... }`:

```
img {
  display: block;
  max-width: 100%;
  height: auto;
}
```

Bước 2: tìm khối `.site-header { ... }` ở phần **T-02B** (đang có `display: flex; ... justify-content: space-between`) và **thay toàn bộ khối đó** bằng:

```
.site-header {
  display: grid;
  gap: var(--space-sm);
  justify-items: start;
```

```
padding-block: var(--space-lg) var(--space-md);
}
```

Bước 3: thêm vào **cuối file**:

```
/* ===== Ex3 T-06A: Hero ===== */
.hero-portrait {
  width: 160px;
  aspect-ratio: 1;
  object-fit: cover;
  border-radius: 50%;
  border: 3px solid var(--color-accent);
}

.hero-pitch {
  max-width: 60ch;
  color: var(--color-text-muted);
}

@media (min-width: 48rem) {
  .site-header {
    grid-template-columns: auto 1fr;
    column-gap: var(--space-lg);
    align-items: center;
  }

  .hero-portrait {
    grid-row: span 3;
    width: 200px;
  }
}
```

Giải thích:

- **Mobile (mặc định):** ảnh, tên, lời giới thiệu và nút **xếp dọc**. Đây là cách làm mobile-first.
- **Từ 48rem (768px) trở lên:** chia 2 cột. Ảnh bên trái chiếm 3 hàng, chữ bên phải.
- `width: 160px` cùng `height: auto` (từ rule `img`) và **tỉ lệ lấy từ thuộc tính `width/height` trong HTML** giúp vẫn giữ chỗ đúng, nên vẫn **CLS = 0**.
- **Không thêm màu mới**, chỉ dùng token.

Kiểm tra: chạy lại lệnh tìm hex, vẫn phải ra **12 dòng**:

```
Select-String -Path style.css -Pattern '#[0-9a-fA-F]{3,8}\b'
```

```
git add style.css
```

```
git diff --staged --stat
```

```
git commit -m "feat(css): hero layout"
```

📸 Minh chứng C3

- **Ảnh A:** `git show --stat HEAD`, chỉ có `style.css`.
- **Ảnh B1:** desktop: ảnh tròn bên trái, chữ bên phải.
- **Ảnh B2:** 375px (Ctrl + Shift + M, gõ 375): xếp dọc, ảnh không tràn; Console gõ lệnh `scrollWidth > clientWidth` ra **false**.
- **Ảnh B3 (CLS):** tab **Lighthouse**, chọn Mobile, tích Performance, bấm Analyze. Chụp **Cumulative Layout Shift: 0** và **LCP**.

C4: Theme Switcher (HTML)

Sửa **chỉ dòng nút** trong `<header>` thành:

```
<button type="button" id="theme-toggle" class="theme-toggle" aria-pre
  <span class="theme-toggle__icon" aria-hidden="true"></span>
  Dark mode
</button>
```

- `<span>` là **chỗ để CSS vẽ icon**. Thở rộng, không chứa emoji trong HTML.
- `aria-hidden="true"` khiến trình đọc màn hình **bỏ qua icon**, chỉ đọc *"Dark mode, toggle button, pressed / not pressed"*.

```
git add index.html
```

```
git commit -m "feat(html): theme toggle icon hook"
```

📸 Minh chứng C4

- **Ảnh A:** `git show --stat HEAD`, chỉ có `index.html`.

- **Ảnh B:** F12, Elements, chọn `<button id="theme-toggle">`, mở tab **Accessibility** bên phải. Chụp mục **Computed properties: Name: "Dark mode", Role: button, Pressed: false** (hoặc true). Ảnh này chứng minh icon không làm sai tên đọc của nút.

C5: Theme Switcher (CSS)

Thêm vào **cuối** `style.css`:

```
/* ===== Ex3 T-06B: State-driven theme icon ===== */  
.theme-toggle {  
  display: inline-flex;  
  align-items: center;  
  gap: var(--space-xs);  
}  
  
.theme-toggle__icon::before {  
  content: "☀️";  
}  
  
.theme-toggle[aria-pressed="true"] .theme-toggle__icon::before {  
  content: "🌙";  
}
```

Điểm "State Modeling" của Ex3: icon **không do JavaScript đổi**. JS (từ Ex2) chỉ cập nhật **một trạng thái duy nhất** là `aria-pressed`, còn CSS tự **vẽ giao diện theo trạng thái đó**. Một nguồn sự thật, không có biến cờ thừa. Đây là câu trả lời tốt khi bị hỏi *"state được quản lý thế nào?"*

```
git add style.css
```

```
git commit -m "feat(css): state-driven theme icon"
```

📷 Minh chứng C5

- **Ảnh A:** `git show --stat HEAD`, chỉ có `style.css`.
- **Ảnh B1:** theme sáng, nút hiện ☀️ **Dark mode**.
- **Ảnh B2:** bấm nút, theme tối, nút hiện 🌙 **Dark mode**. Mở Elements cho thấy `aria-pressed="true"`.
- **Ảnh B3:** bấm toggle nhiều lần, Console **không có lỗi đỏ**.

Kết thúc Đợt 1

Kiểm tra:

```
git --no-pager log --oneline --stat -5
```

Phải thấy **5 commit** C1 → C5, mỗi commit đúng file như bảng ở mục 3. Dán kết quả lên đây, mình sẽ kiểm tra rồi hướng dẫn **Đợt 2** (Skills Matrix và Project Cards).

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git add index.html
assets/avatar.webp fatal: pathspec 'assets/avatar.webp' did not match any files
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git add index.html
assets/avatar.webp fatal: pathspec 'assets/avatar.webp' did not match any files
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1>
```

Claude

Lỗi giống lần làm với file PDF: **Git không tìm thấy assets/avatar.webp**. Vì một đường dẫn bị lỗi, **cả lệnh git add bị hủy**, nên index.html cũng **chưa được stage**.

Bước 1: Xem thư mục assets có gì

```
Get-ChildItem assets
```

Đọc kết quả:

Kết quả	Nguyên nhân	Cách sửa
Báo lỗi <i>Cannot find path ... assets</i>	Chưa tạo thư mục assets, hoặc tạo sai chỗ	Xem Trường hợp A
Thư mục trống	Ảnh chưa được copy vào	Xem Trường hợp A
Có file tên khác, ví dụ IMG_1234.webp, avatar.png	Squoosh tự đặt tên, hoặc ảnh chưa chuyển sang WebP	Xem Trường hợp B
Có avatar.webp.webp	Windows ẩn đuôi file nên bạn gõ thêm .webp khi đổi tên	Xem Trường hợp B

Trường hợp A: chưa có thư mục hoặc chưa có ảnh

```
New-Item -ItemType Directory -Force assets
```

Rồi copy file WebP tải từ Squoosh (thường nằm trong **Downloads**) vào assets. Cách dễ nhất là **kéo thả** file từ File Explorer vào thư mục assets trong khung Explorer của VS Code.

Trường hợp B: tên file sai

Đổi tên bằng lệnh, thay <ten-hien-tai> bằng đúng tên hiện ra ở bước 1:

```
Rename-Item "assets\<ten-hien-tai>" "avatar.webp"
```

Ví dụ: `Rename-Item "assets\avatar.webp.webp" "avatar.webp"`.

Nếu file là .png hoặc .jpg (chưa chuyển WebP): quay lại squoosh.app, chọn **WebP** ở mục Compress rồi tải lại. Hoặc dùng luôn file đó và **sửa src trong HTML** cho khớp, ví dụ `src="assets/avatar.jpg"`. Cách đó vẫn đạt yêu cầu; WebP chỉ là nhẹ hơn.

Bước 2: Kiểm tra lại rồi add

```
Get-ChildItem assets
```

Phải thấy đúng **avatar.webp**, cột *Length* khoảng vài chục nghìn byte.

Mở trang trên Live Server: **ảnh phải hiện**, không bị icon ảnh vỡ. Sau đó:

```
git add index.html assets/avatar.webp
```

```
git diff --staged --stat
```

Phải có **đúng 2 file**: `index.html` và `assets/avatar.webp`. Rồi tiếp tục bước kiểm tra `Select-String` và `commit` như hướng dẫn C2.

Nếu vẫn lỗi, dán kết quả `Get-ChildItem assets` lên đây.

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> Get-ChildItem assets
Get-ChildItem : Cannot find path 'C:\Users\USER\OneDrive\Tài
liệu\24521671_VoVanThien_LAB1\assets' because it does not exist. At line:1 char:1
```

- Get-ChildItem assets

- ```
+ CategoryInfo : ObjectNotFound: (C:\Users\USER\O...ien_LA
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Command
```

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1>
```

### Claude

Thư mục **assets** **chưa tồn tại**. Làm theo thứ tự sau.

## Bước 1: Tạo thư mục assets

```
New-Item -ItemType Directory -Force assets
```

Khung Explorer bên trái VS Code sẽ hiện thư mục assets (đang trống).

## Bước 2: Có file ảnh WebP chưa?

Tìm các file .webp mới nhất trong Downloads:

```
Get-ChildItem "$HOME\Downloads*.webp" | Sort-Object LastWriteTime -Desce
```

**Nếu có file** (Squoosh thường giữ tên gốc của ảnh, ví dụ IMG\_2024.webp): copy và đổi tên luôn, thay <ten-file> bằng tên vừa hiện ra:

```
Copy-Item "$HOME\Downloads\<ten-file>.webp" "assets\avatar.webp"
```

**Nếu không có file nào**, tức là bạn chưa tạo ảnh:

1. Mở **squoosh.app** trên Chrome và kéo ảnh chân dung vào.
2. Bên phải, mục **Resize**: bật lên, đặt **Width 400, Height 400** (ảnh không vuông thì crop vuông trước).
3. Mục **Compress**: chọn **WebP**, Quality **75**.

4. Bấm nút **Download** (biểu tượng mũi tên xuống) ở góc dưới bên phải. File được lưu vào Downloads.
5. Chạy lại lệnh tìm file ở trên, rồi lệnh Copy-Item.

**Cách thay thế:** kéo thả file từ File Explorer vào thư mục assets trong VS Code, rồi chuột phải chọn **Rename** và đổi thành avatar.webp.

## Bước 3: Kiểm tra

```
Get-ChildItem assets
```

Phải thấy **avatar.webp**. Mở trang trên Live Server: ảnh phải hiện (ảnh to 400×400 là đúng, vì chưa có CSS).

## Bước 4: Add và commit

```
git add index.html assets/avatar.webp
```

```
git diff --staged --stat
```

Đúng 2 file thì commit:

```
git commit -m "feat(html): hero section with portrait"
```

Sau đó chụp minh chứng C2 (ảnh A và B1–B3) như hướng dẫn trước.

**Bạn**

hướng dẫn kĩ cách thay thế

**Claude**

Dưới đây là cách **kéo thả bằng chuột**, không cần gõ lệnh copy.

## Bước 0: Bật hiển thị đuôi file (chỉ làm một lần)

Windows mặc định **ẩn đuôi file**, nên bạn dễ đặt nhầm thành avatar.webp.webp. Bật lên trước:

1. Mở **File Explorer** (Win + E).
2. Trên thanh trên cùng, bấm **View** (Xem), chọn **Show** (Hiện), rồi tích **File name extensions** (Phần mở rộng tên tệp).

Từ giờ tên file sẽ hiện đầy đủ, ví dụ IMG\_2024.webp.

## Bước 1: Mở file ảnh trong File Explorer

1. Trong File Explorer, vào **Downloads** (hoặc nơi bạn lưu ảnh từ Squoosh).
2. Tìm file ảnh. Nhìn đuôi file:
  - **.webp**: đúng định dạng, làm tiếp.
  - **.jpg, .jpeg, .png**: vẫn dùng được, xem **lưu ý ở Bước 5**.

## Bước 2: Đặt hai cửa sổ cạnh nhau

1. Mở **VS Code** với thư mục dự án 24521671\_VoVanThien\_LAB1.
2. Đảm bảo khung **Explorer** bên trái VS Code đang hiện (Ctrl + Shift + E), và thấy thư mục **assets** (đã tạo ở bước trước).
3. Đặt File Explorer và VS Code **cạnh nhau**: chọn cửa sổ File Explorer rồi bấm **Win + ←**, sau đó chọn VS Code ở nửa còn lại.

## Bước 3: Kéo thả ảnh vào assets

1. Trong File Explorer, **nhấn giữ chuột trái** lên file ảnh.
2. **Kéo** sang khung Explorer của VS Code, thả **đúng lên chữ assets**. Thư mục assets sẽ được tô sáng khi chuột nằm đúng chỗ.
3. **Thả chuột**.
4. Nếu VS Code hỏi **"Copy"** hay **"Move"**, chọn **Copy** để giữ bản gốc trong Downloads.

Bấm mũi tên > cạnh assets để mở ra. File ảnh phải nằm **bên trong** assets, hiện màu xanh lá kèm chữ **U**.

**Nếu lỡ thả ra ngoài** (file nằm ngang hàng với index.html): kéo file đó trong VS Code vào thư mục assets.

## Bước 4: Đổi tên thành avatar.webp

1. Trong VS Code, **chuột phải** vào file ảnh trong assets và chọn **Rename** (hoặc chọn file rồi bấm **F2**).
2. Xóa tên cũ, gõ **avatar.webp** rồi Enter.

Kết quả trong Explorer:

```
assets
├── avatar.webp U
index.html
...
```

## Bước 5: Nếu ảnh là .jpg hoặc .png

**Cách 1 (nên làm):** chuyển sang WebP bằng **squoosh.app** (Resize 400×400, Compress WebP, Download), rồi làm lại từ Bước 1. Ảnh WebP nhẹ hơn nhiều, tốt cho LCP.

**Cách 2 (nhANH):** giữ nguyên định dạng nhưng **đặt tên khớp đuôi thật**, ví dụ `avatar.jpg`, rồi **sửa HTML** cho khớp:

```

```

Lệnh `git add` sau đó cũng dùng tên `assets/avatar.jpg`. Ảnh phải **vuông** thì `width="400" height="400"` mới đúng tỉ lệ. Nếu không vuông, crop trước bằng **Photos** (mở ảnh, bấm **Edit** hoặc `Ctrl + E`, chọn **Crop**, chọn tỉ lệ **Square**, rồi **Save as copy**).

## Bước 6: Kiểm tra

1. Mở trang trên Live Server: **ảnh chân dung phải hiện** (to, chưa style là đúng). Nếu thấy icon ảnh vỡ thì tên file hoặc `src` chưa khớp.
2. Trong Terminal:

```
Get-ChildItem assets
```

Phải thấy `avatar.webp` (hoặc `avatar.jpg` nếu dùng Cách 2).

## Bước 7: Add và commit

```
git add index.html assets/avatar.webp
```

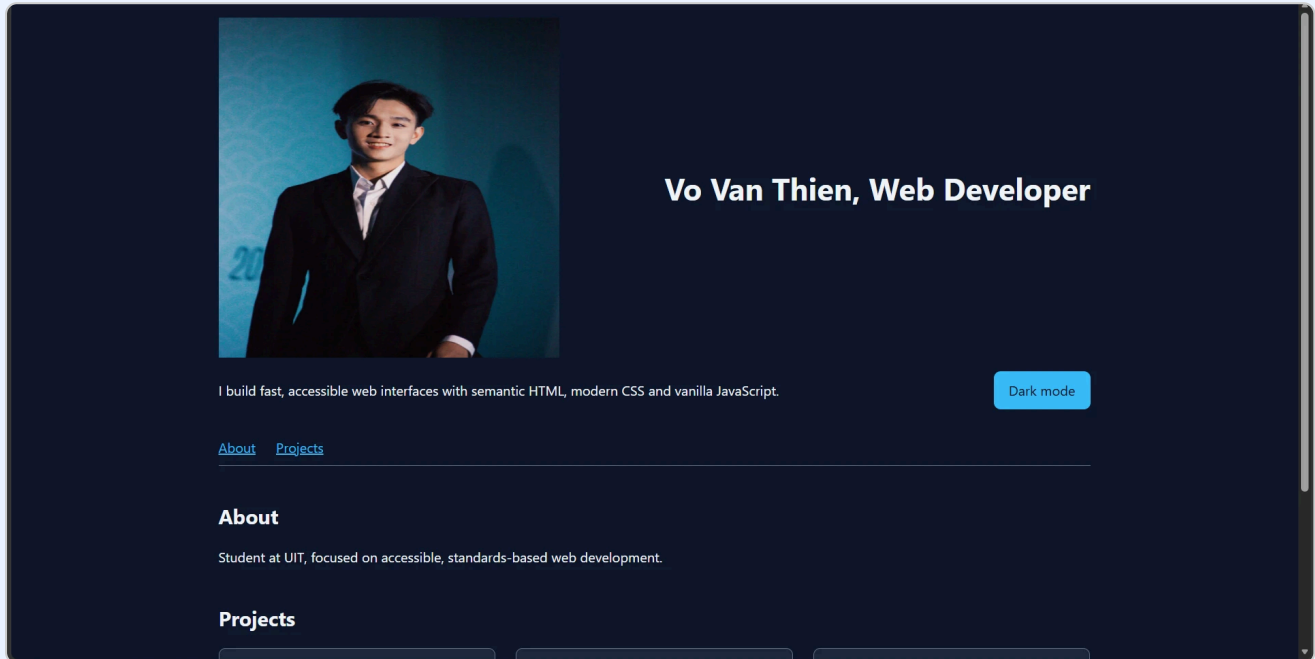
```
git diff --staged --stat
```

Đúng 2 file thì commit:

```
git commit -m "feat(html): hero section with portrait"
```

Nếu ảnh vẫn không hiện hoặc lệnh báo lỗi, chụp màn hình VS Code (thấy khung Explorer và Terminal) gửi mình.

### Bạn



### Claude

Ảnh đã hiện ☒ Bố cục lúc này trông lạ (ảnh to, tên lệch sang phải, nút "Dark mode" nằm cạnh lời giới thiệu) là **đúng như dự kiến ở C2**. Header vẫn đang dùng `display: flex; justify-content: space-between` từ Ex2, nên các phần tử bị dàn ngang. **C3 sẽ sửa** thành bố cục hero: ảnh tròn bên trái, chữ bên phải.

Ảnh có tỉ lệ **vuông**, khớp với `width="400" height="400"`.

## Commit C2

```
git add index.html assets/avatar.webp
```

```
git diff --staged --stat
```

Phải có đúng 2 file: `index.html` và `assets/avatar.webp`.

```
git diff --staged | Select-String -Pattern '<div|style=|<style|onclick=|1'
```

Không in ra gì thì commit:

```
git commit -m "feat(html): hero section with portrait"
```

## Minh chứng C2

- **Ảnh A:** `git show --stat HEAD`, thấy đúng 2 file.
- **Ảnh B1:** chụp lại trang giống ảnh vừa gửi (thấy ảnh chân dung, tên, lời giới thiệu). Nhớ chụp kèm **thanh địa chỉ** `127.0.0.1:5500`.
- **Ảnh B2:** F12, Elements, chọn thẻ `<img>`. Chụp cho thấy `width="400" height="400" fetchpriority="high"` và **không có** `loading="lazy"`.
- **Ảnh B3:** tab **Network**, F5, bấm bộ lọc **Img**. Chụp dòng `avatar.webp` với **Status 200** và cột **Size**.

**Lưu ý:** repo của bạn đang **Public**, nên ảnh chân dung sẽ công khai trên GitHub. Nếu bạn không muốn vậy, có thể dùng ảnh khác (avatar minh họa chẳng hạn) **trước khi commit**. Sau khi push, ảnh sẽ còn trong lịch sử Git.

## Tiếp theo: C3

Làm đúng **3 bước** đã hướng dẫn trong `style.css`:

1. Thêm rule `img { display: block; max-width: 100%; height: auto; }` ngay sau `a { ... }`.
2. **Thay** khối `.site-header { display: flex; ... }` (ở phần T-02B) bằng khối `display: grid` mới.
3. Thêm phần `/* ===== Ex3 T-06A: Hero ===== */` vào **cuối file**.

Sau C3, trang desktop sẽ có **ảnh tròn 200px bên trái**, còn tên, lời giới thiệu và nút xếp dọc bên phải. Chụp gửi mình kiểm tra trước khi commit nếu muốn.

Bạn

tôi đã xong c5, tiếp tục nhé

Claude

Bắt đầu **ĐỢT 2: Skills Matrix và Project Cards** (C6 → C9).

Trước khi làm, kiểm tra nhanh 5 commit vừa xong:

```
git --no-pager log --oneline --stat -5
```

Từ trên xuống phải là: state-driven theme icon (style.css), theme toggle icon hook (index.html), hero layout (style.css), hero section with portrait (index.html + assets/avatar.webp), define Ex3 component contracts (TASK\_DECOMPOSITION.md). Nếu khác, dán lên đây trước khi làm tiếp.

## ĐỢT 2

### C6: Skills Matrix (HTML)

#### 1. Thêm link vào nav

Trong `<ul class="nav-links">`, thêm dòng **Skills** vào giữa About và Projects:

```
About
Skills
Projects
```

#### 2. Thêm section Skills

Dán khối sau **giữa** `</section>` của About và `<section id="projects" ...>`:

```
<section id="skills" aria-labelledby="skills-title">
 <h2 id="skills-title">Skills</h2>
 <ul class="skills-grid">
 <li class="skill-group">
 <h3>Frontend</h3>
 <ul class="badge-list">
 <li class="badge">Semantic HTML5
 <li class="badge">CSS Grid & Flexbox
 <li class="badge">JavaScript ES6+

 <li class="skill-group">
 <h3>Accessibility</h3>
```

```

 <ul class="badge-list">
 <li class="badge">WCAG 2.2 AA
 <li class="badge">ARIA
 <li class="badge">Keyboard navigation

 <li class="skill-group">
 <h3>Tooling</h3>
 <ul class="badge-list">
 <li class="badge">Git
 <li class="badge">VS Code
 <li class="badge">Chrome DevTools

 </section>

```

### Cấu trúc để vấn đáp:

- **Danh sách lồng danh sách:** danh sách các nhóm, mỗi nhóm lại là danh sách kỹ năng. Trình đọc màn hình sẽ báo "*list, 3 items*", rồi "*list, 3 items*" bên trong. Không cần <div>.
- <h3> là tên nhóm, nằm dưới <h2>Skills, nên không nhảy cấp heading.
- &amp; là cách viết ký tự & trong HTML (slide 8, HTML entities).

Bạn có thể đổi tên kỹ năng cho đúng với bản thân.

```
git add index.html
```

```
git diff --staged | Select-String -Pattern '<div|style=|<style|onclick='
```

```
git commit -m "feat(html): skills matrix"
```

### 📸 Minh chứng C6

- **Ảnh A:** git show --stat HEAD, chỉ có index.html.
- **Ảnh B1:** trang có mục **Skills** (chưa style nên các badge xếp dọc như danh sách, đúng ở bước này).
- **Ảnh B2:** bấm link **Skills** trên nav. URL đổi thành #skills và trang cuộn tới đúng mục đó.
- **Ảnh B3:** F12, Elements, bật cây Accessibility (nút hình người). Chụp thấy **region "Skills"** nằm trong main, giữa region "About" và region "Projects".

## C7: Skills Matrix (CSS)

Thêm vào **cuối** style.css:

```
/* ===== Ex3 T-06C: Skills matrix ===== */
.skills-grid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(min(100%, 220px), 1fr));
 gap: var(--space-md);
 list-style: none;
}

.skill-group {
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: var(--space-md);
}

.skill-group h3 {
 margin-block-end: var(--space-xs);
}

.badge-list {
 display: flex;
 flex-wrap: wrap;
 gap: var(--space-xs);
 list-style: none;
}

.badge {
 padding: 0.125rem var(--space-xs);
 border: 1px solid var(--color-accent);
 border-radius: 999px;
 font-size: 0.875rem;
 color: var(--color-text);
}
```

**Giải thích:**

- **Grid 2D** cho các nhóm (.skills-grid): 3 cột trên desktop, 1 cột ở 375px, không cần media query.
- **Flexbox 1D** cho badge trong nhóm (.badge-list): xếp hàng ngang, hết chỗ thì xuống dòng (flex-wrap). Đúng cặp **Grid cho 2D, Flexbox cho 1D** ở slide 16 và 17.
- .badge được viết thành **component dùng lại**: C8 sẽ dùng chính class này trong Project Cards.
- Không có màu mới. Chữ badge dùng --color-text trên --color-surface, tỉ lệ **16.3:1** (sáng) và **13.35:1** (tối).

```
Select-String -Path style.css -Pattern '#[0-9a-fA-F]{3,8}\b'
```

Vẫn **12 dòng**.

```
git add style.css
```

```
git commit -m "feat(css): skills grid & badges"
```

### 📷 Minh chứng C7

- **Ảnh A:** git show --stat HEAD, chỉ có style.css.
- **Ảnh B1:** desktop: **3 khối kỹ năng nằm ngang**, badge bo tròn.
- **Ảnh B2:** 375px: khối xếp **1 cột**, badge tự xuống dòng; Console scrollWidth > clientWidth ra **false**.
- **Ảnh B3:** theme tối, phần Skills vẫn rõ chữ.
- **Ảnh B4:** dùng **Ctrl + Shift + C**, rê chuột lên một badge. Chụp tooltip **Contrast ≥ 4.5** ✓.

## C8: Project Cards (HTML)

Thay toàn bộ nội dung bên trong <ul class="project-grid"> ... </ul> bằng:

```

 <article class="project-card" data-category="frontend">
 <header class="card-header">
 <h3>Developer Portfolio</h3>
 Frontend
 </header>
 <p>Semantic, accessible personal site built with vanilla HTML</p>
 <ul class="tag-list" aria-label="Technologies">
 HTML5
```

```
 CSS Grid
 Accessibility

 <footer class="card-footer">

 </footer>
</article>

 <article class="project-card" data-category="ui-state">
 <header class="card-header">
 <h3>Theme Engine</h3>
 UI State
 </header>
 <p>Light and dark themes driven by CSS custom properties and
 <ul class="tag-list" aria-label="Technologies">
 CSS Variables
 localStorage
 aria-pressed

 <footer class="card-footer">

 </footer>
 </article>

 <article class="project-card" data-category="interactive">
 <header class="card-header">
 <h3>Drum Kit</h3>
 Interactive
 </header>
 <p>Keyboard-driven audio pads using data attributes and the k
 <ul class="tag-list" aria-label="Technologies">
 Web Audio
 keydown
 data-* attributes

 <footer class="card-footer">
 <a href="https://github.com/vanthien1462006-dotcom" aria-la
 </footer>
```

```
</article>

```

### Điểm quan trọng để vấn đáp:

Chi tiết	Ý nghĩa
<code>&lt;article&gt;</code>	Mỗi thẻ là một <b>khối nội dung độc lập</b> , tách ra chỗ khác vẫn có nghĩa
<code>&lt;header&gt;</code> / <code>&lt;footer&gt;</code> <b>bên trong</b> <code>&lt;article&gt;</code>	<b>Không</b> trở thành landmark banner/contentinfo của trang, vì chúng chỉ là đầu và cuối của article. Trang vẫn chỉ có <b>1 banner và 1 contentinfo</b>
<code>data-category</code>	Dữ liệu gắn vào HTML để JS hoặc CSS dùng sau (ví dụ lọc theo loại). Đúng tinh thần <b>"data in HTML, logic in JS"</b> của slide 23
<code>aria-label</code> trên link	Nhiều link cùng chữ "Source code" sẽ khó phân biệt khi trình đọc màn hình liệt kê link. <code>aria-label</code> nói rõ link dẫn tới đâu, và <b>vẫn chứa chữ hiển thị "source code"</b> (đúng WCAG 2.5.3 <i>Label in Name</i> )
Link Drum Kit trở tới <b>GitHub profile</b>	Dự án Drum Kit (HW2) chưa có repo. Tránh dùng <code>href="#"</code> , vì link giả gây khó chịu cho người dùng bàn phím

```
git add index.html
```

```
git diff --staged | Select-String -Pattern '<div|style=|<style|onclick='
```

```
git commit -m "feat(html): project card components"
```

### 📷 Minh chứng C8

- **Ảnh A:** `git show --stat HEAD`, chỉ có `index.html`.
- **Ảnh B1:** phần Projects có đủ tiêu đề, badge, mô tả, danh sách tag và link (chưa style nên chưa đẹp, đúng ở bước này).
- **Ảnh B2 (quan trọng):** cây Accessibility. Chụp thấy vẫn **chỉ một banner** (ở đầu trang) và **một contentinfo** (ở cuối). Các article **không** có banner hay contentinfo bên trong.
- **Ảnh B3:** chọn một link "Source code" trong Elements, mở tab **Accessibility**. Chụp Name: **"View Developer Portfolio source code on GitHub"**.

## C9: Project Cards (CSS)

Thêm vào **cuối** style.css:

```
/* ===== Ex3 T-06D: Project cards ===== */
.project-card {
 display: flex;
 flex-direction: column;
 gap: var(--space-sm);
}

.card-header {
 display: flex;
 flex-wrap: wrap;
 justify-content: space-between;
 align-items: center;
 gap: var(--space-xs);
}

.tag-list {
 display: flex;
 flex-wrap: wrap;
 gap: var(--space-xs) var(--space-sm);
 list-style: none;
 font-size: 0.875rem;
 color: var(--color-text-muted);
}

.tag-list li::before {
 content: "#";
}

.card-footer {
 margin-block-start: auto;
}
```

### Giải thích:

- .project-card đã có nền, viền, bo góc và height: 100% từ Ex2. Ở đây **chỉ thêm** bố cục dọc bằng Flexbox.

- `margin-block-start`: auto trên footer **đẩy link xuống đáy thẻ**, nên 3 thẻ cao bằng nhau có link thẳng hàng, dù mô tả dài ngắn khác nhau.
- `content`: "#" chỉ là ký tự trang trí trước mỗi tag, **không phải mã màu**. Lệnh tìm hex sẽ không bắt nhầm nó.

```
Select-String -Path style.css -Pattern '#[0-9a-fA-F]{3,8}\b'
```

Vẫn 12 dòng.

```
git add style.css
```

```
git commit -m "feat(css): project card styles"
```

### Minh chứng C9

- **Ảnh A**: `git show --stat HEAD`, chỉ có `style.css`.
- **Ảnh B1**: desktop: 3 thẻ ngang hàng, badge ở góc phải tiêu đề, tag có dấu #, **link thẳng hàng ở đáy**.
- **Ảnh B2**: 375px: thẻ xếp 1 cột, không tràn; Console ra **false**.
- **Ảnh B3**: theme tối, các thẻ vẫn rõ chữ.
- **Ảnh B4**: dùng **Tab** để focus vào một link "Source code". Chụp **viền focus** quanh link.

## Kết thúc Đợt 2

```
git --no-pager log --oneline --stat -4
```

Phải thấy 4 commit C6 → C9, xen kẽ **HTML** → **CSS** → **HTML** → **CSS**, mỗi commit đúng 1 file. Dán kết quả lên đây, mình kiểm tra rồi hướng dẫn **Đợt 3: Contact Form** (HTML, CSS, JS state machine), phần quan trọng nhất về *State Modeling*.

**Bạn**

hướng dẫn tôi cập nhật lại file AI chatlog